# AIP-C01 study and practice

Reader: A candidate who needs a first-principles route through the official exam scope.

Read the concepts. Predict the result. Explain the mechanism. Then answer a new scenario without notes.

This book covers all 98 published skills. AWS states that its exam guide is not a comprehensive list of exam content. No book guarantees a pass.

## Use the two-day route

1. Read the prerequisite primer. Explain each term in your own words.
2. Read Chapters 1 to 12. Trace each request from input to result.
3. Read Chapters 13 to 24. Compare each control with its failure case.
4. Answer the mastery questions before you read their answers.
5. Take the timed mock in the HTML course. Explain each rejected choice.
6. Review every weak skill. Use the separate practice booklet for more scenarios.

The route is a study recommendation, not an AWS readiness threshold. Do not replace sleep with a complete reread of the source archive.

## Exam values

AIP-C01 means AWS Certified Generative AI Developer, Professional. AIF-C01 is a different exam.

The exam contains 75 questions in 180 minutes. There are 65 scored questions and 10 unidentified unscored questions.

The scaled score ranges from 100 to 1000. The minimum passing score is 750. This is not a 75% raw score rule.

Domain weights are 31%, 26%, 20%, 12%, and 11%. Select all required responses for a multiple-response question.

AWS expects production AWS experience and hands-on generative AI experience. Proposed lab procedures in this book are not executed lab evidence.

Sources [1].

## Print procedure

1. Open the file whose name contains Two-Up.
2. Select landscape US Letter and actual size.
3. Select one PDF page per printed side. The file already contains two logical pages.
4. Select duplex and flip on the short edge.
5. Print a test sheet before the full job.



Each sheet carries four logical pages with duplex print. The separate practice booklet lets you avoid repeat print of the core book.

# Prerequisite primer

## API requests, responses, and JSON

An API is a contract between a caller and a service.

A client sends a request to a service endpoint. The request names an operation and carries data. The service checks the request, performs an operation, and returns a response.

JSON is a text format for structured data. It uses named fields, values, arrays, and nested objects. A model request can use JSON to carry a model identifier, messages, and settings.

A successful network response confirms that the service handled the request. It does not prove that the generated answer is correct. The caller must inspect the response, error state, and task result.

Sources [2, 3, 4].

## IAM identities, roles, policies, and explicit deny

An identity names who or what makes a request. A policy states which actions that identity may take.

An IAM role is an identity that a trusted user or service can assume. A workload can use temporary role credentials to call AWS services.

A policy is a JSON permission document. It can allow or deny actions on resources under conditions. An explicit deny blocks an action, even when another policy allows it.

Use separate roles for workloads with different duties. Grant only the actions and resources that each role needs. Model instructions do not replace IAM authorization.

Sources [5, 6, 7].

## Regions, Availability Zones, VPCs, and endpoints

A Region is a geographic AWS area. A VPC is a virtual network in one Region.

An Availability Zone is an isolated location within a Region. A VPC can use subnets in multiple Availability Zones. This lets a design place resources across separate locations in one Region.

An endpoint gives a workload a path to a service. An interface VPC endpoint uses AWS PrivateLink to connect a VPC privately to supported AWS services.

Private connectivity does not grant permission by itself. IAM and endpoint policies still control allowed requests. Cross-Region inference can route work across Regions, so check the data-location rules before use.

Sources [8, 9, 5].

## S3 objects and encryption keys

Amazon S3 stores objects in buckets. Encryption protects data at rest and in transit under configured service controls.

An S3 object contains data and metadata. A bucket stores objects under keys. Applications use the bucket and key to identify a source file or result file.

A knowledge base can read source files and metadata from storage, then create derived chunks and embeddings. The source object remains distinct from its search index.

AWS Key Management Service manages cryptographic keys used by supported encryption features. Access to encrypted data can depend on both storage permission and permission to use the key. Protect keys and source data with scoped access.

Sources [10, 6, 11].

## Lambda, containers, queues, and workflows

Compute runs code. A queue stores work for later processing. A workflow controls a sequence of steps.

AWS Lambda runs a function when an event or request invokes it. A container packages code and its runtime dependencies. A service such as Amazon ECS can run container workloads.

A queue such as Amazon SQS holds messages until a consumer processes them. This separates the producer from the worker and supports asynchronous work.

A workflow service coordinates steps, decisions, retries, and state. Use it when the application needs an explicit sequence or branch. Choose a compute pattern from task duration, runtime needs, and operations.

Sources [12, 7, 13, 14].

## Training, inference, and embeddings

Training changes or creates model parameters. Inference uses a model to produce an output. Embedding converts content to a vector representation.

A foundation model learns patterns during training. Customization can use additional data to improve behavior for a task. Training and customization require controlled data and evaluation.

Inference is a model call at application time. The request supplies a prompt or messages. The model returns generated content. Inference does not automatically update model parameters.

An embedding model maps text or other supported content into a vector. A retrieval system compares query and document vectors to find candidates. Similarity is a search signal, not proof that a passage is true.

Sources [15, 2, 16].



# Chapter 1. Token and context

The model receives a bounded sequence of tokens, not an entire application or database.

### The smallest unit

A token is a unit in the model vocabulary. It can represent a word, word part, punctuation mark, or other text unit. Token counts can differ from word counts. The model receives a serialized request that may include system instructions, conversation messages, tool definitions, and retrieved text. The model does not see a user interface or external file unless the application supplies its contents.

### How the mechanism works

The context window is the limit on the request and generated output that a model can process in one inference. The usable space must account for instructions, history, tool schemas, retrieved passages, and requested output. A larger context can hold more evidence, but it does not guarantee that the model will use every detail well. Long prompts can add latency and cost. They can also bury relevant evidence among unrelated text.

### Where it fits

Treat context as a designed input budget. Keep stable rules concise. Add only the history and evidence needed for the current decision. If a request exceeds practical limits, summarize with explicit references or retrieve focused passages. Preserve source identifiers so a later answer can cite the right material.

### Failure and control

The application can measure token use, but it should not assume that one token equals one word. Different models can tokenize the same text differently. Test the chosen model and request shape.

### Causal worked trace

Causal trace: A user asks about a policy exception. The application starts with a short system rule and current message. It adds two retrieved clauses and their source identifiers. The tokenizer maps this serialized request into model tokens. The service checks the request against the selected model context limit. The model processes the supplied sequence and returns text. If the application adds the full archive, input tokens rise before generation starts. The request can then exceed the limit or crowd out the decisive clause.

### Failure diagnosis

Failure diagnosis: A context-length error means the serialized request is too large for that model and request path. Count system text, conversation history, tool schemas, retrieved passages, and output allowance. If the request fits but the answer misses a clause, inspect placement and relevance. Do not diagnose every miss as a token limit problem.

### Transfer case and misconception

Transfer case: A support assistant has a long chat history and one new question. Retain recent turns that resolve references such as “that order.” Summarize older turns with the order identifier and unresolved issue. Retrieve the current policy clause separately. Keep the summary and evidence distinct so the model can tell memory from authority.

Misconception correction: A large context window does not make every included fact equally accessible or authoritative. The application still selects and orders the evidence. Token capacity is a limit, not a relevance score.

### Design implications

When you design memory, state which facts remain durable and which facts expire after one turn. Keep source evidence outside the conversational summary when the source must support a citation. A compact summary should preserve names, dates, decisions, and unresolved references only when the task needs them. Compare the summary against the original history before release. If it changes a condition, the compression step has damaged meaning. Keep an explicit path to retrieve the original record when the summary cannot answer a follow-up.

### Invariant

The serialized request must fit the selected model limit, and the model can use only content that the application includes.

### Controlled experiment

1. Hold the model, question, and output limit constant.

2. Set the history window to the last two turns, then run the same test question.

3. Repeat with the full history and record token use, answer accuracy, and latency.

4. Compare whether the answer needs an older turn or only the current evidence.

The full history uses more input tokens. It may help when the question refers to earlier details, but it may add noise when those details do not matter.

If the short history preserves accuracy, trim irrelevant turns. If it loses a required reference, retain a compact summary or the needed turns.

### Transfer check

Predict what happens when a prompt includes a full policy archive instead of retrieved clauses.



The request may exceed a model limit or include distractors. Retrieve clauses for the question and retain their source identifiers.

## Inspect context use in a conversation

Compare input and output usage for a short and a long conversation.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Prepare two non-sensitive prompts with the same question and different history lengths.

Reason. Isolate context size as the changed variable.

Evidence. Two saved request bodies differ only in history.

2. Send each request with the Converse API using the same model and inference settings.

Reason. Keep model behavior comparable across trials.

Evidence. Both calls return response content and usage data.

3. Record input and output token usage from each response.

Reason. Usage data shows the processed request size.

Evidence. The long-history request reports its input usage.

4. Compare answer quality against the same expected answer.

Reason. More context can add irrelevant material.

Evidence. A reviewer records omissions, distractions, or preserved references.

5. Remove history that does not resolve a current reference, then repeat the call.

Reason. Focused context tests whether old content was necessary.

Evidence. The revised request retains required details.

6. Review the model context limit and leave room for requested output.

Reason. Input and generation share a bounded request capacity.

Evidence. The request fits the selected model contract.

### Expected evidence

1. The requests differ only in history length.

2. Response usage is recorded for both trials.

3. The quality review identifies whether earlier turns matter.

### Failure checks

1. Check that the same model and settings were used.

2. Check that output allowance was not changed between trials.

3. Check the expected answer against the supplied context.

### Cleanup

1. Delete local test prompts if policy requires it.
2. Do not retain sensitive conversation content in logs.

### Exam transfer

For a long support session, retain only turns needed to resolve the current request. Verify references before trimming history.

Sources [3, 17, 16].

Sources [17, 3, 16].

# Chapter 2. Model selection

Choose a model by task quality, modality, latency, cost, and access constraints.

### The smallest unit

A foundation model learns broad patterns from training data. An application sends it an inference request and receives generated output. The model does not become a database because it can produce fluent text. It can make unsupported claims, so the application must define what counts as a correct result.

### How the mechanism works

Model choice starts with the task. A classification task may need a short, stable answer. A complex synthesis task may need stronger reasoning. A multimodal task needs support for the input and output modalities in use. A model can differ in supported parameters, context limits, tool use, Regions, and availability. Confirm those facts in current service documentation before design.

### Where it fits

Compare candidates on a representative evaluation set. Measure answer quality, refusal behavior, latency, cost, and operational fit. Keep the model choice separate from the application interface where possible. Amazon Bedrock provides access to multiple model families through service APIs, but a shared API does not make model behavior identical.

### Failure and control

Check model access and account permissions before a deployment plan depends on an identifier. Model access can require account or provider steps. Avoid choosing by benchmark alone. A benchmark cannot represent every company prompt, data source, or failure cost.



### Causal worked trace

Causal trace: A team defines a set of support questions and expected answers. It runs each candidate model with the same prompt and retrieval evidence. Each model applies its learned parameters to produce an answer. The evaluator compares correctness, refusal behavior, latency, and usage. One candidate may answer faster but omit a critical condition. The task owner weighs that measured defect against the latency gain before selecting a model.

### Failure diagnosis

Failure diagnosis: A fluent answer with a false policy claim is a quality failure, even when the API call succeeds. Check the same prompt and evidence on another candidate. If both fail, inspect task framing and evidence. If one fails, inspect model capability and model-specific request settings. Confirm current access and Region support before attributing the difference to model quality.

### Transfer case and misconception

Transfer case: A document assistant handles both short extraction and multi-step comparison. Define separate routes only when evaluation shows different needs. Keep a shared response contract, then test each route against the same safety and citation rules. Do not route by model reputation alone.

Misconception correction: A shared service catalog does not imply identical model behavior, context limits, modalities, or parameters. Treat each candidate as a separate system under evaluation.

### Design implications

Write acceptance criteria before comparing candidates. Criteria can include a minimum answer quality, allowed refusal behavior, required modality, and an operational latency target. Values must come from the workload owner, not from an assumed universal threshold. Keep the test prompts stable across candidates. Record model identifiers and request settings, because otherwise a later comparison may use different configurations. Re-run the candidates after material model or prompt changes. A selection is a decision for a workload and its controls, not a permanent ranking of all models.

### Invariant

A model is suitable only when it meets the task quality threshold and the deployment constraints for the intended workload.

### Controlled experiment

1. Select one representative evaluation set and one prompt version.
2. Run the set on two models that are available to the account.
3. Record correctness, refusal behavior, latency, and input and output usage for each run.
4. Compare each model against the task owner’s acceptance criteria.

The candidates can differ in both output quality and operational measures, even with the same request structure.

Choose the candidate that meets quality and policy needs at acceptable operational cost. If neither passes, revise the task or evidence before deployment.

**Transfer check**

Predict the effect of choosing the fastest model for a task with costly factual errors.

Low latency can reduce user wait time, but weak task accuracy can raise review and correction cost. Compare models against the task evaluation set.

## Compare model candidates

Use one representative task set to compare available model capabilities and behavior.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. List available foundation models with the Bedrock ListFoundationModels API.

Reason. Availability and model identity constrain candidate choice.

Evidence. The candidate identifiers and modalities are recorded.

2. Check model access and availability for each candidate.

Reason. A listed model may not be ready for this account or Region.

Evidence. Access and availability checks return the candidate state.

3. Select candidates that meet the required modality and task.

Reason. Remove candidates that cannot accept the intended input.

Evidence. A short list maps each candidate to task needs.

4. Send the same test prompts through Converse with matching settings.

Reason. A fixed request allows a fair comparison.

Evidence. Each candidate returns output and usage data.

5. Score correctness, format, refusal behavior, and latency by case.

Reason. Averages can hide a critical task failure.

Evidence. A results table shows each measure by test case.

6. Choose only a candidate that meets the stated acceptance criteria.

Reason. The workload requirement determines suitability.

Evidence. A written choice links evidence to each criterion.

**Expected evidence**

1. Candidates have confirmed access and task support.

2. Each candidate receives the same prompt set.

3. Selection evidence covers quality and operational needs.

**Failure checks**

1. Check for request settings that differ by model.

2. Check access errors before scoring quality.

3. Check critical failure cases separately from overall averages.

**Cleanup**

1. Remove test data from temporary locations under local policy.

2. Do not delete shared model access or account settings.

**Exam transfer**

When a task needs image input, reject candidates without that modality before comparing answer quality.

Sources [18, 19, 17].

Sources [18, 19, 17].

# Chapter 3. Inference APIs

An inference API turns an application request into model input and output.

**The smallest unit**

An API request gives the service a model identifier and a body with messages or a prompt. The request can include generation settings and optional controls. The response can contain generated content, stop information, and usage details. The exact schema depends on the selected API and model.

**How the mechanism works**

Amazon Bedrock offers model-specific invocation patterns and the Converse API for a common message interface across supported models. Converse standardizes message structure and selected controls. Some model-specific fields still require additional model request fields. A common interface can simplify application code, but it does not erase model-specific capabilities or response differences.

**Where it fits**

Separate request construction from business logic. Validate required fields before the call. Check response status and content shape. Record request identifiers and safe usage data for diagnosis. Avoid assuming that a successful HTTP response means the generated answer meets the application requirement.

**Failure and control**

The inference call is one step in a larger flow. The application still owns authentication, authorization, input validation, output handling, timeouts, retries, and user-facing error behavior.

### Causal worked trace

Causal trace: The application creates a Converse request with a model identifier, messages, and inference configuration. The service validates the model route and body. The model receives the messages and produces a response. The application reads the returned content and stop information. If the application sends a model-specific field in the wrong location, validation can fail before generation. A shared message shape simplifies this path but does not remove model-specific fields.

### Failure diagnosis

Failure diagnosis: First separate request rejection from poor generated content. For rejection, check required fields, supported inference settings, model identifier, and access. For malformed application output, inspect the response parser and content blocks. For a valid but unsuitable answer, inspect prompt, evidence, and model behavior. Do not treat an HTTP success as task success.

### Transfer case and misconception

Transfer case: An application migrates from one provider-specific request shape to Converse. Map roles and content to the shared message structure. Keep provider-specific parameters behind a model adapter. Verify both models with the same end-to-end contract, including empty responses and stop conditions.

Misconception correction: A common API reduces integration variation. It does not normalize every capability, parameter, or answer style across models.

### Design implications

Keep an adapter layer around request serialization when the application supports different model families. The adapter should map the application contract to the chosen API without hiding unsupported behavior. Validate the serialized request in a test environment before sending sensitive content. Parse the response into an internal result type that records text, stop state, usage, and error details. Preserve unknown response fields only when the application has a defined reason. This makes API changes easier to locate without treating every response as plain text.

### Invariant

The application must validate both the request contract before inference and the response contract after inference.

### Controlled experiment

1. Keep the prompt and model fixed.

2. Remove one optional inference parameter from the Converse request.

3. Submit the request and record validation outcome, response structure, and generated content.



4. Restore the parameter and compare its effect with the documented model support.

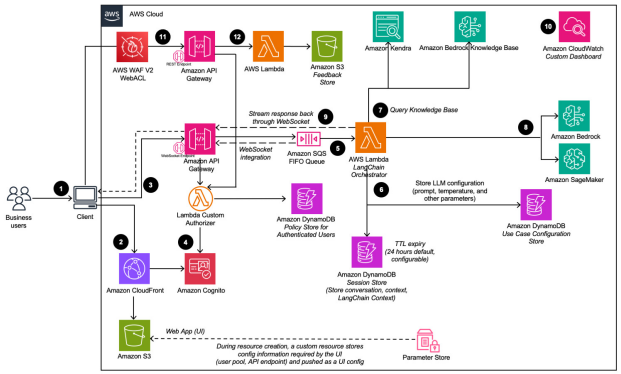
A supported optional parameter may change generation. An unsupported or misplaced parameter may cause request validation failure.

Retain only documented settings that improve the target behavior. Handle validation errors as request defects, not transient service errors.

### Transfer check

Predict what a shared message API will simplify when an application supports two models.

It can standardize message construction and response handling. Model-specific settings and behavior still need explicit treatment.




Application Builder text use case

Trace the text request across application and model services.

1. Follow the text request through the WebSocket, queue, orchestrator, and model path.

2. Locate the optional knowledge base and response path.

This architecture belongs to the named solution. It does not show all text application choices.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html

## Validate a Converse request

Trace request construction, inference response, and error handling.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Build a minimal Converse request with a model identifier and one user message.

Reason. A small request isolates the API contract.

Evidence. The serialized request contains required message fields.

2. Validate the model identifier and required fields against the API reference.

Reason. Model support and request syntax can differ.

Evidence. The request passes local schema review.

3. Invoke the Converse API with non-sensitive test content.

Reason. The service validates and processes the request.

Evidence. The call returns a response or a classified error.

4. Inspect response content, stop reason, and usage fields.

Reason. The application needs more than answer text.

Evidence. The parser records output and completion state.

5. Send one deliberately invalid test field in an isolated request.

Reason. A controlled error tests validation handling.

Evidence. The client classifies the returned validation error.

6. Correct the request and repeat the call.

Reason. A valid request should pass the same integration path.

Evidence. The corrected call returns expected content.

### Expected evidence

1. The valid request passes and returns a parseable response.
2. The invalid request produces a handled error.
3. The application preserves completion and usage state.

### Failure checks

1. Check model access and identifier before request changes.
2. Check message shape and supported optional fields.
3. Do not retry a deterministic validation error.

### Cleanup

1. Remove local request and response artifacts if they contain sensitive data.
2. Leave shared model access unchanged.

### Exam transfer

For a second model, keep the application contract stable and test its model-specific fields separately.

Sources [3, 18, 4].

Sources [3, 18, 4].

# Chapter 4. Retrieval augmented generation

RAG supplies relevant external evidence to a model at answer time.

### The smallest unit

A model has learned patterns from training. It does not automatically know current private company records. RAG connects a question to a data retrieval step. The system then places selected evidence in the model request. The answer can use both the question and the supplied evidence.



### How the mechanism works

A basic RAG path has two phases. Ingestion prepares documents for search. Query processing retrieves candidate passages and forms a prompt. Generation uses the prompt and passages to produce an answer. Retrieval and generation have separate quality limits. A good model cannot cite evidence that retrieval missed. Good retrieval cannot force a model to follow every instruction.

### Where it fits

RAG is useful when facts change, belong to an organization, or need source traceability. It is not a guarantee of correctness. The system must control document access, filter irrelevant passages, preserve source metadata, and state what to do when evidence is missing.

### Failure and control

Evaluate retrieval and answer quality separately. A correct answer can occur by chance when evidence is weak. A retrieved passage can be relevant while the final answer misstates it.

### Causal worked trace

Causal trace: A policy document enters the knowledge base ingestion path. The system parses it, forms chunks, creates embeddings, and indexes them with source metadata. A user asks a policy question. Retrieval returns candidate passages. The application adds those passages to the model request. The model generates an answer from the question and supplied evidence. If retrieval omits the exception clause, generation cannot reliably ground the answer in that clause.

### Failure diagnosis

Failure diagnosis: Inspect retrieval results before changing the model. If the right passage is absent, test query wording, metadata filters, chunk boundaries, and indexing state. If the passage is present but ignored, inspect context order, prompt instructions, and answer evaluation. If the source itself is wrong or stale, repair ingestion and sync first.

### Transfer case and misconception

Transfer case: A finance assistant must answer from current approved guidance. Keep authoritative source identifiers in retrieved context. Require a clear “insufficient evidence” response when retrieval returns no supporting passage. Evaluate whether the answer cites the passage that supports each claim.

Misconception correction: RAG reduces reliance on model memory, but it does not guarantee truth. Retrieval relevance, source authority, and answer faithfulness remain separate checks.

### Design implications

RAG is a pipeline, so assign a quality measure to each transition. The source must parse into usable content. The index must contain expected records. Retrieval must return evidence for the query. Generation must use that evidence. A citation must point to the supporting source. If teams measure only final answer accuracy, they may not know which transition caused a failure. Keep the stages visible during development, then retain enough safe production telemetry to identify regressions without exposing unnecessary content.

### Invariant

A RAG answer can be grounded only in evidence that retrieval supplies and the model uses correctly.

### Controlled experiment

1. Choose a question with a known supporting document and record the retrieved passages.

2. Set the retrieval result limit to a lower value, then repeat the question.

3. Compare passage coverage, answer correctness, citations, and input tokens.

4. Restore the previous limit and test a query with one relevant exception.

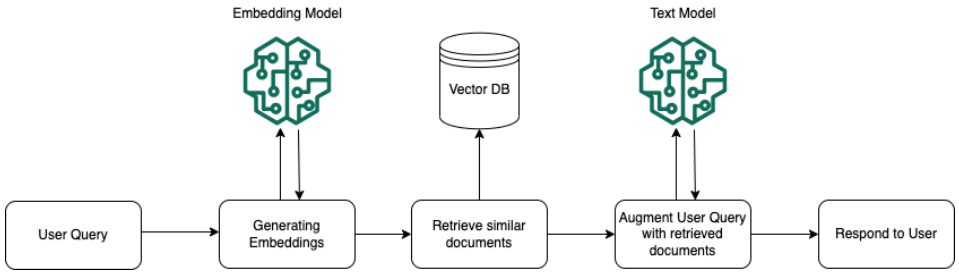
A lower result limit can reduce noise and context use, but it can also omit needed evidence.

Set the result limit from measured evidence coverage and answer quality. Do not select it from answer fluency alone.

### Transfer check

Predict the result when a relevant policy exists but the query retrieves unrelated passages.

The model may answer from general patterns or unrelated context. Improve retrieval, then test whether the answer uses the correct evidence.




Knowledge base retrieval and generation

Trace the query through embedding, retrieval, prompt augmentation, and generation.

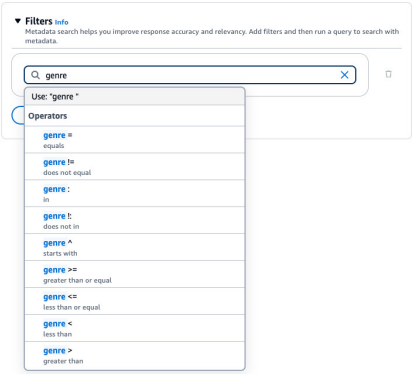
1. Follow the query through embedding, retrieval, prompt augmentation, and generation.

2. Separate retrieval work from answer generation.

The diagram shows a basic RAG runtime path. It does not show evaluation or guardrail steps.



https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-it-works.html




Add a knowledge base metadata filter

Use the operator list to check a metadata expression before you run a knowledge base query.

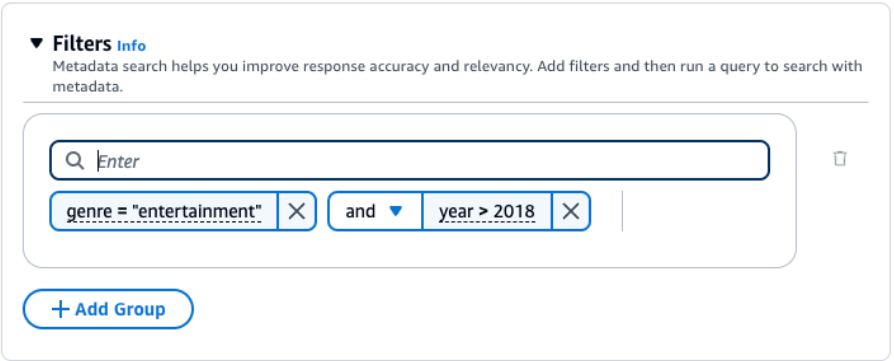
1. Find the Filters section under the knowledge base test configuration.

2. The filter field contains genre.

3. The operator list includes equals, does not equal, in, starts with, and numeric comparisons.

This is a static AWS documentation screenshot. Console labels can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html




Combine metadata filters in one group

Use multiple expressions in one group when every condition must match. The source says a group supports up to five filters.

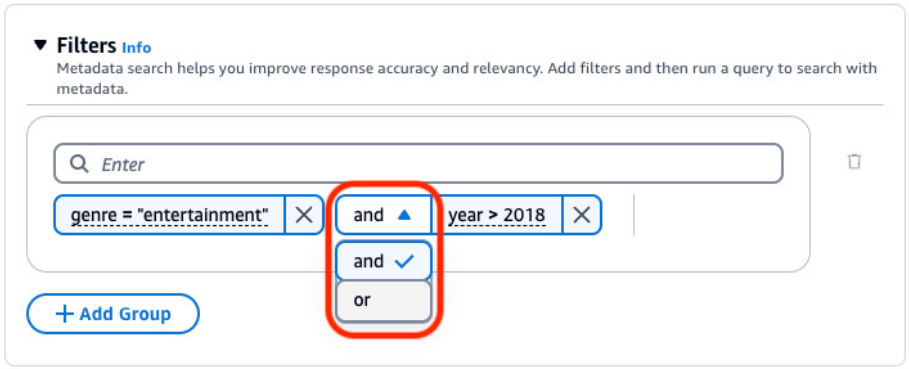
1. The first expression is genre = "entertainment".

2. The second expression is year > 2018.

3. The and control joins both expressions. The screen also shows + Add Group.

This is a static AWS documentation screenshot. Console labels and filter limits can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html



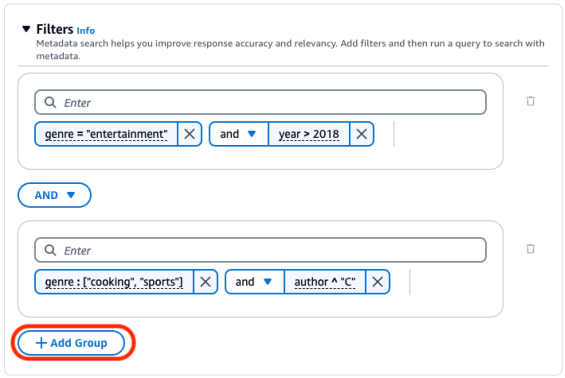

Change the filter operator

Check the connector before you run the query. It changes the Boolean meaning of the filters.

1. The connector menu is between two filter expressions.
2. The menu shows and and or.
3. The selected connector controls whether all or at least one expression must match.

This is a static AWS documentation screenshot. Console labels can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html




Add a second filter group

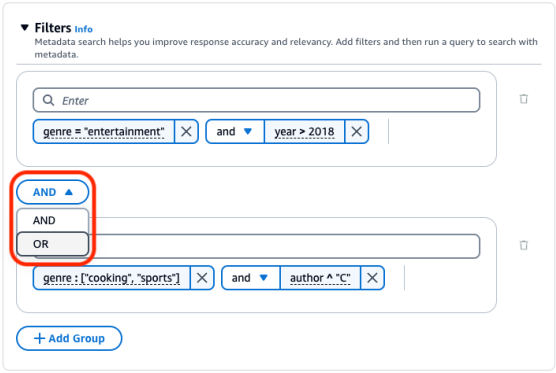
Use a separate group when a query needs nested Boolean conditions. The source says a configuration supports up to five groups.

1. The first group contains genre and year expressions.
2. The second group contains genre and author expressions.
3. The + Add Group control creates another group.

This is a static AWS documentation screenshot. Console labels and filter limits can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html





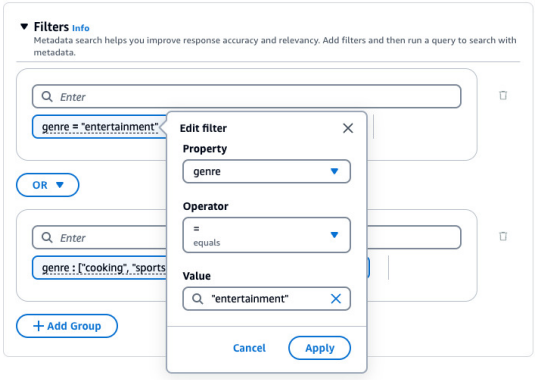

Choose how filter groups combine

Read the outer operator and each inner operator to verify the full filter logic.

1. The outer connector sits between two filter groups.
2. The menu shows AND and OR.
3. The group connector is separate from the connectors inside each group.

This is a static AWS documentation screenshot. Console labels can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html



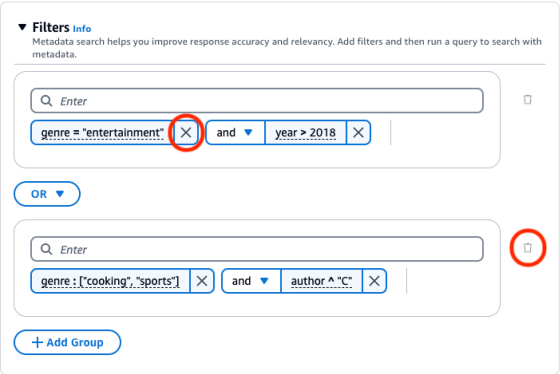

Edit a metadata filter

Check all three filter fields before you apply a change.

1. The dialog title is Edit filter.
2. The fields are Property, Operator, and Value.
3. The dialog has Cancel and Apply controls.

This is a static AWS documentation screenshot. Console labels can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html



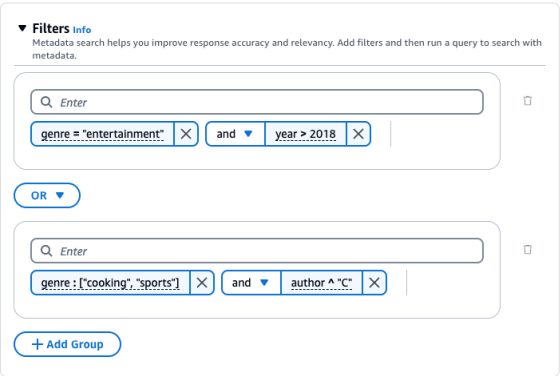

Remove a filter or group

Confirm which control is beside the target before you remove a filter or group.

1. The x control sits beside a single filter.
2. The trash control sits beside a filter group.
3. The source text assigns these controls to different removal actions.

This is a static AWS documentation screenshot. Console labels and icons can change. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html




Read a nested filter example

Translate the displayed groups into a Boolean expression before you run the query.

1. The first group requires genre = "entertainment" and year > 2018.
2. The second group matches genre cooking or sports and author starts with C.
3. The outer operator is OR.

This is a static AWS documentation screenshot. The values are documentation examples. A metadata filter does not replace authorization.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html

## Trace a RAG answer to evidence

Separate retrieval quality from answer quality with a known source passage.



Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Choose a question and identify its authoritative source passage.

Reason. Known evidence supports a meaningful retrieval check.

Evidence. The expected passage and answer criteria are recorded.

2. Call the Retrieve API with the knowledge base identifier and question.

Reason. Retrieve exposes candidate evidence before generation.

Evidence. The response includes source references and retrieved text.

3. Check whether returned passages cover every required fact.

Reason. Evidence coverage is separate from fluent answer quality.

Evidence. A reviewer marks relevant, missing, or distracting passages.

4. Call RetrieveAndGenerate with the same question if enabled for the test.

Reason. This tests the combined retrieval and generation path.

Evidence. The response includes an answer and available citations.

5. Compare each answer claim with its cited source.

Reason. Citation presence alone does not prove support.

Evidence. Each claim maps to a supporting passage or a noted gap.

6. Record retrieval and answer defects as separate findings.

Reason. Separate defects point to different repairs.

Evidence. The findings name retrieval, source, or generation stages.

### Expected evidence

1. The retrieved evidence is reviewed before generation.
2. The final response is checked against its citations.
3. Stage-specific defects are documented.

### Failure checks

1. Check knowledge base sync status and source identity.
2. Check retrieval filters and returned passage relevance.
3. Check whether generation added claims absent from evidence.

### Cleanup

1. Keep the source knowledge base intact.
2. Remove only local notes and temporary request data.

### Exam transfer

If evidence is missing, fix retrieval or ingestion before changing answer style.

Sources [20, 16, 21].

Sources [20, 16, 21].

# Chapter 5. Data ingestion

Ingestion converts source data into material that a retrieval system can index.

### The smallest unit

An ingestion path reads data from a source such as object storage. It parses supported content, extracts text or structure, and prepares records for downstream processing. The result needs a stable relationship to the original source. Without that relationship, an answer cannot reliably point back to its origin.

### How the mechanism works

A knowledge base sync can detect changed or deleted source data and update its indexed representation. The source remains the system of record. The vector index is a derived search structure. A successful sync does not prove that every parsed field has the intended meaning. Inspect parsing behavior for tables, images, metadata, and unusual formats.

### Where it fits

Metadata can support filtering and source attribution. It can also enforce access boundaries only when the system applies trusted identity and filter logic correctly. Do not accept user-controlled metadata as an authorization decision. Decide how to process deletions, updates, duplicate files, and failed records.

### Failure and control

Track ingestion status and source versions. When answers look stale, compare source change time, sync outcome, indexed metadata, and query results. A stale index is a data pipeline defect, not necessarily a model defect.

### Causal worked trace

Causal trace: A source object changes in the data store. A knowledge base sync detects the source change and processes the content. Parsing extracts text and available metadata. Chunking and embedding create index records linked to the source. A query then retrieves a current passage. If sync failed or skipped the changed object, the index can still return old content although the source object is correct.

### Failure diagnosis

Failure diagnosis: Compare source version and sync status first. Check parser output and metadata next. Then inspect indexed content and query results. This sequence separates source defects from ingestion defects and retrieval defects. Check deletion handling when a removed source still appears. Do not use a model prompt change to mask stale indexed data.



### Transfer case and misconception

Transfer case: A team publishes a revised procedure and expects answers to change. Record the object version, run the supported sync, inspect status, and retrieve the revised clause directly. Confirm that the previous clause no longer appears where the source was replaced. Preserve source identifiers for audit.

Misconception correction: A source update does not instantly prove that every derived index reflects the update. A successful job status also does not prove the parser captured every intended field.

### Design implications

Treat source storage, parsing, and indexing as separate states. A file can exist in storage while its parsed text is empty or malformed. A parser can produce usable text while a sync has not yet placed it in the index. Metadata can be present but carry an incorrect classification. Make ingestion checks match the content type and source owner. For high-value records, verify a few expected fields after each pipeline change. Keep a process for removals because stale deleted content can remain discoverable until the derived index updates.

### Invariant

The retrieval result can reflect only the source state that ingestion has parsed and indexed.

### Controlled experiment

1. Choose one test document with a visible revision marker.
2. Change the document text while retaining its source identity.
3. Run the knowledge base sync and inspect its status and retrieved text.
4. Compare the retrieved marker with the source marker.

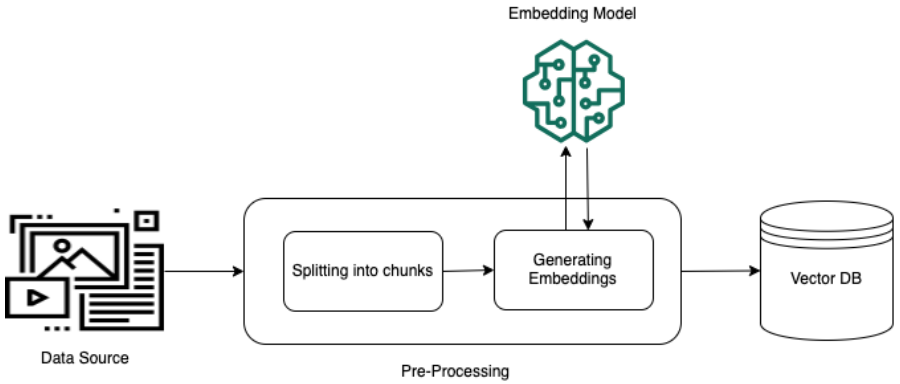
The indexed answer material changes only after the sync path processes the revision successfully.

If the marker remains old, inspect sync status and parsed output before changing retrieval or prompt settings.

### Transfer check

Predict why a corrected document may not change an answer immediately.

The retrieval index may not yet reflect the update, or retrieval may still select an older record. Check sync status and indexed results.




Knowledge base data preprocessing

Trace source data through chunking, embedding, and vector storage.

1. Follow source data through chunking, embedding, and vector storage.
2. Check where the image stops and the source page gives configuration detail.

The diagram gives a high-level ingestion path. It does not compare chunking strategies.

https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-it-works.html

## Verify ingestion freshness

Trace a safe source update through sync and retrieval.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Record the test document source identifier and visible revision marker.

Reason. The marker distinguishes source state from indexed state.

Evidence. The original marker and source reference are noted.

2. Update the test document with a new revision marker.

Reason. A controlled edit creates a known change.

Evidence. The source object shows the new marker.

3. Start the test sync with StartIngestionJob for its knowledge base and data source.

Reason. Sync updates the derived retrieval representation.

Evidence. The sync job reports a completion or failure state.

4. Track the returned job with GetIngestionJob until it reaches a terminal state.

Reason. A successful job may still skip or fail one record.

Evidence. The record status confirms processing.

5. Retrieve the document passage using the Retrieve API.

Reason. A query checks the user-visible index state.



Evidence. Retrieved text contains the new marker.

6. Check whether obsolete text still appears in results.

Reason. Replacement and deletion behavior affect answer freshness.

Evidence. The old marker is absent or its persistence is explained.

### Expected evidence

1. The source and index revision markers agree.
2. The sync result is recorded for the test source.
3. Retrieval confirms the updated content.

### Failure checks

1. Check sync errors and parser output if the marker is absent.
2. Check that the query targets the revised document.
3. Check for duplicate source records.

### Cleanup

1. Restore or remove the test document according to the sandbox plan.
2. Do not remove shared knowledge base resources.

### Exam transfer

A user reports stale answers. Compare source revision, sync state, index result, then generation.

Sources [10, 20, 6].

Sources [10, 20, 6].

# Chapter 6. Chunk design

A chunk is a retrievable segment that balances local meaning and search precision.

### The smallest unit

A long document must often be split before indexing. A chunk is a piece of that document, with metadata that links it to the source. If a chunk is too large, it can contain several topics and dilute matching. If it is too small, it can lose definitions or conditions that explain a statement.

### How the mechanism works

Fixed-size chunking uses size and overlap rules. Semantic chunking uses content boundaries or meaning changes. Hierarchical chunking keeps smaller child passages associated with larger parent sections. These choices affect what the retriever can find and what the model sees. There is no universally best chunk size. Test the source formats and questions that matter.

### Where it fits

Preserve headings, page or section references, and document identifiers when available. Keep related conditions together. Avoid splitting a rule from its exception. For tables, retain labels with values. Chunking that looks tidy in a text preview can still break the meaning of a record.

### Failure and control

Tune chunking with retrieval metrics and answer review. Use representative questions that target definitions, exceptions, tables, and cross-section facts. Change one chunking choice at a time so the experiment can explain a result.

### Causal worked trace

Causal trace: A source section contains a rule followed by an exception. Fixed-size chunking cuts text at configured token boundaries. An embedding represents each chunk. Retrieval matches the rule chunk but not the separate exception chunk. The model receives only the rule. A parent-child strategy can retrieve a small child and return broader parent context, while a semantic boundary can keep a coherent passage together.

### Failure diagnosis

Failure diagnosis: When an answer omits a condition, inspect whether the rule and condition occupy different chunks. Check parser output, chunk previews, and retrieval results. Test tables separately because parsed table or page context can affect returned content. If the required fact is absent from parsed text, adjust parsing before chunk settings.

### Transfer case and misconception

Transfer case: A compliance document has headings, tables, and exceptions. Preserve headings with clauses. Keep table labels alongside values. Compare fixed-size, semantic, or hierarchical chunking on queries that require a condition and its exception. Consider source behavior for figures and tables before designing citations.

Misconception correction: Smaller chunks are not always more precise for the final answer. A small match can lack the surrounding context needed to interpret it.

### Design implications

Choose chunk boundaries by the questions readers ask, not by a universal size rule. A definition question may work with a short chunk. A question about exceptions may need a broader passage. A table question may need its heading and row labels. Source parsing can alter what the chunker receives, so inspect parsed input before tuning boundaries. Keep source location metadata through each transformation. This lets a reviewer move from a retrieved passage to the original page or section and check whether context was lost.



### Invariant

A chunk must retain enough local context to preserve the meaning of the fact that its embedding retrieves.

### Controlled experiment

1. Select a document where a rule and exception appear near a likely chunk boundary.

2. Set the fixed-size chunk maximum to a symbolic value C, then set it to a larger value C2 that keeps both clauses together.

3. Re-ingest and query the same rule question.

4. Compare retrieved text and exception coverage.

The larger boundary can return both clauses, while it may also add unrelated text.

Keep the boundary that preserves required conditions with acceptable retrieval precision. Use a parent context strategy when child matching needs broader explanation.

### Transfer check

Predict how tiny chunks affect a question whose answer depends on a rule and its exception.

Search may find the rule without its exception. Use a chunk boundary or parent retrieval method that keeps both conditions available.

## Compare chunk boundaries

Observe how one chunking choice changes returned context.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Inspect parsed source text and mark the rule and exception.

Reason. Chunking operates on parsed content.

Evidence. Both statements and their source location are identified.

2. Create or use a test data source with fixed-size chunking.

Reason. A baseline makes the boundary effect visible.

Evidence. The configured chunking strategy is recorded.

3. Sync the document and inspect available chunk previews.

Reason. Preview reveals whether the two statements stay together.

Evidence. The chunk boundary is documented.

4. Retrieve the rule question and save returned passages.

Reason. The query tests whether both conditions surface.

Evidence. The response shows the matched chunk and source.

5. Repeat with a larger configured chunk size or hierarchical strategy.

Reason. A single controlled configuration tests broader context.

Evidence. The revised chunking setting is recorded.

6. Compare exception coverage and irrelevant context.

Reason. Larger context can preserve qualifications and add noise.

Evidence. A reviewer compares both retrieval outputs.

### Expected evidence

1. Parsed text preserves the rule and exception.

2. The baseline and revised settings are known.

3. Retrieval comparison records both coverage and noise.

### Failure checks

1. Check parser output before tuning chunk size.

2. Check whether returned table or page context differs from the matched chunk.

3. Check result count when parent chunks replace children.

### Cleanup

1. Use a disposable data source for re-ingestion.

2. Remove only test documents and temporary evaluation notes.

### Exam transfer

For a table question, preserve labels and values together before adjusting model prompts.

Sources [22, 16, 20].

Sources [22, 16, 20].

# Chapter 7. Embeddings

An embedding maps content to a numeric representation for similarity search.

### The smallest unit

An embedding model converts text into a vector. The vector represents patterns learned by that model. Similar meanings can map to nearby points, which supports semantic search. The vector is not a summary and does not preserve every fact in readable form.

### How the mechanism works

A retrieval system embeds documents during ingestion and embeds a query during search. The index compares vectors using a configured similarity measure. The document and query representations must be compatible. A model change can require a coordinated re-embedding of indexed content.



### Where it fits

Embeddings can retrieve conceptually related passages when query words differ from source words. They can also miss exact identifiers, rare terms, or domain-specific meanings. Include metadata and lexical search where the task needs exact matching. Assess results on the actual query set, not on intuition about vector distance.

### Failure and control

Embedding quality depends on source quality, chunk design, model choice, and search configuration. A vector store cannot repair ambiguous source text. A strong embedding model cannot enforce access control by itself.

### Causal worked trace

Causal trace: Ingestion text is sent to an embedding model, which maps it to a vector. The vector store indexes that vector with metadata. At query time, the same compatible embedding path maps the question to a vector. Similarity search returns nearby records. If the document and query vectors come from incompatible embedding models, the distance comparison may not represent useful semantic similarity.

### Failure diagnosis

Failure diagnosis: Verify the embedding model and dimensions for indexed records and live queries. Check whether a model change triggered re-embedding. Then inspect source text, chunk quality, and query formulation. If exact identifiers fail, test lexical search rather than assuming embedding quality is the only cause.

### Transfer case and misconception

Transfer case: A team upgrades its embedding model to improve domain retrieval. Create a replacement index or a controlled re-embedding plan. Compare both indices on the same labeled questions. Switch query embedding and index together, then retain a rollback path until evaluation passes.

Misconception correction: An embedding is not a human-readable summary or a truth score. It supports a similarity operation under a compatible representation and search configuration.

### Design implications

Embedding migrations affect the full indexed corpus, not only the query code. Plan how to create the new representation, compare it with the existing index, and switch traffic. Avoid mixing vectors from incompatible models in one search space unless the service explicitly supports that design. Track the embedding model with each index version. Keep a rollback route to the old model and index during evaluation. Include exact identifiers and domain terms in the evaluation set because semantic similarity can behave differently on those queries.

### Invariant

Document and query vectors must use compatible embedding representations for similarity results to carry their intended meaning.

### Controlled experiment

1. Record the embedding model used for the current index and query path.

2. Change the query embedding configuration to a different available model without re-indexing.

3. Run the same labeled questions and record retrieved source identifiers.

4. Restore compatibility, then compare results.

The incompatible query vectors can produce degraded or unstable retrieval because index vectors remain in the prior representation.

Deploy an embedding change as a coordinated index and query migration. Keep the old pair available during evaluation.

### Transfer check

Predict what can happen if documents use one embedding model and queries use an incompatible model.

Similarity scores can become unreliable because both sides no longer share a compatible representation space. Keep model configuration consistent or re-embed.

## Test embedding compatibility

Compare retrieval with a compatible embedding and a controlled model change.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Record the embedding model used for the test index.

Reason. Index identity determines query compatibility.

Evidence. The current embedding configuration is documented.

2. Select labeled questions and expected source passages.

Reason. Expected matches support relevance checks.

Evidence. The test set maps each query to known evidence.

3. Run retrieval with the compatible query embedding configuration.

Reason. This establishes the baseline.

Evidence. Returned source identifiers and relevance judgments are saved.

4. Change the query embedding configuration only in a test path.

Reason. This isolates the compatibility risk.

Evidence. The test request uses the alternate query embedding.

5. Compare returned sources with the baseline.

Reason. Different representations can change similarity behavior.



Evidence. The comparison shows lost, gained, or reordered evidence.

6. Restore compatibility or re-index a test copy before a migration trial.

Reason. A coordinated migration changes both sides.

Evidence. The selected test setup uses matching query and index representations.

### Expected evidence

1. The index embedding model is known.

2. The baseline query set has expected passages.

3. The migration decision reflects paired index and query models.

### Failure checks

1. Check vector dimensions and model identifiers.

2. Check whether the corpus was re-embedded.

3. Do not treat similarity scores as truth probabilities.

### Cleanup

1. Keep the production index unchanged.

2. Delete only a disposable test index if one was created.

### Exam transfer

During embedding migration, change query and index representations as a coordinated versioned pair.

Sources [16, 20, 23].

Sources [16, 20, 23].

# Chapter 8. Vector stores

A vector store holds vectors and metadata so a system can retrieve related records.

### The smallest unit

A vector store indexes records for similarity search. A record commonly includes an embedding and metadata, and may include text or a reference to text. The query vector identifies candidate records by proximity under the store configuration.

### How the mechanism works

The vector store is one component in a retrieval path. It does not decide whether a user may see a record unless the application or service applies access filters correctly. It does not verify that retrieved facts are true. It can return a close match that still fails the question.

### Where it fits

Choose a store by integration, scale, filter needs, operational model, and data controls. Amazon Bedrock Knowledge Bases can connect managed ingestion and retrieval functions with supported storage options. A custom retrieval system can offer more control, but it also adds components to operate. Confirm current service support and limits before implementation.

### Failure and control

Inspect returned passages, scores, metadata, and source identifiers. Do not interpret similarity score as a truth probability. Compare retrieval output to a labeled set of relevant records. Use filters from trusted application identity to constrain the candidate set.

### Causal worked trace

Causal trace: The ingestion path stores a vector with source text or a source reference and filter metadata. A query vector reaches the store. The store applies configured filters and similarity search. It returns candidate records. The application reads their text and identifiers and places selected content in the prompt. If authorization filters are missing, the store can return a close match from a record the caller should not see.

### Failure diagnosis

Failure diagnosis: Review the query filter, its trusted identity source, and the returned metadata. Confirm that filters apply before the model receives content. Then evaluate candidate relevance and source authority. A vector score cannot diagnose permission errors or establish factual correctness.

### Transfer case and misconception

Transfer case: A multi-tenant assistant stores documents for several business units. Derive the tenant filter from authenticated identity, not from a free-form user parameter. Test one allowed and one denied source. Confirm that denied text is absent from both retrieved results and model input.

Misconception correction: The vector store locates candidates. It does not automatically enforce every application access policy or verify the truth of a candidate.

### Design implications

A vector store can filter by metadata only when the configured fields and query path support that operation. Confirm the field is indexed for the required filter behavior. Keep authorization metadata under trusted control. Avoid using a client-supplied tenant name as the sole access check. Test the empty-result case and the denied-result case. Both should produce safe application behavior. A user should not infer the existence of a restricted record from an error message, count, or citation.



### Invariant

A candidate may reach generation only if it passes both retrieval relevance and authorization checks.

### Controlled experiment

1. Create a test identity with access to one metadata partition.
2. Set the retrieval filter to that identity's trusted partition value.
3. Query for a passage that exists in both an allowed and a denied partition.
4. Inspect result metadata and model input for denied content.

A correctly applied filter excludes records outside the trusted partition, even when their vectors are close matches.

If denied records appear, fix filter derivation or enforcement before evaluating answer quality.

### Transfer check

Predict why a nearest-neighbor result can still be wrong.

The nearest vector is the closest available representation, not proof that the passage answers the question. Validate relevance and source authority.

## Inspect vector filtering

Confirm trusted metadata filters constrain retrieval before generation.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Choose two test records with distinct trusted metadata values.

Reason. Paired records expose filter behavior.

Evidence. Source identifiers and metadata values are recorded.

2. Call Retrieve without a filter using a query that matches both records.

Reason. The baseline shows available candidates.

Evidence. The response includes both records or explains search behavior.

3. Call Retrieve with a trusted metadata filter for one partition.

Reason. Filtering should constrain the candidate set.

Evidence. Only permitted partition results appear.

4. Repeat with the second partition and compare results.

Reason. Symmetric tests catch hard-coded behavior.

Evidence. Each request returns its intended source set.

5. Inspect metadata fields in the response.

Reason. Returned metadata supports authorization diagnosis.

Evidence. The filter field and value match the trusted test identity.

6. Review the application path that supplies the filter.

Reason. A user-controlled value must not establish authorization.

Evidence. The identity-to-filter mapping has a trusted source.

### Expected evidence

1. Filtered retrieval returns only the expected test partition.
2. The application derives filters from authenticated identity.
3. No denied passage reaches model input.

### Failure checks

1. Check metadata field support and filter syntax.
2. Check whether the filter is applied before retrieval.
3. Check the final model context, not only the visible answer.

### Cleanup

1. Do not alter production access policies.
2. Remove local test identifiers and saved outputs as required.

### Exam transfer

Apply the same identity-bound filter test to every tenant or department route.

Sources [20, 16, 24].

Sources [20, 16, 24].

# Chapter 9. Hybrid search and reranking

Hybrid search combines different retrieval signals, while reranking reorders candidates for a query.

### The smallest unit

Vector search finds passages by semantic similarity. Lexical search matches terms and identifiers. Hybrid search combines these signals so a query can benefit from both meaning and exact words. This matters for product codes, names, legal terms, and paraphrased questions.

### How the mechanism works

A first retrieval stage can return a broader candidate set. A reranker then scores candidates against the full query and orders them by likely relevance. The model that generates the answer receives a smaller, more focused set. Each stage adds cost and latency. A reranker cannot recover a passage that the first stage never returned.

### Where it fits

Tune candidate count and final context size together. Too few candidates can omit needed evidence. Too many can add noise and consume context. Preserve filters across retrieval stages. Verify that a reranker does not move a relevant exception below the cut line.



### Failure and control

Compare vector-only, lexical-only, hybrid, and reranked retrieval on the same labeled queries. Measure recall and precision, then review failure cases.

### Causal worked trace

Causal trace: A user asks for a product by its exact SKU and description. Vector search finds semantically similar product text. Lexical search matches the exact SKU. Hybrid search combines both candidate sets. A reranker scores candidates against the full query and orders them. The generator receives the final candidates. If the first stage excludes the exact record, reranking cannot restore it.

### Failure diagnosis

Failure diagnosis: Inspect candidates before and after reranking. If the exact record is missing at first stage, examine lexical fields, filters, and candidate count. If present but ranked low, inspect reranker inputs and query. If the right record reaches generation but answer is wrong, inspect prompt use and output validation.

### Transfer case and misconception

Transfer case: A support system searches error codes and natural-language descriptions. Compare semantic retrieval for paraphrases with lexical retrieval for exact codes. Use hybrid only where the store supports it. Verify the text field is configured for filtering and lexical search as required by the selected service path.

Misconception correction: Reranking improves order among available candidates. It cannot repair missing source data, an incorrect filter, or a first-stage recall gap.

### Design implications

Measure candidate recall before tuning reranking. If the relevant item is absent from the candidate set, reranking has no opportunity to help. If candidates include the answer but order it poorly, reranking can address that stage. Keep exact-term queries and paraphrase queries in separate evaluation groups. This shows whether a change helps one class while hurting another. Also inspect the final prompt because a better ranking can still fail if the application truncates the useful passage or sends duplicate context.

### Invariant

A later ranking stage cannot select a relevant passage that an earlier candidate stage never returned.

### Controlled experiment

1. Choose labeled queries that include exact identifiers and paraphrases.
2. Set the knowledge base search type to SEMANTIC, then record candidate recall.
3. Set the search type to HYBRID on a supported store and repeat.

4. If configured, apply the same reranker and compare final ranking and answer evidence.

Hybrid search can improve exact-term recall while semantic search can serve meaning-based matches.

Use the search mode that improves the target query class without unacceptable noise. Add reranking only when candidate order remains a measured defect.

### Transfer check

Predict which query gains from lexical search when the source contains an exact SKU.

The exact SKU is a strong lexical signal. Hybrid retrieval can retain the exact record even if its semantic vector is not the closest match.

## Compare search modes

Compare semantic and hybrid search for exact terms and paraphrases.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Select queries with exact identifiers and paraphrased descriptions.

Reason. Different query types exercise distinct search signals.

Evidence. Each query has expected source records.

2. Call Retrieve with semantic search and save candidate sources.

Reason. This establishes semantic retrieval behavior.

Evidence. Results are recorded by query type.

3. Call Retrieve with retrievalConfiguration set to HYBRID on a supported store.

Reason. The search type combines vector and raw-text signals where supported.

Evidence. The request records overrideSearchType and returns candidate sources.

4. Compare exact identifier recall and paraphrase relevance.

Reason. One mode may help one query class more than another.

Evidence. A comparison table records source coverage and noise.

5. Inspect first-stage candidates before testing an available reranker.

Reason. Reranking addresses ordering among returned candidates.

Evidence. A candidate absent from first stage is marked as recall loss.

6. Keep the mode that meets the test criteria and document the store boundary.

Reason. Hybrid support depends on the configured store.

Evidence. The decision includes capability and relevance evidence.



### Expected evidence

1. Both search modes are compared on the same queries.
2. Exact-term and meaning-based results are reviewed separately.
3. Store support is confirmed before choosing hybrid.

### Failure checks

1. Check that the text field is filterable as required.
2. Check that expected records enter first-stage candidates.
3. Check noise and final context size.

### Cleanup

1. Do not change a shared knowledge base search mode without owner approval.
2. Remove local test result files.

### Exam transfer

Use exact product codes and paraphrases in separate retrieval evaluation groups.

Sources [16, 20, 21, 24].

Sources [16, 20, 21].

# Chapter 10. Prompt management

Prompt management stores reusable instructions and separates stable rules from request data.

### The smallest unit

A prompt shapes how a model interprets its input. It can define a role, task, constraints, examples, and output format. A prompt does not grant capabilities or verify truth. Those controls belong in application logic and supporting systems.

### How the mechanism works

A prompt resource can make a template reusable across application calls. Variables can hold request-specific values. Versioning helps teams compare prompt changes and return to a known version. Keep stable policy instructions separate from user content and retrieved evidence. Clearly label untrusted text so it cannot silently act as a higher-level instruction. For Amazon Bedrock Knowledge Bases, hybrid search is supported only with specified vector stores that contain a filterable text field. Otherwise, the query uses semantic search.

### Where it fits

Examples can teach format and task boundaries. Output schemas can improve structure where supported. Validate generated output after inference. A prompt change is a behavior change, so review it against evaluation cases before release. Do not place secrets or unnecessary personal data in a prompt.

### Failure and control

Store prompt identifiers and versions with safe evaluation records. This makes a response easier to reproduce. A model update, prompt edit, or retrieval change can each alter the result.

### Causal worked trace

Causal trace: A prompt template stores stable instructions and named variables. At runtime, the application supplies the user question and retrieved text in their designated variables. The prompt resource version resolves into a request. The model interprets instructions and data, then returns a response. If untrusted retrieved text is mixed with trusted rules, instruction-like text inside the evidence can confuse the task boundary.

### Failure diagnosis

Failure diagnosis: Compare prompt version, resolved variable values, and model request. If the format fails, validate schema and examples. If behavior changed, compare the deployed prompt version with the evaluated version. If content injection appears, strengthen boundaries and application controls. A template system does not validate the output automatically.

### Transfer case and misconception

Transfer case: A team changes a prompt to require citations. Create a new version, run the citation evaluation set, and compare support for claims. Keep retrieval data unchanged during that comparison. If quality declines, inspect whether the prompt asks for unsupported citations or changes evidence ordering.

Misconception correction: Prompt instructions guide model behavior. They do not grant permissions, make retrieved text trusted, or guarantee schema-valid output.

### Design implications

Store prompts with a clear owner, purpose, and version. Keep variables typed or validated where the prompt system allows it. Escape or delimit variable data according to the application format. Do not place secrets in examples. Review examples for accidental bias toward a narrow output. When a prompt changes, record the change and its test result. A short prompt can still be unsafe if it grants broad tool use or asks the model to infer missing facts.

### Invariant

Stable instructions, untrusted data, and output constraints must remain distinguishable through prompt assembly and response validation.



### Controlled experiment

1. Record the active prompt resource version and its resolved request.

2. Change only the prompt version to add explicit delimiters around retrieved passages.

3. Run a test passage that contains an instruction-like sentence.

4. Compare whether the model follows the task or the embedded sentence.

Clear boundaries can reduce confusion between the application instruction and quoted evidence, but they do not replace tool authorization.

Retain the prompt change only if the evaluation shows better boundary behavior without harming normal answers.

### Transfer check

Predict the risk of pasting a user document into the same text block as trusted system rules.

The model may treat document instructions as task instructions. Delimit and label untrusted content, and enforce rules outside the prompt.

## Version a prompt template

Create and compare a prompt version without changing the model or evidence.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Record the active prompt resource and version.

Reason. A baseline is needed for comparison.

Evidence. The prompt ID and version are documented.

2. Identify a specific failure and select a test case.

Reason. A change should target an observed defect.

Evidence. The expected behavior is written before editing.

3. Create a new prompt variant with the same variables and model route.

Reason. A variant isolates prompt behavior.

Evidence. The new version or variant is identifiable.

4. Run both versions with identical variable values.

Reason. Equal inputs support a fair comparison.

Evidence. Outputs and inference settings are saved.

5. Check output structure and evidence use.

Reason. Fluency alone does not show prompt quality.

Evidence. The test result records parse and support checks.

6. Promote only the evaluated version through the application path.

Reason. Runtime linkage prevents an untested draft from serving users.

Evidence. The application references the reviewed prompt version.

### Expected evidence

1. The test compares identifiable prompt versions.
2. The model, evidence, and variable values stay constant.
3. The deployed reference matches the reviewed version.

### Failure checks

1. Check for untrusted text that can resemble instructions.
2. Check that variables resolve to the intended values.
3. Check that output validation occurs after inference.

### Cleanup

1. Keep the current production prompt unchanged during comparison.
2. Remove only unused test variants under prompt ownership rules.

### Exam transfer

For a citation defect, keep retrieval fixed while comparing prompt instructions and citation checks.

Sources [25, 3, 24].

Sources [25, 3, 6, 24].

# Chapter 11. Agents

An agent uses a model to select steps and may call tools to complete a goal.

### The smallest unit

A basic language model returns text. An agent loop can also expose tools, receive a tool choice, run the selected action, and send the observation back to the model. The application or agent service controls which actions are available.

### How the mechanism works

An agent needs a goal, instructions, tools, state, and a stopping rule. Each tool has an input contract and an effect. Read-only tools can retrieve information. Write tools can change systems. The agent can select a tool incorrectly, send invalid arguments, or repeat a step. Guard the tool boundary with validation and least privilege. Amazon Bedrock Agents Classic is no longer open to new customers. Existing customers can continue to use it, and AWS directs new capability exploration to Amazon Bedrock AgentCore.

### Where it fits

Use an agent when the task requires conditional steps or tool choice. Use a fixed workflow when the sequence is known and predictable. Limit each tool to the smallest required action. Require human approval for consequential actions when the application risk calls for it.



### Failure and control

Measure task completion, tool selection, invalid calls, retries, and unwanted side effects. Do not treat a fluent transcript as proof that the external task succeeded. Confirm outcomes through the system of record.

### Causal worked trace

Causal trace: A user requests an action. The agent receives a goal and a set of tool definitions. The model selects a tool and supplies arguments. The runtime validates and invokes the tool. The result returns as an observation, and the agent can choose another step or stop. If the action is a known fixed sequence, every model decision adds a possible branch without adding useful flexibility.

### Failure diagnosis

Failure diagnosis: Trace the goal, selected action, arguments, tool response, and stop decision. A wrong tool choice points to tool descriptions or model reasoning. Invalid arguments point to schema or validation. A duplicate action points to retry or stopping logic. Verify external state because a transcript does not prove an action succeeded.

### Transfer case and misconception

Transfer case: An agent can look up an invoice and issue a refund. Separate read and write permissions. Require a user approval before the refund tool call. After the tool returns, verify invoice state through a trusted system query before telling the user the refund completed.

Misconception correction: An agent is not a model with unrestricted authority. The runtime determines available actions, validates calls, and enforces access controls.

### Design implications

Keep the agent's action set small and aligned with the task. Tool descriptions should state when the tool applies and what effect it has. Inputs should have required fields and validation. Define a maximum step or time budget where the runtime supports it. Make the stopping condition observable. If an agent reaches the same state repeatedly, stop it and surface a controlled error. For important actions, verify the resulting state from the system of record before reporting completion.

### Invariant

The agent can perform only actions exposed by its runtime, and each action still requires application authorization.

### Controlled experiment

1. Use a test agent with one read-only lookup tool and one write tool disabled.
2. Submit a task that asks for the write action.
3. Inspect the selected tool request and runtime response.

4. Enable the write tool only behind an approval check and repeat with approval absent.

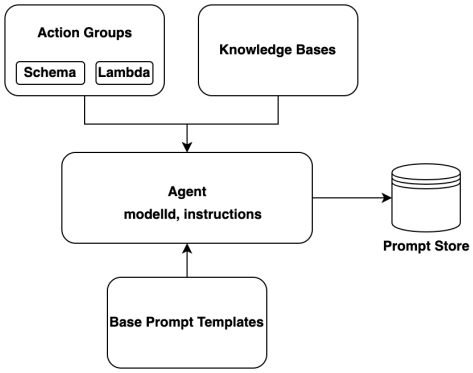
The agent may request a write action, but the runtime should deny it while the tool is disabled or approval is absent.

Keep authorization in runtime logic. Use agent reasoning to select among permitted actions, not to grant permission.

### Transfer check

Predict why a fixed sequence can be safer than an agent for a two-step approval task.

The known sequence can use explicit states and checks. An agent adds model decisions without clear value for a deterministic path.



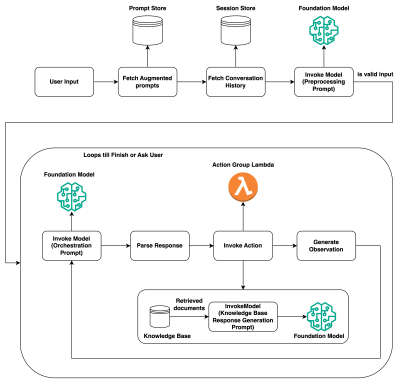

Amazon Bedrock Agents Classic build-time setup

Trace how prompts and knowledge sources connect to the agent.

1. Locate the model, instructions, action groups or knowledge bases, prompts, and prompt store.

This is Amazon Bedrock Agents Classic. AWS says it is closed to new customers. Check the source for migration guidance.

https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html




Amazon Bedrock Agents Classic runtime loop

Follow the request through orchestration, action groups, knowledge bases, and response.

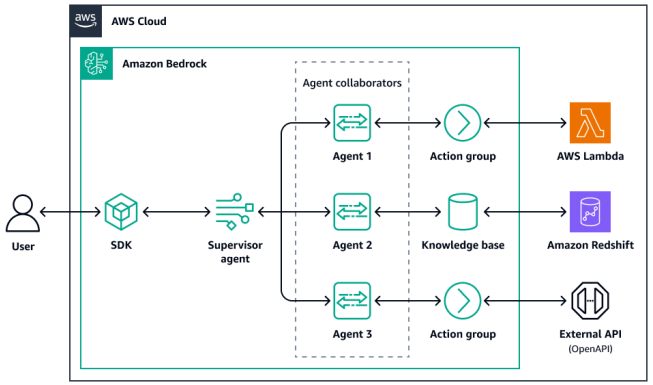


1. Follow preprocessing, orchestration, optional action or retrieval work, and post-processing.

2. Find where orchestration can repeat before the agent returns a response.

This is Amazon Bedrock Agents Classic. AWS says it is closed to new customers. Check the source for migration guidance.

https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html




Supervisor agent and collaborators

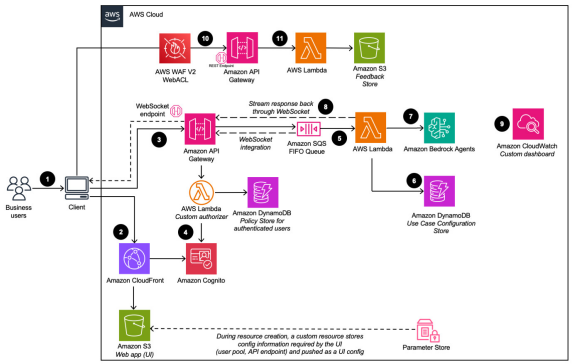
Identify the supervisor, specialist agents, and shared result path.

1. Trace how the supervisor assigns subtasks to collaborators.

2. Find the point where results return for synthesis.

The diagram illustrates a pattern. It does not specify a Bedrock API contract.

https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/saga-orchestration-patterns.html




Application Builder Bedrock Agent use case

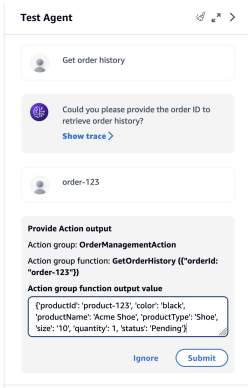
Trace the agent request through orchestration, tools, and data services.

1. Follow the user request through the queue, Lambda, and Bedrock Agent.

2. Locate the response path back to the client.

This architecture belongs to the named solution. It does not show every agent design.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html




Test an agent return-control action

When an agent returns control, enter a representative action result and submit it to continue the test.

1. The test window shows the action group OrderManagementAction.
2. The function shown is GetOrderHistory with orderId order-123.
3. The form requests an Action group function output value.
4. The buttons are Ignore and Submit.

This is a static AWS documentation screenshot. The order and product fields are sample values. Agents Classic is closed to new customers according to a separate AWS documentation snapshot.

https://docs.aws.amazon.com/bedrock/latest/userguide/agents-test.html

## Trace a bounded agent action

Inspect tool selection, runtime authorization, and outcome verification.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Review the agent instruction and available tool schemas.

Reason. The action set shapes possible behavior.

Evidence. The approved tool list and schemas are recorded.

2. Submit an illustrative request that requires a read operation.

Reason. A test task exercises model selection.

Evidence. The trace shows the chosen tool and arguments.

3. Validate arguments and caller access before invoking the tool.

Reason. The host owns validation and permission checks.

Evidence. The runtime records an allowed or denied decision.

4. Inspect the tool result returned as an observation.

Reason. The agent needs external evidence to continue.

Evidence. The trace links the result to the selected call.

5. Verify the external state through the system of record.



Reason. A transcript does not prove the operation succeeded.

Evidence. The record confirms the observed outcome.

6. Test a denied or malformed request in the sandbox.

Reason. Failure paths reveal control gaps.

Evidence. The invalid request stops before external action.

### Expected evidence

1. The trace captures the tool request and observation.
2. The host checks identity and arguments.
3. The system of record confirms the result.

### Failure checks

1. Check whether the selected tool was exposed intentionally.
2. Check repeated calls and stopping behavior.
3. Check for side effects before retrying an ambiguous call.

### Cleanup

1. Use read-only tools only for this lab.
2. Do not enable or invoke write operations.

### Exam transfer

Choose a deterministic workflow when the task path is known and agent selection adds no value.

Sources [26, 27, 28].

Sources [26, 27, 28].

# Chapter 12. Tools and MCP

A tool gives a model a controlled way to request an application action or retrieve data.

### The smallest unit

A tool definition describes a name, purpose, and input shape. The model can request that tool. The host application validates the request, invokes the implementation, and returns an observation. The model cannot call a real service by naming it in text alone.

### How the mechanism works

Model Context Protocol defines a client and server pattern for exposing tools and resources to a model-connected application. MCP can standardize discovery and calls across systems. It does not remove the need to authenticate, authorize, validate, and audit each operation. Tool output is data. Treat it as untrusted input to later model steps.

### Where it fits

Expose narrow tools with clear argument schemas. Keep credentials in the host environment. Apply identity checks at the service boundary. Return only the data needed for the task. For write actions, add idempotency or approval controls where the operation can cause duplicate or harmful effects.

### Failure and control

Test malformed arguments, denied access, timeouts, malicious tool output, and partial success. Log the tool name and outcome without storing unnecessary sensitive content.

### Causal worked trace

Causal trace: A host discovers an MCP server's tool description and schema. The model sees the available tool contract. It requests a call with arguments. The host validates the request and authenticates to the backing service. The server returns data. The host passes that result to the model as untrusted content. If the server returns hostile instructions, those instructions remain data and must not change host policy.

### Failure diagnosis

Failure diagnosis: Follow discovery, model call, host validation, server authorization, and result handling. A missing tool can be a discovery or configuration defect. A denied call can be an identity or permission issue. Unexpected behavior can come from unvalidated arguments or unsafe tool output. Inspect each boundary rather than blaming MCP as a single component.

### Transfer case and misconception

Transfer case: An MCP tool reads customer records. Expose only the required fields. Derive the user identity at the host boundary and pass a scoped credential. Test denied customer IDs and malformed arguments. Keep write tools separate from read tools and log the action outcome without excess customer content.

Misconception correction: MCP standardizes communication patterns. It does not make a server trustworthy or remove the host's duty to authorize each call.

### Design implications

MCP can make tool discovery and invocation more consistent across clients, but the security model still depends on host and server setup. Review server identity, transport, authentication, tool schemas, and returned data. Avoid enabling every discovered tool by default. Present only the tools needed for the task. A server update can change a tool description or behavior, so track versions and review changes. Treat tool output as external input, even when it comes from an approved server.



### Invariant

Every tool call and result crosses a trust boundary that requires validation and least privilege.

### Controlled experiment

1. Configure a test tool with a schema that requires a customer identifier.
2. Change the host validator to reject identifiers outside the authenticated test identity.
3. Request an allowed identifier, then a denied identifier.
4. Inspect service calls and returned content for both requests.

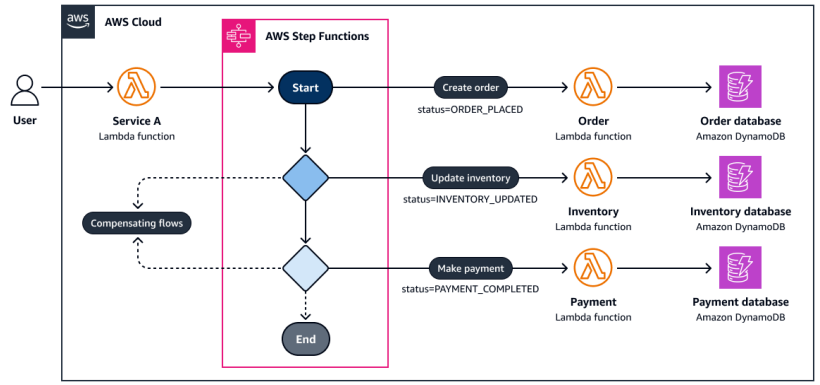
The allowed call reaches the service. The denied call stops at the host authorization check.

Keep identity-derived validation before the tool operation. Do not rely on model instructions to restrict identifiers.

### Transfer check

Predict what fails if an MCP server returns a string that contains an instruction to ignore system rules.

The host should treat the string as untrusted tool data. Preserve the instruction hierarchy and validate any follow-up action.



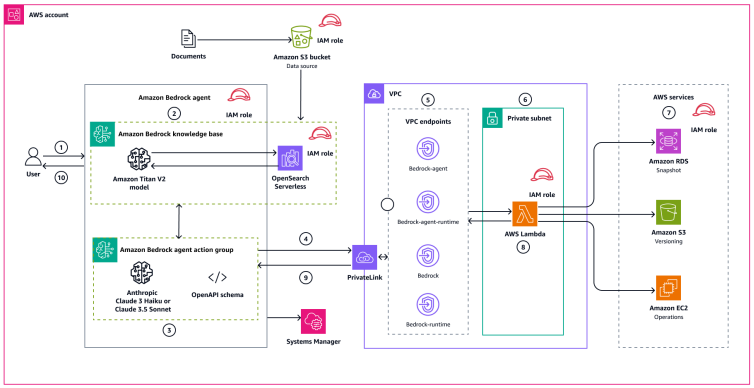

Saga workflow with AWS Step Functions

Trace the sequence, decision branches, and completion path.

1. Follow order creation, inventory update, and payment.
2. Locate the parallel, retry, timeout, and failure paths described in the source text.

The diagram illustrates a pattern. It does not prescribe transaction guarantees.

https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/saga-orchestration-patterns.html




Bedrock infrastructure operations workflow

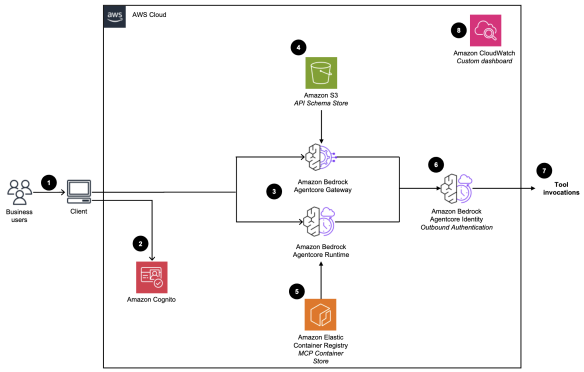
Follow the request through the agent workflow and infrastructure actions.

1. Follow a request from the Bedrock chat interface through the action group and Lambda.

2. Locate the VPC endpoints and target AWS operations.

The workflow is tied to the documented solution pattern. It does not cover every approval control.

https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/automate-aws-infrastructure-operations-by-using-amazon-bedrock.html




Application Builder MCP Server use case

Trace how the application connects to an MCP server and AWS services.

1. Locate the MCP server and its connection to application tools or resources.

2. Use the source text to distinguish gateway and runtime deployment methods.

This architecture belongs to the named solution. It does not define the full MCP protocol.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html

## Validate a tool or MCP boundary

Test schema validation, authorization, and untrusted tool output.

Proposed, not executed.



This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Inspect the discovered tool name and input schema.

Reason. Discovery defines the callable surface.

Evidence. The exposed tools match the approved list.

2. Prepare an allowed argument and a malformed argument.

Reason. Paired inputs test validation behavior.

Evidence. Both test cases are documented.

3. Send the allowed call through the host client.

Reason. The host should validate before server execution.

Evidence. The server returns a bounded result.

4. Send the malformed call and inspect host behavior.

Reason. Invalid arguments should stop at the boundary.

Evidence. No backing service operation occurs.

5. Return instruction-like text from a local stub.

Reason. Tool output is untrusted data for later model steps.

Evidence. The host retains it as data and applies existing policy.

6. Review server identity, authentication, and permissions.

Reason. Protocol compatibility does not establish trust.

Evidence. The security boundary is documented.

### Expected evidence

1. Allowed calls return only required fields.
2. Malformed calls stop before service action.
3. Tool output cannot override host policy.

### Failure checks

1. Check tool discovery scope and server identity.
2. Check argument validation before invocation.
3. Check the host treats results as untrusted input.

### Cleanup

1. Use a local stub or read-only test server.
2. Do not store credentials in tool output or test fixtures.

### Exam transfer

Apply the same boundary checks to every tool server and discovered operation.

Sources [27, 26, 5].

Sources [26, 27, 5].

# Chapter 13. Deployment choices

Deployment places inference behind a service path that fits control and workload needs.

### The smallest unit

A managed model API reduces the need to operate model serving infrastructure. A custom hosted model can provide more control over model selection and serving configuration. The right choice depends on workload, model rights, performance needs, and operational capacity.

### How the mechanism works

Amazon Bedrock provides managed access to supported foundation models. Amazon SageMaker AI supports model deployment options for inference workloads. Managed endpoints, serverless options, and other deployment patterns have different scaling, startup, capacity, and cost behavior. Check current product documentation for availability and limits.

### Where it fits

Map the request path from application to endpoint. Include network access, identity, secrets, monitoring, scaling, and failure response. Test startup and peak behavior. Keep model artifacts, inference code, and endpoint configuration under version control where applicable.

### Failure and control

Deployment is not only a model endpoint. The application needs a controlled release process for prompt, retrieval, model, and infrastructure changes. Roll back the smallest changed component when evidence points to it.

### Causal worked trace

Causal trace: A team selects managed model inference or a hosted deployment path. The application builds a request and authenticates with its runtime role. Network and service controls route the call. The chosen endpoint runs inference. The application records result and error data. A health check can pass while a real request fails because model access, payload validation, or a downstream dependency differs from the probe.

### Failure diagnosis

Failure diagnosis: Test the full path with the same identity, Region, model route, network, and request body as production. Separate endpoint health from request success. Check deployment configuration, capacity, model availability, and response parsing. Review current service capabilities before assuming an option is supported in the target Region.

### Transfer case and misconception

Transfer case: A team moves a prototype into a production endpoint. Define the caller role, network path, request schema, scaling plan, monitoring, and rollback version. Run representative traffic before cutover. Record which component owns model artifact updates and endpoint settings.



Misconception correction: A managed endpoint reduces some infrastructure work. It does not remove application operations, IAM, capacity planning, or release responsibility.

### Design implications

Select a deployment path from workload characteristics and ownership limits. Consider whether traffic is steady or bursty, whether a model artifact must be customized, and which team owns scaling and patching. Compare the total operational path, including monitoring and recovery. Confirm model rights, service support, and Region availability. Run a representative request through the deployed path with the production identity. Keep a rollback configuration that the on-call team can apply without rebuilding the system.

Treat model packaging, endpoint configuration, and application code as separate release items. Assign an owner to each item and test rollback across all three.

### Invariant

Deployment success requires the complete production request path to work under its intended identity and load.

### Controlled experiment

1. Use a staging endpoint and a production-equivalent runtime role.
2. Change the endpoint configuration to the planned deployment option.
3. Send the same representative request through the application path.
4. Record request success, latency, scaling behavior, and errors.

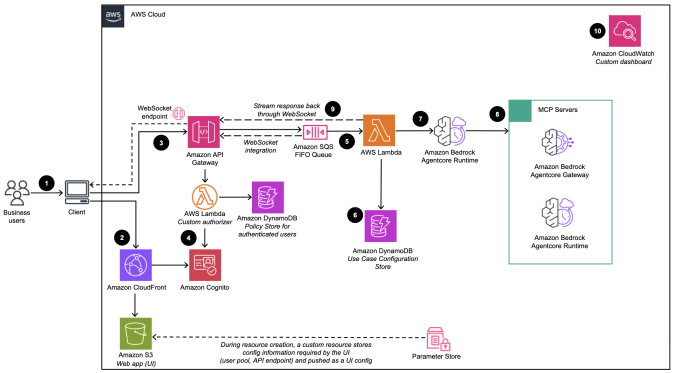
A complete-path test can reveal identity or payload failures that an endpoint health check does not exercise.

Promote only after the application path meets the workload criteria. Keep rollback tied to a known endpoint configuration.

### Transfer check

Predict why a model endpoint can pass a health check while user requests still fail.

The health check may not exercise authentication, request schema, model availability, or downstream dependencies. Test the full request path.




Application Builder Agent Builder deployment

Trace the agent builder deployment components and their AWS connections.

1. Trace the Agent Builder components and their AgentCore connections.
2. Use the source text to identify the separate Workflow Builder component.

This architecture belongs to the named solution. It does not cover all production deployment controls.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html

## Compare inference deployment paths

Map workload needs to an appropriate inference deployment option.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Describe request volume, burst pattern, latency, and model control needs.

Reason. Deployment choice depends on workload characteristics.

Evidence. The profile lists qualitative needs and assumptions.

2. Review current Bedrock model and provisioned throughput documentation.

Reason. Managed inference and provisioned options have different constraints.

Evidence. Supported routes and limits are noted.

3. Review SageMaker AI inference options for any hosted model requirement.

Reason. Hosted deployment may fit different control needs.

Evidence. Candidate options map to the deployment profile.

4. Send representative test requests through the existing approved route.

Reason. A baseline provides operational evidence without provisioning.

Evidence. Latency, errors, and usage are recorded.

5. Compare options against operations, capacity, and rollback needs.

Reason. Performance alone does not determine fit.



Evidence. A decision matrix records tradeoffs.

6. Do not create endpoints until ownership and cost review is complete.

Reason. Deployment resources can incur ongoing cost.

Evidence. The lab ends with a recommendation, not a deployed resource.

### Expected evidence

1. The workload profile drives the candidate list.
2. Existing route data informs the decision.
3. Any endpoint creation remains a separately approved action.

### Failure checks

1. Check current model availability and Region.
2. Check whether capacity needs are steady or bursty.
3. Check operational ownership and cost exposure.

### Cleanup

1. Do not create, modify, or delete endpoints.
2. Keep test calls within approved sandbox budgets.

### Exam transfer

For a customized model, include artifact lifecycle and deployment ownership in the decision.

Sources [29, 18, 5].

Sources [29, 18, 5].

# Chapter 14. Enterprise gateway

A gateway creates a controlled entry point for model requests across an organization.

### The smallest unit

Applications often need a common path for identity checks, policy, logging, rate limits, and model routing. A gateway can enforce shared rules before requests reach a model service. It can also simplify application integration by hiding service details behind a stable interface.

### How the mechanism works

A gateway should authenticate the caller and decide what model and data operations that caller can use. It can apply request validation, quotas, allowed model lists, and routing rules. It can also attach trace identifiers. The gateway does not make unsafe prompts safe by itself. Each tool and data source still needs its own authorization.

### Where it fits

Keep policy decisions explicit and testable. Avoid logging full prompts by default when they can contain confidential data. Define who can change routing and model allowlists. Track gateway errors separately from provider errors so teams can locate the failing boundary.

### Failure and control

Centralization can create a shared outage or bottleneck. Use health checks, capacity planning, and documented fallback behavior. A fallback model may have different capabilities, so validate the answer contract before failover.

### Causal worked trace

Causal trace: A client sends a model request to an enterprise gateway. The gateway authenticates the principal, checks the requested route against policy, applies quotas, and forwards an allowed request. The provider returns a result. The gateway adds trace data and returns the response. If the client chooses any model identifier and the gateway trusts it, callers can bypass route policy or cost controls.

### Failure diagnosis

Failure diagnosis: Determine whether the request failed at caller authentication, route authorization, quota, provider invocation, or response handling. Use a trace identifier across boundaries. Do not log full sensitive prompts as a shortcut. Check whether fallback routing preserves model capabilities and the application response contract.

### Transfer case and misconception

Transfer case: An organization exposes a single internal model endpoint to several teams. Map each team to allowed model routes and quotas. Apply caller-specific authorization before forwarding. Give policy owners a controlled review path for route changes. Test a disallowed route and confirm that no provider call occurs.

Misconception correction: A gateway centralizes policy enforcement only for requests that pass through it. Direct service credentials or unguarded tools can create another route.

### Design implications

A gateway should publish a stable contract to client teams and keep policy rules reviewable. Define which identity claims are trusted, which routes are allowed, and how quotas are assigned. Include a path for policy updates and emergency disablement. Monitor gateway rejection separately from upstream model errors. If routing falls back, record the selected route and ensure that the fallback meets required output and safety behavior. Never let a client bypass the gateway with a broader credential than the gateway uses.



### Invariant

The gateway must authenticate the caller and authorize the model route before forwarding the request.

### Controlled experiment

1. Configure a test caller with one permitted model route.

2. Change its requested model identifier to a route outside the allowlist.

3. Send the request through the gateway and inspect gateway and provider traces.

4. Restore the permitted route and compare outcomes.

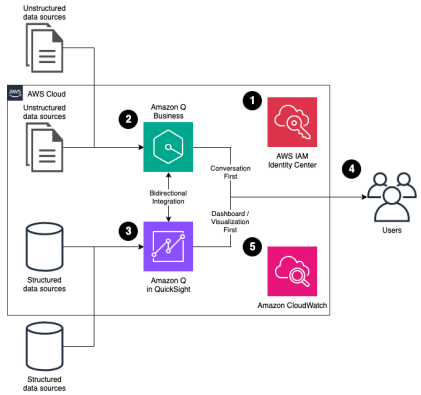
The disallowed request is rejected before provider invocation when route authorization is enforced at the gateway.

Keep the allowlist check before forwarding. Audit other paths that can invoke models outside the gateway.

### Transfer check

Predict the risk of a gateway that accepts every model identifier supplied by a client.

A caller can invoke a model outside approved policy or cost limits. Enforce an allowlist and caller-specific authorization.




Managed generative BI reference architecture

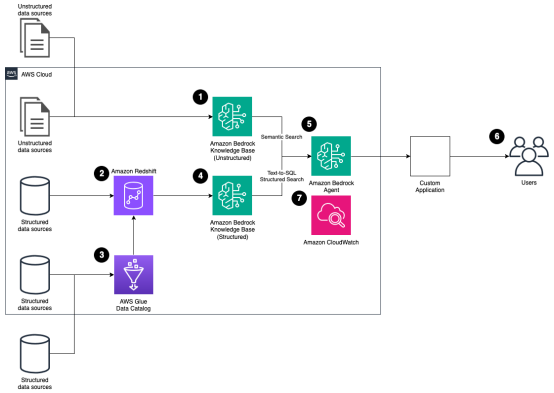
Trace the user request through managed data and model services.

1. Locate identity, source indexing, query, and user interface services.

2. Compare the unstructured and structured data roles.

This is a reference architecture for generative BI. It is not a general RAG design.

https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-business-intelligence.html



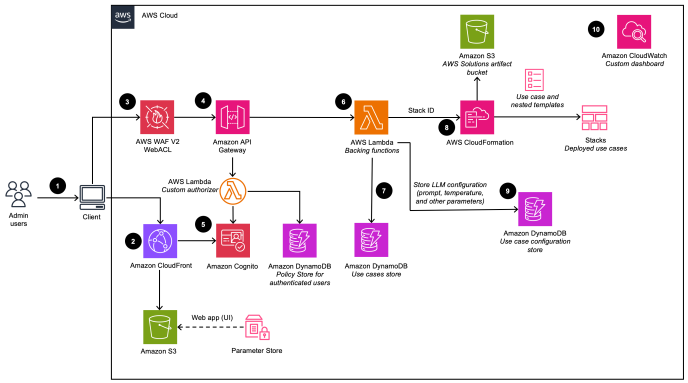

Custom generative BI reference architecture

Compare application responsibilities with the managed BI design.

1. Trace unstructured data through a Bedrock knowledge base.
2. Trace structured data through Redshift and its query path.

This is a reference architecture for generative BI. Adapt services to the workload.

https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-business-intelligence.html




Application Builder dashboard deployment

Trace the web application and supporting AWS services.

1. Trace the admin interface through CloudFront, API Gateway, and the backend services.
2. Locate deployment configuration storage and monitoring.

This architecture belongs to the named solution. It is not a universal deployment template.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html

## Review a model gateway policy

Trace authentication, route authorization, forwarding, and logging at an API boundary.

Proposed, not executed.



This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. List intended caller roles and allowed model routes.

Reason. The policy needs explicit caller-to-route rules.

Evidence. A role and route matrix is documented.

2. Review IAM policies and gateway request validation.

Reason. Authorization must precede provider invocation.

Evidence. The policy shows allowed actions and route checks.

3. Send an allowed test request through the gateway if available.

Reason. The positive path confirms expected forwarding.

Evidence. A trace shows the selected route and outcome.

4. Send a disallowed route request in the sandbox.

Reason. A negative test checks the enforcement boundary.

Evidence. The request is rejected before upstream inference.

5. Inspect invocation logs and redact sensitive request content.

Reason. Telemetry should support diagnosis without excess data.

Evidence. Logs identify route, outcome, and trace reference.

6. Document fallback behavior and disablement ownership.

Reason. Central routing can create a shared dependency.

Evidence. The design names an owner and recovery path.

### Expected evidence

1. Allowed requests pass through the policy boundary.
2. Disallowed routes stop before model invocation.
3. Logs contain safe diagnostic fields.

### Failure checks

1. Check alternate direct credentials that bypass the gateway.
2. Check route allowlists and caller identity derivation.
3. Check what prompt and response content enters logs.

### Cleanup

1. Do not change shared enterprise routing policy.
2. Remove only sandbox test request records under policy.

### Exam transfer

A fallback model must preserve required output and safety behavior before it can serve traffic.

Sources [5, 30, 4].

Sources [5, 30, 4].

# Chapter 15. Streaming and retries

Streaming returns response parts over time, while retries repeat a failed request under controlled rules.

### The smallest unit

A non-streaming call returns after the response is ready. A streaming call can deliver pieces as the model produces them. This can reduce perceived wait, but the application must handle partial output, stream errors, and final completion signals.

### How the mechanism works

Retries can help with transient service errors or throttling. They can also duplicate work, add latency, and raise cost. Use a retry policy with bounded attempts, backoff, and jitter. Check whether the operation is safe to repeat. Distinguish a failed connection from a completed response whose delivery status is unknown.

### Where it fits

Do not show partial text as a final answer until the stream completes. If a stream fails, decide whether to resume, retry, or display a clear error. For tool actions, use idempotency keys or a status check to prevent duplicate effects.

### Failure and control

Classify errors before retry. A validation error or access denial needs a request or policy fix, not repeated attempts. Record attempt count and final outcome.

### Causal worked trace

Causal trace: A client opens a response stream and receives partial content. The service later emits completion or an error. Separately, a retry policy can resend a request after a transient failure. If a tool operation completed but its response was lost, a blind retry can execute the side effect twice. Stream completion and operation completion therefore need explicit state handling.

### Failure diagnosis

Failure diagnosis: Record attempt number, stream state, error class, and whether a side effect may have occurred. Validation and access errors need correction, not retries. Throttling and transient service failures may support bounded backoff. A connection timeout after a write needs status lookup or idempotency before another call.

### Transfer case and misconception

Transfer case: A chat UI streams a generated response while also allowing a payment lookup tool. Render partial text as provisional. Mark the answer complete only after the final stream event. Give any consequential tool call an idempotency key and verify operation state after an ambiguous timeout.



Misconception correction: Streaming can improve time to first visible content. It does not guarantee lower total latency or make a partial response complete.

### Design implications

Retry policy needs an error taxonomy and an operation-safety check. Use bounded attempts so one failing request cannot consume unlimited capacity. Backoff reduces synchronized retry bursts when many clients see the same outage. Jitter can spread retries across time. For streaming, the client also needs clear states for open, partial, complete, and failed. Do not append a restarted stream to a partial answer unless the service contract supports resume. Mark incomplete output clearly and avoid treating it as an accepted result.

### Invariant

A retry is safe only when the operation is repeatable or the application can detect duplicate completion.

### Controlled experiment

1. Set the retry policy to one attempt for the test call.

2. Change the maximum retry count to a bounded value greater than one and set backoff for throttling errors.

3. Simulate a transient throttling response on a read-only request.

4. Compare attempt count, completion time, and final result.

A bounded retry can recover from a transient read failure but increases worst-case time.

Keep retries limited to retryable errors. Use idempotency or status checks for operations with side effects.

### Transfer check

Predict what a client may do when it retries a tool call after a timeout with no idempotency control.

The external action may run twice if the first call succeeded but the response was lost. Use idempotency or verify operation status.

## Exercise streaming and retry states

Observe partial response events and error-specific retry handling.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Open a streaming request with non-sensitive test content.

Reason. The client receives response events incrementally.

Evidence. The client records event order and time to first content.

2. Track partial content separately from terminal completion.

Reason. Partial output is not a final answer.

Evidence. The UI or client state shows open, partial, and complete.

3. Use the ConverseStream API and inspect its response events.

Reason. The API exposes stream-specific response events.

Evidence. The SDK event parser handles content and completion events.

4. Test a controlled client disconnect or local parser error.

Reason. Failure can occur after some content arrives.

Evidence. The client records incomplete state without marking success.

5. Review SDK retry behavior for a transient error class.

Reason. Retries should match error type.

Evidence. Retry configuration is bounded and documented.

6. Do not retry an ambiguous write action without status or idempotency.

Reason. A lost response can hide successful completion.

Evidence. The test design includes a status check or idempotency control.

### Expected evidence

1. The client distinguishes partial output from completion.
2. A midstream failure is visible and recoverable.
3. Retry behavior is bounded and error-specific.

### Failure checks

1. Check whether stream terminal events are parsed.
2. Check that validation errors do not retry.
3. Check for duplicate side effects after ambiguous timeouts.

### Cleanup

1. Close local stream clients after testing.
2. Do not invoke consequential write tools.

### Exam transfer

Use streaming for early feedback, but measure total completion time separately.

Sources [31, 32, 4].

Sources [33, 31, 4].

# Chapter 16. Guardrails

Guardrails apply configured safety and data controls to model inputs or outputs.

### The smallest unit

A guardrail can detect configured categories of harmful content or sensitive data patterns. It can also support topic controls and grounding-related checks in supported workflows. These controls reduce selected risks. They do not prove that every answer is safe or correct.



### How the mechanism works

Apply guardrails at the appropriate points in the request path. Input controls can screen a user request. Output controls can inspect a response. RAG flows may need checks for grounding or source relevance. The supported options depend on API and model integration. Test behavior with representative content and expected outcomes.

### Where it fits

A guardrail can block legitimate content if its policy is too broad. A policy can miss content outside its coverage. Keep a route for review and controlled exceptions. Do not rely on a prompt instruction as a substitute for a configured safety control.

### Failure and control

Measure block rates and inspect false positives and false negatives through an approved review process. Protect sensitive test data. Update policy when the application use case changes.

### Causal worked trace

Causal trace: A user submits content to a guarded inference path. Input policy checks inspect the request. The model receives allowed content and returns a response. Output checks inspect that response before delivery. In a RAG application, authorization and retrieval occur earlier. An output guardrail cannot prevent an unauthorized record from entering the model context before generation.

### Failure diagnosis

Failure diagnosis: Identify whether the issue is input screening, output screening, grounding, or retrieval authorization. Review the configured policy, API path, and result. Test false blocks and missed blocks with approved cases. If private evidence reached generation, correct retrieval permissions first, then assess output controls.

### Transfer case and misconception

Transfer case: A health support assistant screens harmful content and retrieves care guidance. Apply access checks to source records before prompt assembly. Configure appropriate input and output policies. Route uncertain high-impact cases to the approved human review process. Evaluate both allowed and disallowed test content.

Misconception correction: Guardrails cover configured categories and supported checks. They do not guarantee that every unsafe, unsupported, or unauthorized answer is detected.

### Design implications

Guardrail configuration must match the application use case and API integration. Test expected blocks as well as expected passes. Review false positives with the content owner, and review missed risks with the safety owner. Keep policy version and outcome metadata for audits. Do not store sensitive examples beyond the approved test controls. An output check can reduce harmful responses, but it cannot correct inaccurate source data or weak authorization. Use separate controls for content safety, data access, and answer grounding.

### Invariant

Safety checks must run at the relevant boundary, and upstream authorization cannot be delegated to an output filter.

### Controlled experiment

1. Select an input and output policy test set with expected allow or block results.

2. Change one guardrail policy threshold or category setting supported by the configured API.

3. Run the same cases and record blocks, passes, and false decisions.

4. Keep retrieval authorization tests separate from content policy tests.

The setting change can alter which content the guardrail blocks or allows. It cannot repair an upstream access leak.

Tune policy against reviewed cases. Keep authorization enforcement before retrieval results enter model input.

### Transfer check

Predict why an output guardrail alone cannot prevent an unauthorized record from entering the prompt.

The output check occurs after retrieval. The retrieval path must enforce authorization before it sends a record to the model.

## Test a guardrail configuration

Evaluate configured input and output checks with approved test cases.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Review the guardrail configuration and attached policy version.

Reason. The lab needs a known policy baseline.

Evidence. Policy categories and version are recorded.

2. Select expected-pass and expected-block examples.

Reason. Both sides reveal false positives and missed controls.

Evidence. Expected outcomes are approved before invocation.

3. Call ApplyGuardrail with the test content and selected guardrail version.



Reason. The API returns policy assessments without requiring model inference.

Evidence. The response records the action and any assessments.

4. Test output inspection separately from input screening.

Reason. The controls apply at different points in the request path.

Evidence. The result identifies which boundary was evaluated.

5. Review grounding or sensitive-data checks only where configured.

Reason. Guardrail features depend on integration and policy setup.

Evidence. The test report records supported checks.

6. Compare observed outcomes with expected policy decisions.

Reason. A setting is useful only against its intended cases.

Evidence. False blocks and missed cases have owners.

### Expected evidence

1. Policy version and API path are documented.

2. Input and output checks are distinguished.

3. Expected and observed outcomes are compared.

### Failure checks

1. Check whether the selected API path applies the guardrail.

2. Check upstream authorization before retrieval content enters context.

3. Check false positives and missed blocks separately.

### Cleanup

1. Do not modify a production guardrail.

2. Use approved test content and remove temporary artifacts.

### Exam transfer

An output guardrail cannot correct an unauthorized retrieval result that already reached the model.

Sources [34, 35, 36].

Sources [34, 35, 36].

# Chapter 17. Privacy and IAM

Privacy controls protect data use, while IAM controls who can call services and resources.

### The smallest unit

A request can include personal, confidential, or regulated data. The application should send only the data needed for the task. AWS Identity and Access Management policies define which principals can perform service actions on resources. These are related controls with different jobs.

## How the mechanism works

Use least privilege for model invocation, data access, logging, and administration. Separate roles where workloads have different access needs. Protect data in transit and at rest through supported service controls. Review retention and logging behavior before sending sensitive content. Do not assume that a model endpoint enforces document-level authorization for a RAG application.

## Where it fits

RAG authorization should derive from trusted user identity and source permissions. Apply filters before content reaches generation. A prompt that says “do not reveal private data” cannot replace access control. Review tool permissions with the same care as database permissions.

## Failure and control

Minimize sensitive data in logs and evaluation sets. Define retention and access rules. Verify controls in the deployed path, including support staff and diagnostic access.

## Causal worked trace

Causal trace: A user authenticates to an application. The application derives the user’s trusted identity and queries a data source with scoped permissions. Retrieval filters limit candidate records. The application sends allowed passages to the model. Logs and storage follow separate access and retention controls. If a shared role searches all records without user filters, the model can receive data beyond the caller’s rights.

## Failure diagnosis

Failure diagnosis: Trace principal, role assumption, data query, filter source, retrieved records, model input, and logs. Locate the first point where access exceeds policy. Check whether user-supplied filter values can override trusted identity. Review log destinations for sensitive payload exposure.

## Transfer case and misconception

Transfer case: A multi-tenant knowledge assistant must isolate departments. Bind each authenticated user to an approved department claim. Apply that claim to retrieval through a trusted control. Test with paired allowed and denied documents. Inspect the final model input, not only the user-visible answer.

Misconception correction: A prompt that tells the model not to reveal records is not access control. Privacy minimization and IAM authorization solve different parts of the problem.



## Design implications

Map data flows before choosing privacy controls. Identify what enters the prompt, what reaches logs, where outputs persist, and which principals can access each store. Use only the fields needed for the task. Define retention and deletion behavior for prompts, outputs, traces, and evaluation data. IAM policies should scope service actions and resources. Application authorization should scope records for each user. Review both controls together because a tightly scoped AWS role can still expose too many records through application logic.

## Invariant

Only data authorized for the authenticated caller may enter the model request or its accessible logs.

## Controlled experiment

1. Create two test identities with different permitted metadata partitions.

2. Change the retrieval filter to derive its partition from authenticated identity.

3. Ask each identity for a record in both partitions.

4. Inspect retrieved results and model inputs for cross-partition content.

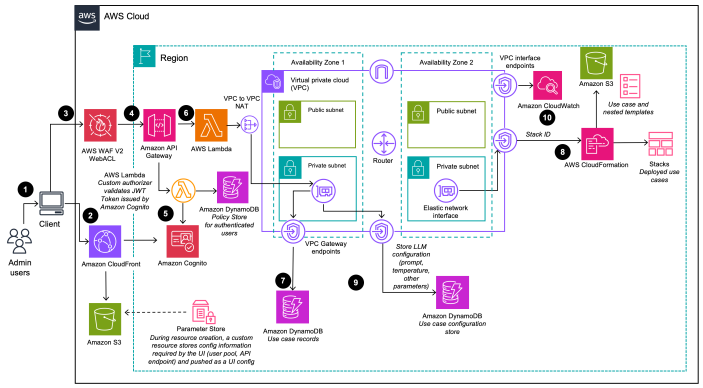
Correct identity-bound filters return only records allowed for each identity.

If any denied content reaches model input, block release and repair authorization before evaluating answer text.

## Transfer check

Predict the exposure if a shared retrieval role searches every document for every user.

The model can receive records the user cannot access. Restrict retrieval with trusted identity and authorization filters.




Application Builder dashboard with VPC option

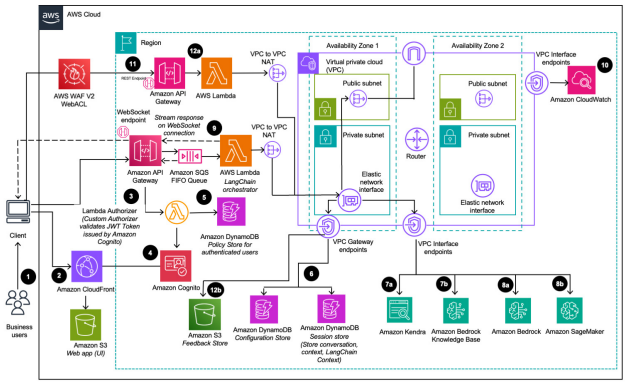
Trace network boundaries and service connections.

1. Locate the VPC boundary and the dashboard services inside or outside it.

2. Compare this deployment with the VPC-disabled diagram.

This diagram belongs to the named solution. Its detailed labels may need full-size viewing.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html



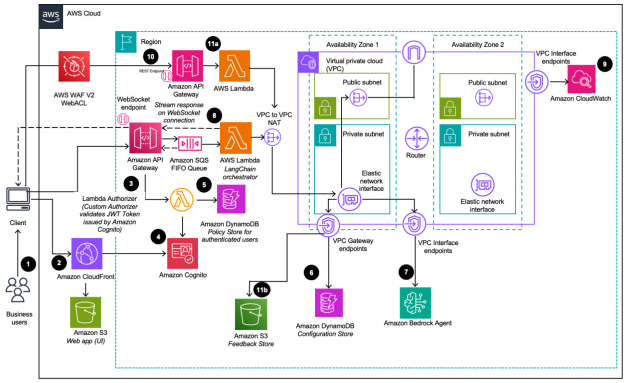

Application Builder text use case with VPC option

Trace network boundaries and the text inference path.

1. Locate the VPC boundary and the text request path.
2. Compare this diagram with the VPC-disabled text use case.

This diagram belongs to the named solution. Some labels need full-size viewing.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html




Application Builder Bedrock Agent use case with VPC option

Trace network boundaries and agent service connections.

1. Locate the VPC boundary and the Bedrock Agent request path.
2. Compare this diagram with the VPC-disabled agent use case.

This diagram belongs to the named solution. Some labels need full-size viewing.

https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/architecture-overview.html



## Trace data access and privacy

Map identity, retrieval scope, model input, logs, and retention for a test request.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Draw the request data path from sign-in to response storage.

Reason. Privacy review needs every data copy and boundary.

Evidence. The map identifies input, retrieval, inference, and logs.

2. Review IAM permissions for the application role.

Reason. The role controls service-level access.

Evidence. Allowed actions and resources are listed.

3. Run an allowed retrieval request using a test identity.

Reason. The test checks the permitted path.

Evidence. Returned records match the identity scope.

4. Run a denied-record query with the same identity.

Reason. The negative case tests access enforcement.

Evidence. Denied content is absent from retrieved and model input.

5. Inspect configured logging and retention destinations.

Reason. Logs can create another sensitive data store.

Evidence. Content, metadata, access, and retention are documented.

6. Record required data minimization or deletion actions.

Reason. The lab should leave no unnecessary test data.

Evidence. A cleanup owner and retention path are identified.

### Expected evidence

1. Trusted identity controls service and record access.
2. Denied data does not reach model context.
3. Logs and retained data follow approved policy.

### Failure checks

1. Check for client-supplied filter values.
2. Check the full model request for restricted content.
3. Check log permissions and retention settings.

### Cleanup

1. Delete test records only when permitted by the source owner.
2. Do not change shared IAM policies.

### Exam transfer

A safe prompt cannot replace an identity-bound retrieval filter.

Sources [5, 6, 11].

Sources [5, 6, 11].

# Chapter 18. Governance

Governance assigns owners and evidence for model, data, and application decisions.

### The smallest unit

An AI application combines models, data, prompts, tools, and deployment settings. Each can change behavior or risk. Governance defines who approves these changes, which records teams keep, and how they respond to incidents.

### How the mechanism works

Maintain a record of model choice, use case, data sources, evaluation results, prompt versions, guardrail settings, owners, and release approvals. Keep the record useful for the risk and regulatory setting. Do not collect sensitive prompt content when metadata can answer the audit question.

### Where it fits

Set boundaries for acceptable use and human review. Define how users report an incorrect or unsafe answer. Specify how teams suspend a model route or data source. Governance must connect to actual release and operational controls, or it remains a document with no enforcement.

### Failure and control

Review material changes to model, prompt, retrieval, tools, or data. A change in one layer can affect another. Keep a known-good version and a rollback path.

### Causal worked trace

Causal trace: A model, prompt, dataset, and tool policy receive named versions. A change request identifies an owner and risk. Evaluation runs against the candidate versions. An approver reviews results and policy evidence. The release process deploys the approved set and retains a rollback target. If a prompt changes outside version control, a later incident cannot be tied reliably to the evaluated configuration.

### Failure diagnosis

Failure diagnosis: Compare deployed and approved versions for model route, prompt, source data, guardrails, and tools. Identify the owner of each change. Check whether release evidence used representative cases. A missing record is a governance gap, while a poor answer can also be a technical defect.



### Transfer case and misconception

Transfer case: A team adds a write tool to an agent. Update the system inventory, threat review, tool permissions, evaluation cases, human approval policy, and release record. Verify that the tool owner can disable the route during an incident.

Misconception correction: A governance document alone does not enforce policy. Controls need owners, release gates, technical checks, and a response path.

### Design implications

Governance should scale with impact and data sensitivity. A low-risk internal summary may need fewer approval steps than an agent that changes financial records. Define an owner for each risk and an incident path. Keep records that help reproduce a decision, such as model route, prompt version, data source version, and evaluation result. Review vendor and service changes that can affect the system. Retire unused models and tools so stale permissions do not remain available.

The record should support an incident review without collecting more user data than needed. Limit access to review artifacts and define when teams must remove them.

### Invariant

A production behavior change must map to an identifiable version, owner, approval, and evidence set.

### Controlled experiment

1. Select a released prompt and evaluation record.

2. Change the prompt resource version without updating the release record in a test branch.

3. Run the release check and inspect whether the version mismatch is detected.

4. Update the record and repeat the check.

A version-aware release gate should flag the untracked prompt change before promotion.

Make prompt and configuration versions explicit release inputs. Keep the gate tied to the deployed artifact.

### Transfer check

Predict why an untracked prompt edit can block incident analysis.

The team cannot link the response to the prompt version that shaped it. Version prompts and associate versions with evaluation and release records.

## Build a governance record

Create a traceable record for one illustrative GenAI release decision.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Record the use case, owner, data sources, and intended users.

Reason. Scope and ownership support review.

Evidence. A release record identifies the system boundary.

2. Record model route, prompt version, retrieval setup, and guardrails.

Reason. These settings can change output behavior.

Evidence. Configuration identifiers are linked to the release.

3. Attach relevant evaluation results and known limits.

Reason. Evidence supports the release decision.

Evidence. The record names tested cases and unresolved risks.

4. Identify who approves release and who responds to incidents.

Reason. A policy needs accountable owners.

Evidence. Approval and response roles are explicit.

5. Check whether the record contains unnecessary user content.

Reason. Governance records should minimize sensitive data.

Evidence. The record keeps metadata needed for audit.

6. Review how rollback or route disablement would occur.

Reason. Incident response needs an operational control.

Evidence. A named owner can identify the recovery action.

### Expected evidence

1. The record ties configuration to evaluation and owner decisions.
2. Sensitive user content is minimized.
3. Incident and rollback responsibilities are stated.

### Failure checks

1. Check whether prompt and model versions match test evidence.
2. Check for undocumented data sources or tools.
3. Check record access and retention.

### Cleanup

1. Store the record in an approved location.
2. Remove unnecessary test content before retention.

### Exam transfer

An untracked prompt edit prevents reliable reconstruction of an answer path.

Sources [1, 6, 30].

Sources [1, 6, 30].



# Chapter 19. Caching and cost

Caching can reuse stable work, while cost control requires visibility into request components.

### The smallest unit

Inference cost can depend on input, output, model, and service configuration. Retrieval, storage, evaluation, and orchestration also have costs. A low-cost model call can still create an expensive system if prompts are long or agents make many calls.

### How the mechanism works

Prompt caching can reduce repeated processing for supported model and API configurations. Cache behavior, eligible content, and billing rules vary by model. Keep stable prompt prefixes consistent where caching applies. Verify cache metrics instead of assuming reuse. Never allow caching to cross user or data boundaries in a way that exposes private content.

### Where it fits

Track request volume, input and output use, cache use, retrieval work, and retries. Set budgets or quotas at the right application boundary. Reduce unnecessary context before reducing answer quality. Route simple tasks to an adequate lower-cost model only after evaluation.

### Failure and control

Use cost per successful task as a decision measure. This includes retries, review, and correction. Compare designs with the same task set and quality threshold.

### Causal worked trace

Causal trace: A request repeats a stable prompt prefix and adds a changing user question. A supported model path can reuse eligible prefix computation through prompt caching. The service reports cache-related usage where supported. The application pays for remaining input, output, retrieval, and orchestration work. If the changing content appears before the stable prefix, cache reuse can decline.

### Failure diagnosis

Failure diagnosis: Check model support, API configuration, cache eligibility, prefix order, and returned usage signals. Compare a repeated stable prefix with a changed prefix. If cost remains high, inspect total request volume, output length, retrieval, retries, and agent calls. Do not assume caching removes the cost of all work.

### Transfer case and misconception

Transfer case: A system uses a long stable policy prompt for many requests. Place stable instructions before request-specific data when the selected model's caching behavior supports that pattern. Exclude private user data from cross-user cache assumptions. Measure cache use and answer quality after the change.

Misconception correction: Cache reuse is conditional on supported models and request details. It is neither automatic for every request nor permission to share private context.

### Design implications

Cost control begins with attribution. Associate requests with an application, task type, and model route where policy permits. Compare cost per accepted task instead of raw cost per call. A system with retries or agent loops can make several calls for one user request. Cache only eligible stable content, and protect isolation across users. Reduce irrelevant retrieved text and repeated history before lowering model capability. Review cache behavior after prompt edits because a small prefix change can affect reuse.

### Invariant

Caching can reduce repeated eligible processing, while each request still requires correct isolation and full cost accounting.

### Controlled experiment

1. Choose a supported model and record current prompt token usage and cache metrics.

2. Move the stable instruction block before the variable user content without changing text.

3. Repeat the same request sequence and record cache signals, latency, and answer quality.

4. Compare with a sequence that changes the stable prefix.

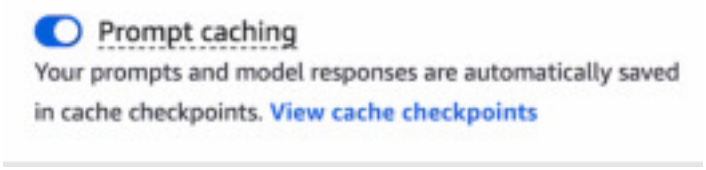
A stable repeated prefix can improve reuse when the model and API support the cache path.

Retain the ordering only when measured reuse improves and request isolation remains correct.

### Transfer check

Predict why a longer prompt can raise cost even when the output stays short.

The request still processes more input context. Reuse stable content where supported and remove irrelevant context.






Enable prompt caching in the playground

Check model support and the current API path before you use this setting. Prompt caching reuses prompt content, not a final application answer.

1. The setting label is Prompt caching.

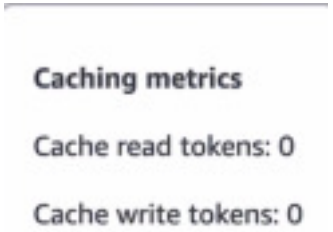
2. The toggle is on in this screenshot.

3. The panel says prompts and model responses are saved in cache checkpoints.

4. The panel includes View cache checkpoints.

This is a static AWS documentation screenshot. Model support and console labels vary. The screenshot does not guarantee a cache hit.

https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html




Read prompt cache token metrics

Inspect read and write token counts after each model response. Treat the values in this screenshot as example values.

1. The panel heading is Caching metrics.

2. The panel lists Cache read tokens.

3. The panel lists Cache write tokens.

4. Both values are zero in this example image.

This is a static AWS documentation screenshot. The zero values are not a measured run. Metric availability varies by model and API.

https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html

## Measure prompt cache behavior

Compare repeated stable prompt prefixes on a supported model path.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Verify cache support and requirements for the selected model.

Reason. Caching behavior varies by model and request pattern.

Evidence. The supported setup is recorded from current documentation.

2. Prepare a stable prefix and separate variable request content.

Reason. Stable content is the candidate for reuse.

Evidence. The prompt boundaries are visible.

3. Send repeated requests with the same prefix.

Reason. Repeated use tests cache behavior.

Evidence. Usage or cache indicators are captured when returned.

4. Change the prefix in a separate trial.

Reason. A changed prefix tests reuse sensitivity.

Evidence. The request versions are recorded.

5. Compare cache signals, latency, and answer quality.

Reason. Reuse should not harm task behavior or isolation.

Evidence. The comparison records each measure.

6. Review cost and data isolation before adopting the pattern.

Reason. Caching does not remove all cost or privacy duties.

Evidence. The adoption decision names eligible content and boundaries.

### Expected evidence

1. Cache support is confirmed for the chosen path.
2. Stable and changed prefix trials are distinguishable.
3. Adoption considers quality and data isolation.

### Failure checks

1. Check model support and required cache controls.
2. Check response usage indicators.
3. Check that user-specific data is not shared across requests.

### Cleanup

1. Do not change production prompts for this test.
2. Remove local prompt test data under policy.

### Exam transfer

Caching can help stable repeated input, but does not eliminate output or system costs.

Sources [37, 17, 30].

Sources [37, 17, 30].

# Chapter 20. Throughput and latency

Throughput measures completed work over time, while latency measures time for one request.

### The smallest unit

Users experience latency from the full request path. It can include authentication, retrieval, model queueing, generation, tool calls, and network transfer. Throughput depends on service capacity, request shape, concurrency, and downstream limits.



### How the mechanism works

Streaming can show early output sooner without reducing total generation time. A smaller prompt can reduce input work. Parallel retrieval can reduce sequential wait when dependencies allow it. More concurrency can increase throughput until a shared limit or bottleneck appears. Measure under realistic request sizes and traffic patterns.

### Where it fits

Use percentiles to understand slow requests, not only averages. Separate time spent in retrieval, model calls, tools, and application code. Apply timeouts that fit the user task. Avoid unbounded concurrency or retries because they can worsen overload.

### Failure and control

Load tests should include normal and peak patterns, cold paths, large requests, and service errors. Confirm current quotas and capacity plans in the relevant account and Region.

### Causal worked trace

Causal trace: A request enters the application, passes retrieval, calls a model, and may call tools. Each stage adds time. Streaming can expose the first model output before generation ends. Concurrency increases active work. When a downstream quota or capacity limit is reached, requests queue or fail. More clients can then increase tail latency instead of throughput.

### Failure diagnosis

Failure diagnosis: Break latency into application, retrieval, model, tool, and network spans. Compare percentiles and error rates under realistic load. If only high concurrency degrades, inspect queues and quotas. If time to first token improves but total time does not, streaming changed perceived latency rather than total work.

### Transfer case and misconception

Transfer case: A help desk sees long waits during peak traffic. Measure the full path at normal and peak request rates. Test a smaller prompt and bounded concurrency separately. Confirm current account quotas and deployment capacity before choosing a scaling strategy.

Misconception correction: More parallel requests do not always increase completed work per second. Shared limits can create queueing, throttling, and retry amplification.

### Design implications

Latency budgets should reflect user needs and dependency behavior. Measure time to first output separately from time to final completion. A streaming design can improve the first measure while total work remains unchanged. Load tests should preserve realistic prompt sizes and retrieval patterns. Include throttling and timeout behavior. Scale one bottleneck at a time. If model time dominates, application parallelism may not help. If retrieval dominates, changing model capacity may have little effect.

Keep load-test results with the tested model route and request profile. A result from a different prompt size or Region may not predict production behavior.

Include a quota test and a recovery test after the load test.

### Invariant

Throughput improves only while added concurrency stays within the capacity of every required dependency.

### Controlled experiment

1. Set a representative request mix and fixed request rate.

2. Change the application concurrency limit from baseline L to a higher bounded value L2.

3. Measure completion rate, latency percentiles, throttling, and errors.

4. Repeat at a lower limit to identify the saturation point.

Completion rate may rise before saturation, then level off or fall as queues and throttling increase.

Choose a concurrency limit from measured completion rate and latency, with headroom for dependency limits.

### Transfer check

Predict why higher concurrency may worsen both latency and error rates.

A downstream service can saturate. Requests queue or fail, so control concurrency and observe each dependency.

## Profile latency and concurrency

Separate time to first output, total latency, and throughput under controlled load.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Record request sizes and stage timing for a baseline run.

Reason. A baseline identifies the dominant latency stage.

Evidence. The run records request shape and stage times.

2. Send a bounded workload at the baseline concurrency.

Reason. The baseline measures completed work and errors.



Evidence. Completion rate and latency percentiles are saved.

3. Increase concurrency within approved sandbox limits.

Reason. A controlled change tests saturation behavior.

Evidence. The test records rate, throttling, and latency.

4. Compare time to first content with total completion time.

Reason. Streaming may change perceived delay without changing total work.

Evidence. Both timing measures are reported.

5. Inspect errors and dependency spans at each load level.

Reason. Averages can hide saturation and retries.

Evidence. The trace points to the constrained dependency.

6. Stop the test when the approved load boundary is reached.

Reason. Unbounded tests can create cost or service impact.

Evidence. The workload ends within the approved envelope.

### Expected evidence

1. The workload and concurrency levels are documented.

2. Latency and throughput measures are distinct.

3. A bottleneck is identified from stage evidence.

### Failure checks

1. Check account quotas and endpoint capacity first.

2. Check retry amplification at higher concurrency.

3. Check request token lengths, not only request counts.

### Cleanup

1. Stop the test client and load generator.

2. Do not purchase throughput or create endpoints for this lab.

### Exam transfer

More concurrency can reduce service quality after a dependency saturates.

Sources [33, 38, 4].

Sources [33, 38, 4].

# Chapter 21. Observability

Observability links request traces, service metrics, and logs to explain system behavior.

### The smallest unit

Metrics show rates and distributions. Logs record selected events. Traces connect stages in one request. Together they help locate delay, failure, and quality changes. Model text can be sensitive, so observability must include data minimization.

### How the mechanism works

Record a request identifier, model or route, prompt version, retrieval outcome, tool names, latency, usage, and error class where policy allows. Amazon Bedrock model invocation logging can send invocation data to configured CloudWatch Logs or Amazon S3 destinations. Review what content enters those destinations and who can access it.

### Where it fits

A trace should distinguish retrieval failure from generation failure. It should also show retries and partial stream outcomes. Avoid treating model output alone as a diagnostic record. Keep retention and access aligned with the data classification.

### Failure and control

Use dashboards for service health and sampled review for answer quality. Alert on actionable conditions such as elevated error rate, latency, or empty retrieval. Do not alert on a metric without an owner and response path.

### Causal worked trace

Causal trace: The application creates a trace identifier and sends a request. The gateway records route and policy result. Retrieval records safe source identifiers and timing. The model call records route, usage, and outcome. The response path records completion or error. These linked events let an operator locate a failure without storing every sensitive prompt or answer.

### Failure diagnosis

Failure diagnosis: Check trace continuity across application, gateway, retrieval, model, and tool spans. A missing span indicates instrumentation or propagation gaps. A latency spike points to the slow stage. A quality issue needs evaluation evidence and prompt or retrieval versions. Logs without versions cannot explain behavior changes reliably.

### Transfer case and misconception

Transfer case: A user reports a stale answer. Use the request identifier to inspect the route, prompt version, retrieval sources, sync age, and model outcome. Limit access to sensitive logs. Compare the retrieved source version with the authoritative object before changing model settings.

Misconception correction: More logging is not automatically better observability. Unneeded prompt capture can raise privacy risk without locating the failure boundary.



### Design implications

Telemetry should answer specific operational questions. Decide which fields identify a route, version, stage, error, and outcome. Avoid high-cardinality labels that make metrics hard to use. Protect trace access because even metadata can reveal sensitive activity. Set retention based on incident and audit needs. Sample routine requests when full traces are costly, while preserving error traces when policy allows. Ensure operators can find the trace from a user report without exposing the report to every support role.

CloudTrail can support account activity review, while invocation logs support request diagnosis. Select each source for its operational purpose and retention policy.

### Invariant

A request must have enough safe, linked telemetry to locate its failing stage and configuration.

### Controlled experiment

1. Choose one synthetic request and enable a trace identifier across application stages.

2. Add a retrieval timing and source identifier event without full passage text.

3. Run the request with an induced retrieval delay in a test environment.

4. Inspect whether the trace identifies retrieval as the delayed stage.

Linked stage timing should locate the delay while avoiding full sensitive content in logs.

Keep telemetry fields that answer an operational question. Add content capture only under approved need and controls.

### Transfer check

Predict what diagnostic gap remains if logs record only the final answer.

The team cannot see which evidence, prompt, model route, or tool result shaped the answer. Record safe request metadata across stages.

## Trace an invocation

Follow one request across model, retrieval, gateway, and log records.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Create or record a request correlation identifier in the test client.

Reason. A shared identifier links stages.

Evidence. The identifier appears in the client record.

2. Send one request through the application path.

Reason. A single request limits trace ambiguity.

Evidence. The application records route and outcome.

3. Review existing model invocation logging settings in the AWS Console.

Reason. The settings show what data may be retained.

Evidence. Log destinations, fields, and access are reviewed.

4. Find the request across available service logs or traces.

Reason. Correlation can reveal the failing stage.

Evidence. The record links to invocation metadata.

5. Compare stage timing, model route, and error state.

Reason. These fields locate operational defects.

Evidence. A diagnosis names the first failing boundary.

6. Remove or restrict sensitive trace content under policy.

Reason. Observability must respect data controls.

Evidence. Access and retention follow the test plan.

### Expected evidence

1. A request can be located across available telemetry.
2. The operator can identify its route and outcome.
3. Sensitive content exposure is reviewed.

### Failure checks

1. Check log delivery configuration and permissions.
2. Check whether the correlation identifier propagates.
3. Check CloudTrail activity separately from invocation content.

### Cleanup

1. Disable temporary test logging only if this lab created it.
2. Do not delete shared audit records.

### Exam transfer

Use a source revision and prompt version to explain a stale answer from a trace.

Sources [30, 39, 21].

Sources [30, 39, 21].

# Chapter 22. Evaluation

Evaluation tests whether a system meets defined quality and safety requirements.

### The smallest unit

A model output can sound plausible and still be wrong. Evaluation starts with explicit criteria and representative examples. A dataset should cover normal requests, edge cases, and known failure risks. Human review may be needed for subjective judgments.



### How the mechanism works

For RAG, separate retrieval measures from answer measures. Retrieval metrics can assess whether relevant evidence appears among results. Answer checks can assess correctness, relevance, completeness, and grounding. Safety evaluation can test configured policy boundaries. Use the same versioned test set to compare changes.

### Where it fits

Automated metrics are proxies. They can disagree with expert judgment or reward superficial patterns. Calibrate them against reviewed examples. Record model, prompt, data, and configuration versions with the result. Do not claim quality from a small or unrepresentative sample.

### Failure and control

Run evaluation before and after changes to model, prompt, chunking, retrieval, or guardrails. Investigate regressions by category. A higher average score can hide a severe failure on a critical use case.

### Causal worked trace

Causal trace: An evaluation dataset pairs questions with expected evidence or outcomes. A retrieval-only run returns candidate passages. Metrics assess relevance and coverage. A retrieve-and-generate run also evaluates the answer, including correctness, completeness, faithfulness, or citation behavior. If retrieval coverage is poor, answer changes alone can hide the upstream defect.

### Failure diagnosis

Failure diagnosis: Separate retrieval metrics from generated answer metrics. Inspect examples where context relevance is high but coverage is low. Inspect faithfulness failures when evidence is present. Confirm evaluator limits and human-reviewed examples. An aggregate score can hide critical failure categories or an unrepresentative dataset.

### Transfer case and misconception

Transfer case: A team changes chunking. Run retrieval-only evaluation first to compare whether expected source passages appear. Then run retrieve-and-generate evaluation to assess answer impact. Keep dataset, model, and prompt fixed for the comparison, then inspect examples behind metric changes.

Misconception correction: A single evaluation score does not prove a system is reliable. Metrics are measures with scope, assumptions, and blind spots.

### Design implications

Evaluation sets need stable versions and clear expected evidence. Include cases that expose known risks, such as stale sources, missing context, ambiguous questions, and policy exceptions. Keep a holdout set for decisions when repeated tuning may overfit a development set. Record evaluator model and configuration. Review samples where automated metrics disagree with human judgment. A metric can track a trend, but an owner must decide whether the change matters for the intended user and risk.

Do not tune against the final holdout set. Preserve that set for a later check after the chosen configuration is stable.

Document the reason for each acceptance threshold and the owner who approved it.

### Invariant

A quality claim must match the evaluated task, data, configuration, metric, and known limits.

### Controlled experiment

1. Select a fixed dataset with questions and expected supporting passages.

2. Change the retrieval result count from baseline K to K2.

3. Run retrieve-only evaluation and record context relevance and coverage.

4. Run retrieve-and-generate evaluation on the same cases and inspect faithfulness and citations.

A higher result count may improve coverage while lowering relevance or increasing context noise.

Choose K from both retrieval and answer measures. Review individual failures before accepting an aggregate improvement.

### Transfer check

Predict why answer accuracy alone does not diagnose a RAG failure.

The answer can be wrong because retrieval missed the evidence or because generation misused it. Measure both stages.

## Evaluate retrieval and answer stages

Run separate checks for retrieved context and generated response quality.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Select a fixed dataset with questions and expected evidence.

Reason. Stable cases support comparison.

Evidence. Each test maps to expected passages or outcomes.

2. Run retrieve-only evaluation for the test set.

Reason. This isolates retrieval quality.



Evidence. Context relevance and coverage results are captured when supported.

3. Review cases with missing or irrelevant evidence.

Reason. Metrics need example-level diagnosis.

Evidence. Failure cases identify filters, chunks, or query issues.

4. Run retrieve-and-generate evaluation on the same cases.

Reason. This tests answer behavior with retrieved context.

Evidence. Answer metrics and citations are captured.

5. Compare answer claims with retrieved passages.

Reason. Faithfulness and citation quality need evidence review.

Evidence. The review records unsupported or missing claims.

6. Repeat after one controlled retrieval change.

Reason. A single change helps explain metric movement.

Evidence. The comparison ties outcomes to configuration versions.

### Expected evidence

1. Retrieval and answer results are separate.

2. Example failures are reviewed.

3. The test records dataset and configuration versions.

### Failure checks

1. Check evaluator coverage and ground truth quality.

2. Check whether retrieval misses evidence before blaming generation.

3. Check for score changes hidden by aggregate results.

### Cleanup

1. Retain approved evaluation data under its data policy.

2. Remove temporary result exports when no longer needed.

### Exam transfer

A prompt improvement cannot repair evidence that the retriever never returned.

Sources [28, 21, 16].

Sources [28, 21, 16].

# Chapter 23. Troubleshooting

Troubleshooting locates the first failed boundary and tests a targeted cause.

### The smallest unit

A model application crosses boundaries: client, identity, API, model access, retrieval, network, and output parsing. Start with the error type and request identifier. Compare expected and actual request shape. Confirm the caller, Region, model route, and permissions.

### How the mechanism works

Access denied points toward identity or resource policy. Throttling points toward quota or burst behavior. Validation errors point toward request shape or unsupported settings. A timeout can mean the client ended before the service completed. A service error can be transient, but repeated retries can hide the real cause.

### Where it fits

Reproduce with the smallest request that preserves the fault. Change one variable at a time. Check current service documentation for the named error and model support. Keep secrets out of diagnostic output. Confirm whether any side effect completed before retrying a timed-out tool action.

### Failure and control

After a fix, test the original path and the failure case. Record the root cause and prevention control. A workaround that increases retries may reduce symptoms while raising cost and load.

### Causal worked trace

Causal trace: A client sends a request with a model identifier and body. The service returns an error class and request information. The application classifies the error as access, validation, throttling, timeout, or service failure. It checks configuration or applies a bounded retry only where appropriate. If the same invalid parameter is retried, the same validation failure repeats without progress.

### Failure diagnosis

Failure diagnosis: Read the error class before changing code. Access denial requires identity or policy review. Validation requires payload or model setting correction. Throttling requires quota and rate handling. Timeouts require checking completion state. Service errors may be transient. Preserve request identifiers while removing secrets from diagnostics.

### Transfer case and misconception

Transfer case: An application sees throttling during a burst. Inspect attempt rates and account quota. Apply bounded backoff with jitter and cap concurrent work. Re-test under expected traffic. If errors persist at normal demand, review quota or capacity rather than increasing retries.

Misconception correction: Retrying every error does not improve resilience. It can repeat permanent failures and amplify load during an outage.



### Design implications

Use a decision tree that starts with error class and request state. Confirm whether the service received the request and whether an external action may have completed. Capture request identifiers and relevant configuration while redacting credentials and personal data. Check service status and quotas only when the error points there. Change one cause at a time. Document the verified root cause and prevention step. This avoids turning a temporary workaround into permanent retry behavior.

Separate client-side timeouts from service errors. A client timeout does not prove that the service stopped processing the request.

### Invariant

The recovery action must match the diagnosed error class and the operation's repeat safety.

### Controlled experiment

1. Use a test request that returns a known throttling response.

2. Set a bounded retry policy with backoff for throttling only.

3. Repeat with a validation error and confirm that it receives no retry.

4. Record attempts, final result, and elapsed time.

The throttled request can recover when service capacity returns. The invalid request stops after its first validation failure.

Keep error-specific retry rules and bounded attempts. Correct permanent request or permission defects directly.

### Transfer check

Predict the right first action for a validation error caused by an unsupported parameter.

Remove or correct the parameter according to the model API. Retrying the same request will repeat the same failure.

## Classify and resolve API failures

Use error class and request state to choose a recovery action.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Capture request ID, model route, and error class from a failed call.

Reason. These details support targeted diagnosis.

Evidence. The error record omits credentials and sensitive content.

2. Classify the failure as validation, access, throttling, timeout, or service error.

Reason. Each class has a different likely cause.

Evidence. The classification matches the returned error.

3. Check request shape for validation failures.

Reason. A permanent payload defect will repeat.

Evidence. Unsupported fields or missing requirements are identified.

4. Check IAM and model access for authorization failures.

Reason. Access policy determines permitted actions.

Evidence. The caller and required permission are compared.

5. Review rate and bounded backoff for throttling.

Reason. Transient capacity errors may need controlled retry.

Evidence. Attempts and delay are documented.

6. Check operation state before retrying after an ambiguous timeout.

Reason. A timeout may hide successful completion.

Evidence. The status check prevents duplicate side effects.

### Expected evidence

1. The recovery action matches the error class.
2. Permanent failures are not blindly retried.
3. Timeout recovery checks operation state.

### Failure checks

1. Check if the request reached the service.
2. Check access and Region before changing payload.
3. Check retries do not amplify the fault.

### Cleanup

1. Do not repeat consequential operations without status confirmation.
2. Remove local diagnostic files containing sensitive details.

### Exam transfer

For a validation error, correct the request instead of retrying it.

Sources [4, 3, 5].

Sources [4, 3, 5].

# Chapter 24. CI/CD for AI applications

CI/CD applies repeatable checks and controlled release steps to AI system changes.

### The smallest unit

An AI application has code plus behavior configuration. Prompts, evaluation data, retrieval settings, guardrails, model routes, and infrastructure can all change output. A release process should identify each changed component and its owner.



### How the mechanism works

Use source control for prompts and infrastructure where possible. Run schema checks, security checks, and evaluation before release. Deploy to a test environment with representative data controls. Promote a version only when it meets the release criteria. Keep the prior version available for rollback.

### Where it fits

Model behavior can change when a provider updates a model or when the application changes its context. Pin supported versions where the service offers that control. Record the exact route and settings used in evaluation. Use staged rollout for changes with broad user impact.

### Failure and control

A deployment pipeline cannot prove every answer is safe. It can make known checks repeatable and expose regressions before broad release. Pair automated checks with human review for high-impact cases.

### Causal worked trace

Causal trace: A developer changes a prompt, model route, or retrieval setting. Source control records the change. CI validates schemas and runs the evaluation set. A release gate compares results with acceptance criteria. Deployment promotes the approved version. A rollback restores the prior version if production signals regress. If prompts are outside source control, code checks can pass while behavior changes unnoticed.

### Failure diagnosis

Failure diagnosis: Identify which artifact changed, then compare its evaluated version with the deployed version. Check data and model route alongside code. If CI passed but production fails, compare environment, identity, source freshness, and traffic pattern. An evaluation pass covers only its cases and must not be treated as universal proof.

### Transfer case and misconception

Transfer case: A team updates a prompt and tool schema in one release. Version both artifacts. Run schema checks, security checks, and task evaluation. Deploy to a test environment with safe representative data. Promote only after approval, and retain the prior prompt and configuration for rollback.

Misconception correction: CI/CD can repeat known checks and control release steps. It cannot prove every possible model response is correct or safe.

**Design implications**

Treat prompts, evaluation cases, retrieval configuration, guardrail settings, and infrastructure as release artifacts. A pipeline can lint schemas, run tests, compare evaluation results, and require approvals. Use environment-specific identities and data. Keep secrets in managed secret stores, not source control. Stage broad changes when practical and monitor post-release indicators. Define rollback triggers before release. A rollback must restore compatible model, prompt, index, and tool versions, not only application code.

For RAG releases, record the index or sync state used by evaluation. A prompt check against stale test data may not predict production answers.

**Invariant**

Every behavior-affecting artifact must be versioned, evaluated, and traceable to the deployed release.

**Controlled experiment**

1. Select a prompt and evaluation baseline under version control.

2. Change one prompt instruction and commit the new version.

3. Run the release pipeline and compare evaluation output with the baseline.

4. Attempt promotion with the old evaluation record, then with a record for the new version.

A version-aware gate should reject a candidate whose evaluation record does not match the changed prompt.

Bind evaluation evidence to exact artifact versions. Keep promotion and rollback tied to those versions.

**Transfer check**

Predict why a code-only pipeline misses a prompt regression.

Prompt text can change behavior without changing application code. Version and evaluate prompts as release inputs.

## Gate a prompt and route release

Tie a configuration change to evaluation evidence and a reversible release.

Proposed, not executed.

This proposed lab can incur AWS charges through inference, retrieval, logging, or service resources. Review current pricing before execution.

1. Record current model route, prompt version, and evaluation baseline.

Reason. A baseline identifies the released state.

Evidence. The source and deployed identifiers are linked.

2. Change one prompt or route configuration in a test branch.

Reason. One change supports causal review.

Evidence. The diff shows the exact changed artifact.



3. Run schema, security, and task evaluation checks.

Reason. These checks cover known contract and behavior risks.

Evidence. Results link to the changed version.

4. Compare evaluation against approved release criteria.

Reason. Promotion should depend on explicit acceptance.

Evidence. A pass or failure has a recorded reason.

5. Deploy only to a test environment for the lab.

Reason. A test release validates deployment behavior without production impact.

Evidence. The test environment runs the candidate version.

6. Confirm rollback restores the prior compatible configuration.

Reason. Recovery must cover more than application code.

Evidence. Prompt, route, and related config return to baseline.

**Expected evidence**

1. The candidate artifact has matching evaluation evidence.

2. The lab uses only a test environment.

3. Rollback restores the prior known configuration.

**Failure checks**

1. Check for prompts or settings outside source control.

2. Check evaluation data and model route match the candidate.

3. Check rollback includes compatible prompt and retrieval settings.

**Cleanup**

1. Do not deploy to production.

2. Remove test branch artifacts only under repository policy.

**Exam transfer**

A code-only pipeline misses prompt and retrieval changes that alter model behavior.

Sources [28, 25, 29].

Sources [28, 25, 29].

# All 98 skill cases

Each case adds a decision boundary, causal trace, failure test, and transfer check. These are authored study scenarios.

## 1.1.1 Architecture from requirements

Architecture begins with the user outcome, data boundary, and operating constraint. A model choice is one part of the design, not the design itself.

Translate requirements into a request path, trust boundaries, data flow, failure behavior, and measurable acceptance checks. Map each requirement to a component that owns it.

**Decision**

Use this skill when requirements span model quality, latency, data access, operations, or deployment.

Choose a managed inference path or a hosted model path from the required control and operating capacity.

The choice must meet the use case and constraints together.

Reject a model-first design that ignores identity, data freshness, or failure recovery.

**Causal trace**

1. A business owner requires answers from current internal policy.

2. The architect identifies source authority, user access, and answer latency needs.

3. The design selects retrieval, model inference, identity filters, and monitoring.

4. A test question checks evidence, access, and response behavior end to end.

**Failure test**

The design meets a feature request but fails a data or latency constraint.

Trace each requirement through a component and acceptance test.

Revise the path that fails, then rerun the requirement test.

**Experiment**

Change the retrieval path from unrestricted to identity-filtered access in an illustrative design.

Observe whether allowed questions retain evidence and denied records stay out of model input.

This checks one access boundary. It does not prove every policy or traffic condition.

An illustrative assistant must answer from current internal guidance. What must architecture specify before model choice?

Specify authoritative sources, caller identity, retrieval controls, answer checks, latency needs, and failure behavior.

Sources [40, 1, 20, 12].

## 1.1.2 Proof of concept

A proof of concept tests uncertain assumptions before a team commits to full deployment.



Keep the prototype narrow. Use representative inputs, a defined success measure, and the intended service path where possible.

**Decision**

Use this skill when feasibility, quality, service fit, or business value remains uncertain.

Build a small end-to-end slice that exercises the riskiest assumption.

A representative slice can expose integration limits earlier than a model-only demo.

Reject a polished demo that uses hand-picked inputs and no acceptance criteria.

**Causal trace**

1. An illustrative team doubts whether policy retrieval can support a support workflow.

2. It selects representative questions and expected source passages.

3. It connects a small corpus to retrieval and one inference route.

4. Reviewers compare answer evidence, errors, latency, and required operations.

**Failure test**

The demo looks useful but fails on ordinary or difficult cases.

Compare test inputs with actual workload categories and edge cases.

Expand or revise the test set before drawing a scale conclusion.

**Experiment**

Change the PoC corpus from curated examples to a representative approved sample.

Observe retrieval coverage, answer quality, and integration defects.

A PoC cannot establish production capacity or prove every risk is controlled.

Why can a successful model playground demo fail as a proof of concept?

It may omit the real data, identity, retrieval, application path, and measurable acceptance criteria.

Sources [18, 20, 28, 12].

## 1.1.3 Standardized components

Standard components reduce variation across applications while preserving explicit choices for different requirements.

Define reusable interfaces, security defaults, observability fields, deployment patterns, and review checks. Use a framework such as the Generative AI Lens to assess design areas.

**Decision**

Use this skill when teams build similar model features in multiple deployment scenarios.

Publish a reference component with documented extension points and controls.

Teams can repeat known safeguards and focus review on deliberate deviations.

Reject a single rigid template that cannot express different data or latency requirements.

**Causal trace**

1. A platform team sees repeated model invocation code.
2. It defines a shared request adapter and identity requirements.
3. Teams adopt the component and document approved variations.
4. A design review checks the shared defaults and each exception.

**Failure test**

Applications call the same model service with different unreviewed policies.

Compare deployed patterns against the component contract and approved exceptions.

Update the component or close unexplained gaps before rollout.

**Experiment**

Change an example application to use the shared invocation component.

Observe whether request validation, telemetry, and identity controls match the standard.

A standard component cannot ensure correct use if teams bypass it or supply poor data.

What should a shared component standardize, and what should remain configurable?

Standardize stable interfaces and safeguards. Keep model, data, and workload choices configurable with reviewed limits.

Sources [40, 5, 3, 12].

## 1.2.1 Foundation model choice

Choose a foundation model by demonstrated fit for a defined task and operating constraints.

Compare capability, modality, limitations, supported settings, access, Region, output quality, and latency on representative cases.

**Decision**

Use this skill when several models could perform the same application task.

Run a comparative evaluation against the workload acceptance criteria.

Measured task behavior gives stronger evidence than general reputation.

Reject a choice based only on a benchmark or a single successful prompt.



**Causal trace**

1. An illustrative team needs extraction from support messages.
2. It defines expected fields and failure costs.
3. It tests candidate models with the same examples and request shape.
4. It compares correctness, refusal behavior, latency, and supported operation.

**Failure test**

A model scores well overall but fails a high-impact task category.

Review per-category failures and current model constraints.

Choose a model that passes the critical criteria or revise the workflow.

**Experiment**

Change only the candidate model while holding the test set and prompt fixed.

Observe task quality, latency, and output validity by category.

Evaluation results apply to the tested task, prompt, and model configuration.

Why is the highest general benchmark score not enough to select a model?

It may not reflect this workload, required modality, constraints, or critical failure categories.

Sources [18, 19, 17, 12].

## 1.2.2 Provider and model switching

A flexible architecture separates application intent from provider-specific request details.

Use a stable internal request contract and adapters that map it to supported provider APIs. Centralize route configuration outside business logic where suitable.

**Decision**

Use this skill when an application must change model route or provider with limited code changes.

Introduce an adapter and controlled route configuration.

This limits code coupling and supports policy-based route changes.

Reject the claim that one common API makes model behavior identical.

**Causal trace**

1. A client sends a task through an internal model interface.
2. Configuration selects an approved provider route.
3. An adapter maps the task to that route's request format.
4. The response maps back to the application result contract.

**Failure test**

A route switch causes missing fields or different answer behavior.

Check adapter mapping, supported settings, and evaluation for the new route.

Add explicit route-specific handling and rerun contract tests.

**Experiment**

Change the configured route to a second approved model without editing business logic.

Observe request validity and answer contract behavior for both routes.

Switching still requires model-specific evaluation and may need adapter updates.

What does a provider abstraction simplify, and what does it not standardize?

It simplifies routing and request mapping. It does not standardize capability or output behavior.

Sources [3, 18, 17, 12].

## 1.2.3 Resilient inference design

Resilience keeps a useful service path available when a dependency fails or becomes unavailable.

Use timeouts, bounded retries, circuit breakers, cross-Region options, or graceful degradation according to the failure mode.

**Decision**

Use this skill when inference availability or regional capacity is a business requirement.

Choose a fallback that preserves an acceptable user outcome and policy boundary.

A controlled fallback can prevent cascading failure while exposing reduced capability.

Reject unbounded retries or silent fallback to an untested model.

**Causal trace**

1. A primary inference route returns repeated transient errors.

2. The application classifies the error and opens its circuit after the configured condition.

3. A permitted fallback route or safe degraded response serves the request.

4. Telemetry records the route and later recovery.

**Failure test**

Retries amplify load or fallback answers break the response contract.

Inspect error class, retry count, circuit state, and fallback evaluation.

Bound retries and validate each fallback against the same task needs.

**Experiment**

Change the circuit-breaker threshold or fallback route in a test configuration.



Observe recovery time, failed calls, and fallback answer suitability.

A fallback cannot guarantee identical quality, region, or feature support.

Why must a fallback route be evaluated before an outage?

It can differ in capability and output behavior, so the application must validate its safe use in advance.

Sources [4, 9, 13, 12].

## 1.2.4 Customized model lifecycle

A customized model needs controlled versioning, deployment, evaluation, and retirement.

Track training or adaptation inputs, model artifact, deployment configuration, evaluation, and rollback path. Use a registry and automated release process where applicable.

**Decision**

Use this skill when a base model does not meet a validated domain requirement and customization is justified.

Deploy a versioned customized artifact through a tested pipeline.

Traceability supports comparison, rollback, and later retirement.

Reject an untracked model replacement based only on a promising sample.

**Causal trace**

1. A domain task shows a stable capability gap after prompt and retrieval checks.

2. The team prepares a customization dataset and records its provenance.

3. It registers and deploys a candidate model version.

4. Evaluation gates promotion and preserves a rollback target.

**Failure test**

A new deployment regresses quality or cannot be tied to its training data.

Compare artifact, data lineage, evaluation, and deployment versions.

Restore the last approved version and repair the release record.

**Experiment**

Change the deployment from the current registered model version to a candidate version.

Observe task quality, endpoint behavior, and rollback readiness.

Customization does not remove the need for data rights, evaluation, or lifecycle review.

What evidence should support promotion of a customized model?

Versioned artifact and data lineage, task evaluation, deployment checks, owner approval, and rollback readiness.

Sources [15, 29, 28, 12].

## 1.3.1 Input data validation

Validation catches data defects before they affect model behavior.

Check schema, required fields, types, ranges, missing values, duplicates, and domain rules. Send failures to a visible correction path.

**Decision**

Use this skill before inference when input quality affects answer correctness or safety.

Validate at ingestion and again at the application boundary for changing inputs.

Early checks reduce downstream ambiguity and make defects easier to locate.

Reject reliance on the model to repair malformed or missing facts silently.

**Causal trace**

1. An illustrative workflow receives tabular customer requests.
2. A validation stage checks required columns and allowed formats.
3. Valid rows proceed to feature or prompt preparation.
4. Invalid rows are quarantined with an actionable error record.

**Failure test**

The model produces inconsistent answers from malformed inputs.

Inspect validation results and compare source values with prompt values.

Correct upstream records or reject invalid input before inference.

**Experiment**

Change a required-field rule from permissive to explicit rejection in a test flow.

Observe invalid-row counts and downstream answer errors.

A validation rule can reject valid exceptions if domain rules are incomplete.

Why should data checks run before inference?

They prevent malformed or missing values from becoming ambiguous model input and support a clear correction path.

Sources [41, 6, 42, 12].

## 1.3.2 Multimodal data processing

Each modality needs processing that preserves information needed by the task.

Images, audio, video, text, and tables have different parsing and representation needs. Use suitable services or model inputs for each type.

**Decision**

Use this skill when the source contains more than plain text or a specialized content form.



Build a modality-aware path and retain links to original source objects.

The path can extract the relevant structure without treating every file as plain text.

Reject lossy conversion that drops labels, timing, layout, or source identity.

**Causal trace**

1. An illustrative case receives an audio support recording and an image attachment.
2. The workflow transcribes audio and extracts relevant image content.
3. It associates both outputs with the case identifier.
4. The model receives task-relevant content with source references.

**Failure test**

The model misses a table value or an audio detail lost in conversion.

Inspect extracted content and compare it with the original artifact.

Change parsing or modality handling before adjusting the model prompt.

**Experiment**

Change the preprocessing route for one modality while keeping the task constant.

Observe extracted fields, omissions, and downstream response correctness.

Extraction quality depends on source clarity and the selected processing path.

Why should image and audio inputs use modality-aware preprocessing?

Their structure and useful signals differ from plain text and can be lost through generic conversion.

Sources [23, 41, 20, 12].

## 1.3.3 Model-specific input formatting

Valid input formatting is part of model integration, not cosmetic cleanup.

Serialize roles, content blocks, parameters, and structured data according to the selected model API. Validate the request before invocation.

**Decision**

Use this skill whenever the application sends prompts, messages, tables, or structured payloads.

Build the request from a typed internal representation and model-specific serializer.

This catches shape errors while keeping business data separate from transport details.

Reject manually concatenated JSON or assumptions that all models accept the same fields.

**Causal trace**

1. The application stores a user question and system policy separately.
2. A serializer maps them to supported message roles and content blocks.
3. The API validates and invokes the model.
4. The response parser extracts expected content and usage state.

**Failure test**

The service rejects a request or the application misreads returned content.

Compare serialized body against the API contract and model support.

Fix the serializer or response parser, then rerun boundary tests.

**Experiment**

Change one message field to the documented format for the selected API.

Observe request validation and parsed output shape.

A valid request shape does not prove the generated content is correct.

What should an application verify before it sends an inference request?

It should verify required fields, roles, content structure, supported settings, and model-specific constraints.

Sources [3, 43, 2, 12].

## 1.3.4 Input data quality enhancement

Data enhancement makes source content clearer or more consistent before inference.

Normalize formatting, extract entities, remove irrelevant markup, or reformat text while preserving meaning and provenance.

**Decision**

Use this skill when noisy or inconsistent inputs reduce model consistency.

Apply deterministic transformations first and use model assistance only for tasks that need it.

Deterministic normalization is easier to inspect and reproduce.

Reject enhancement that silently changes facts or discards source context.

**Causal trace**

1. A source contains inconsistent date formats and copied markup.
2. A normalizer standardizes dates and removes irrelevant markup.
3. An entity step extracts names while retaining source references.
4. The model receives cleaner content and a trace to the original.

**Failure test**

Cleaned text changes a value or removes a qualifying phrase.

Compare transformed fields with the original source and transformation logs.

Correct the transformation and add a preservation check.



**Experiment**

Change one normalization rule for date or markup handling.

Observe field consistency and preservation of original meaning.

Normalization can improve input form but cannot establish source truth.

How can a team improve text quality without losing evidence?

Apply traceable transformations, preserve original values and provenance, and validate semantic preservation.

Sources [41, 25, 20, 12].

## 1.4.1 Vector database architecture

A vector architecture must connect source content, embeddings, metadata, retrieval, and model context.

Choose managed knowledge base or custom storage based on integration, query, scale, filter, and operational needs. Keep source records authoritative.

**Decision**

Use this skill when semantic retrieval augments model answers over organizational data.

Design the ingestion and query path together, including permissions and update behavior.

The index is useful only when it returns authorized, relevant, current content.

Reject a vector store selection based only on similarity search support.

**Causal trace**

1. A source document is parsed and split into retrievable records.
2. Each record receives an embedding and source metadata.
3. A query retrieves records under trusted filters.
4. The application adds selected evidence and source references to the model request.

**Failure test**

Relevant documents exist but retrieval returns stale or inaccessible records.

Inspect ingestion state, metadata, filters, and retrieved passages.

Fix the broken stage instead of replacing the model first.

**Experiment**

Change the managed or custom retrieval path for one controlled dataset.

Observe update behavior, filter correctness, and evidence relevance.

Vector stores do not enforce every document rule unless the system applies those controls.

What must a vector architecture define beyond storage choice?

It must define ingestion, embeddings, metadata, access filters, retrieval, freshness, and source traceability.

Sources [20, 16, 22, 12].

## 1.4.2 Metadata framework

Metadata gives retrieved content context such as source, date, author, type, and access scope.

Define field names, types, trusted origins, allowed filters, and update rules. Preserve metadata through parsing and indexing.

### Decision

Use this skill when recency, authorship, domain, or access scope affects retrieval.

Create a small governed schema tied to real query and policy needs.

Consistent metadata supports filtering and interpretation.

Reject arbitrary tags that lack a producer or clear use.

### Causal trace

1. A policy source has a department and revision date.
2. Ingestion extracts or receives those attributes from a trusted source.
3. A query applies a caller-authorized department filter.
4. The result retains revision and source identity for answer review.

### Failure test

A relevant record is filtered out or an unauthorized record appears.

Check field mapping, source trust, and filter derivation.

Repair metadata generation and enforce filters from authenticated context.

### Experiment

Change the metadata schema to include a trusted document-status field.

Observe retrieval of current approved records and exclusion of obsolete ones.

Metadata filters work only when fields are correctly populated and supported.

Why is metadata governance part of retrieval design?

Filters and context rely on stable, trusted values that preserve source meaning and access boundaries.

Sources [24, 10, 20, 12].

## 1.4.3 High-performance vector architecture

Search performance depends on index design, query load, data shape, and retrieval requirements.

Use workload measurements to guide sharding, index separation, and hierarchical approaches where supported. Keep relevance and freshness checks alongside speed.



### Decision

Use this skill when retrieval scale or response time threatens an application objective.

Measure the bottleneck first, then adjust the relevant index or query structure.

A targeted change avoids adding complexity where the model or network is actually slow.

Reject sharding or multiple indices as automatic performance fixes.

### Causal trace

1. A retrieval service shows slow results on a growing corpus.
2. Tracing shows index query time dominates the request.
3. The team tests an index layout suited to domain partitions.
4. It compares latency and relevant-result coverage under representative queries.

### Failure test

Latency improves but the relevant passage is no longer returned.

Measure retrieval latency with recall or coverage on the same test set.

Tune layout or query behavior until both constraints are acceptable.

### Experiment

Change a domain index partition strategy in a test copy.

Observe query latency, result coverage, and operational complexity.

Index tuning depends on store capabilities and actual query distribution.

What evidence should precede a vector index architecture change?

Stage-level latency and retrieval-quality data under representative query and corpus conditions.

Sources [16, 20, 23, 12].

## 1.4.4 Resource integration

Integration connects source systems to the content and metadata used by a GenAI application.

Use connectors, APIs, events, or scheduled jobs to ingest data with identity, change, and error handling.

### Decision

Use this skill when useful knowledge sits across document systems, wikis, or business stores.

Choose the integration path that preserves source authority and update signals.

Correct synchronization keeps downstream retrieval aligned with source state.

Reject bulk copying without ownership, deletion, and access rules.

**Causal trace**

1. An internal wiki publishes a revised procedure.
2. An integration detects the change and reads content with approved access.
3. The pipeline maps content and metadata into the knowledge source.
4. A retrieval check confirms the revised procedure is searchable.

**Failure test**

A source update never appears or the integration exposes excess content.

Trace source permissions, change detection, mapping, and sync status.

Repair the integration boundary and re-run a targeted sync.

**Experiment**

Change the sync trigger from a scheduled refresh to a source event in an illustrative test.

Observe freshness, failed records, and access scope.

Event delivery and source APIs can have limits that require backfill handling.

What should an integration preserve when it moves enterprise content?

Source identity, permissions, metadata, update and deletion state, and an auditable error path.

Sources [10, 20, 41, 12].

## 1.4.5 Vector store maintenance

A vector index is derived data and needs refresh, correction, and deletion controls.

Use incremental sync or scheduled refresh based on source change behavior and freshness needs. Track failures and stale records.

**Decision**

Use this skill when source material changes after initial indexing.

Select a maintenance path that detects updates and removals reliably.

The retrieval system must reflect authoritative source changes.

Reject a one-time ingestion plan for a changing knowledge source.

**Causal trace**

1. A source owner corrects a published procedure.
2. The sync mechanism detects the modified source object.
3. The ingestion path replaces its derived index records.
4. A query verifies the new procedure and no obsolete clause.

**Failure test**

Users continue to receive content from an older source version.



Compare source revision, sync result, indexed metadata, and retrieval output.

Retry or repair the sync, then verify replacement and deletion behavior.

**Experiment**

Change the refresh trigger or schedule for a test data source.

Observe update delay, failed records, and stale retrieval.

The best refresh interval depends on change patterns and the use case.

Why is index freshness an application quality concern?

Answers can remain stale when derived retrieval data does not reflect current source state.

Sources [10, 20, 16, 12].

## 1.5.1 Document segmentation

Segmentation determines which surrounding facts can travel with a retrieved passage.

Use fixed-size, semantic, hierarchical, or custom chunking based on document structure and query needs. Preserve headers and source locations.

**Decision**

Use this skill when long documents need focused retrieval.

Select boundaries by representative questions that require rules, exceptions, tables, or cross-section context.

Chunk boundaries affect both matching and final evidence completeness.

Reject a universal chunk size detached from content and task.

**Causal trace**

1. A policy sentence and its exception sit near a boundary.
2. The chunker splits the document into indexed passages.
3. A query retrieves the rule passage without the exception.
4. The answer omits the condition because it never entered context.

**Failure test**

Answers omit qualifications despite a relevant source document.

Inspect parsed content, chunk boundaries, and returned passages.

Adjust boundary strategy or parent context and test again.

**Experiment**

Change the chunk size or strategy for the targeted source set.

Observe exception coverage, irrelevant text, and result count.

Different formats and queries may need different segmentation.

Why can a highly precise small chunk still produce a wrong answer?

It may omit a nearby condition or exception needed to interpret the matched statement.

Sources [22, 16, 20, 12].

## 1.5.2 Embedding selection

Embedding choice determines how text and queries map into a similarity space.

Compare supported embedding models for domain fit, dimensions, language, throughput, and compatibility with the index. Re-embed when representation changes.Keep embedding identifiers attached to index versions for later comparison.

### Decision

Use this skill when selecting or changing semantic search representations.

Evaluate candidate embeddings on labeled retrieval queries and source passages.

Measured relevance reveals domain behavior more directly than vector size alone.

Reject mixing incompatible query and document representations.

### Causal trace

1. Ingestion encodes each source chunk with one embedding model.
2. The index stores those vectors with text and metadata.
3. The query path embeds a question compatibly.
4. Similarity search returns passages that evaluation can assess.

### Failure test

Relevant terms fail to retrieve after an embedding migration.

Check model compatibility, index version, chunk content, and query cases.

Re-embed the corpus or restore the compatible query model.

### Experiment

Change the embedding model and re-index a controlled corpus.

Observe retrieval relevance, query latency, and migration completeness.

Embedding scores are not truth probabilities and model changes require migration planning.

What must remain compatible for meaningful vector similarity?

Document and query vectors must use compatible embedding representations and index configuration.

Sources [16, 20, 23, 12].

## 1.5.3 Vector search deployment

Vector search deployment connects a compatible index to query and filtering operations.



Configure vector fields, dimensions, distance behavior, metadata fields, access, and index lifecycle according to the selected store.

### Decision

Use this skill when enabling semantic search in a custom or managed retrieval path.

Choose a store based on supported query features and operating requirements.

The store must support the application's vector, filter, and scale needs.

Reject a storage choice before confirming integration and query constraints.

### Causal trace

1. The ingestion service creates a vector for each record.
2. The index schema accepts the vector and trusted metadata.
3. A query embeds user text and applies required filters.
4. The store returns records that the application validates and uses.

### Failure test

Index creation succeeds but queries return errors or poor matches.

Verify dimensions, schema, distance configuration, and filter support.

Align index and query configuration, then test known records.

### Experiment

Change the index schema or search configuration in a test environment.

Observe query success, relevance, and filter behavior.

Deployment correctness does not establish answer grounding.

What must align between vector creation and vector search?

Embedding representation, vector dimensions, index schema, and query configuration must agree.

Sources [20, 16, 12].

## 1.5.4 Advanced search relevance

Search quality can improve by combining retrieval signals and reranking candidates.

Use lexical, semantic, hybrid, and reranking stages to address distinct retrieval defects. Keep filters and candidate trace visible.

### Decision

Use this skill when simple semantic retrieval misses exact terms or ranks evidence poorly.

Use hybrid retrieval for exact terms plus meaning, and rerank when candidate order is the measured issue.

Each stage addresses a different failure point.

Reject reranking as a fix for a passage absent from first-stage candidates.

**Causal trace**

1. A query contains an exact code and a natural-language description.
2. Semantic search returns related guidance but misses the exact record.
3. Hybrid search adds lexical matches and a reranker adjusts order.
4. Evaluation checks whether the final context contains the exact source.

**Failure test**

A reranker cannot surface the expected item.

Inspect first-stage candidate recall before reranker output.

Improve lexical fields, filters, or candidate generation first.

**Experiment**

Change search mode from semantic to hybrid on a supported store.

Observe exact-term recall, irrelevant candidates, and final answer evidence.

Hybrid support depends on the configured store and searchable text fields.

When should a team add a reranker?

When relevant candidates exist but their order remains a measured retrieval defect.

Sources [16, 24, 21, 12].

## 1.5.5 Query handling

Query transformation can improve retrieval when user wording hides the needed search intent.

Expand, decompose, normalize, or route queries while preserving the original question and evidence relationship.

**Decision**

Use this skill when one query contains multiple issues or unfamiliar phrasing.

Transform only the query classes shown to fail, then compare with the original retrieval.

Targeted transformation can improve coverage while limiting added noise.

Reject untested rewriting that changes a user's intent.

**Causal trace**

1. A question asks for two independent policy conditions.
2. A query step decomposes it into focused subqueries.
3. Each subquery retrieves its own evidence.
4. The application combines passages and checks both parts before generation.

**Failure test**

The rewrite retrieves related content but misses the user's actual constraint.

Compare original and transformed query, then inspect each result set.



Preserve key entities and test query-level relevance before combining results.

**Experiment**

Change the query decomposition rule for multi-part questions.

Observe evidence coverage for each subquestion and total retrieval noise.

Transformation can add latency and can distort intent.

How can query decomposition help a multi-part question?

It can retrieve evidence for each part separately, provided the application preserves the original constraints.

Sources [16, 20, 13, 12].

## 1.5.6 Standard retrieval interfaces

A stable retrieval interface lets applications call search without coupling every client to one store.

Define query, filter, result, source-reference, and error contracts. Implement the adapter with a controlled API, tool, or MCP client.

**Decision**

Use this skill when multiple applications or models need the same retrieval capability.

Expose a narrow API that returns evidence and provenance in a stable shape.

A stable contract supports changes behind the boundary and consistent access checks.

Reject a generic tool that exposes unrestricted database operations.

**Causal trace**

1. A model-connected application receives a user question.
2. The retrieval client sends a validated query and trusted identity filters.
3. The retrieval service returns passages with source identifiers and scores.
4. The host selects authorized context and records the result status.

**Failure test**

A client receives content without provenance or applies inconsistent filters.

Inspect interface fields, caller identity, errors, and source links.

Tighten the contract and enforce shared authorization in the service.

**Experiment**

Change the interface response to require a source identifier for each passage.

Observe client compatibility and evidence traceability.

A common interface does not eliminate store-specific limits or model behavior.

What must a retrieval contract return besides passage text?

It should return source identity, relevant metadata, status, and fields needed for safe filtering and review.

Sources [24, 26, 27, 12].

## 1.6.1 Instruction frameworks

Instructions define task behavior, boundaries, and expected output for the model.

Separate stable system rules from user input and retrieved evidence. Use templates, examples, output formats, and supported guardrails.

### Decision

Use this skill when model behavior or response structure needs controlled guidance.

Write explicit instructions and validate outputs against the application contract.

Clear boundaries reduce ambiguity and support repeatable evaluation.

Reject the assumption that prompt text enforces permissions or guarantees truth.

### Causal trace

1. The application loads a versioned instruction template.
2. It inserts the task request and clearly marked evidence.
3. The model generates a response under those instructions.
4. A validator checks structure and required evidence before delivery.

### Failure test

The model follows an instruction embedded in untrusted source text.

Inspect prompt assembly and variable boundaries.

Mark external text as data and enforce permissions outside the prompt.

### Experiment

Change the template to delimit retrieved evidence explicitly.

Observe instruction adherence and task quality on injection and normal cases.

Prompt controls guide behavior but do not replace authorization or validation.

What does a prompt instruction framework control, and what remains external?

It guides model response behavior. Identity, permissions, truth checks, and side-effect controls remain application responsibilities.

Sources [25, 3, 34, 12].



## 1.6.2 Interactive context

Interactive systems retain useful conversation state while keeping each turn grounded and manageable.

Use session identifiers, bounded history, summaries, intent recognition, and clarification states. Persist only what the application needs.

### Decision

Use this skill when later turns depend on prior user context or need clarification.

Store task-relevant state with explicit ownership and expiry behavior.

State supports continuity without sending an unlimited transcript every time.

Reject reliance on model memory for facts absent from the current request.

### Causal trace

1. A user asks “What about the second option?” after a comparison.
2. The application resolves the reference from recent session state.
3. It asks a clarification if the reference is ambiguous.
4. It sends the selected option and current evidence to inference.

### Failure test

The assistant uses the wrong referent from an old turn.

Inspect stored state, recency, session identity, and reference resolution.

Ask a clarifying question or use a bounded state summary.

### Experiment

Change the history policy from full transcript to relevant recent turns plus a summary.

Observe reference resolution, token use, and correction rate.

Summaries can omit facts, so preserve source state for consequential decisions.

When should an interactive system ask for clarification?

When session state does not identify the user’s intended reference with sufficient confidence.

Sources [3, 25, 26, 12].

## 1.6.3 Prompt governance

Prompt governance controls ownership, versions, approval, use, and audit of prompts.

Store reusable templates with parameters and versions. Record who changes them and connect versions to evaluation and release evidence.

### Decision

Use this skill when prompt changes can alter production behavior or policy.

Manage prompts as versioned release artifacts with review and audit records.

A version link supports reproduction and rollback.

Reject untracked edits to a production prompt.

**Causal trace**

1. An owner proposes a prompt change.
2. The change receives a new version and review.
3. Evaluation runs against that version and known cases.
4. The release record maps the deployed prompt to approval evidence.

**Failure test**

A production answer cannot be linked to the prompt version that produced it.

Compare deployed prompt reference with logs and release records.

Version the template and capture the active identifier safely.

**Experiment**

Change the approval rule to require evaluation for each new prompt version.

Observe whether promotion blocks when evidence is missing.

Versioning cannot guarantee quality unless test cases match actual use.

Why must an incident record identify the prompt version?

Prompt changes can alter behavior, so the version is needed to reproduce and review the response.

Sources [25, 30, 39, 12].

## 1.6.4 Prompt quality assurance

Prompt quality assurance tests whether a prompt meets its expected task behavior.

Use fixed evaluation cases, output validators, edge cases, regression checks, and review of failures.

**Decision**

Use this skill when prompts change or output reliability matters.

Run regression checks against a versioned test set before release.

The comparison reveals behavior changes caused by prompt edits.

Reject a prompt review based only on one manually selected output.

**Causal trace**

1. A prompt currently extracts policy owner and effective date.
2. A change adds a required citation field.
3. The test suite runs ordinary, missing-field, and exception cases.
4. Reviewers compare validity and evidence against the baseline.



**Failure test**

A new prompt passes common cases but fails on missing information.

Inspect per-case output and validator results, not only aggregate score.

Add the failure class to regression coverage and revise the prompt or logic.

**Experiment**

Change one prompt instruction while keeping model and test set fixed.

Observe schema validity, evidence support, and edge-case regressions.

Evaluation quality depends on case coverage and explicit expected outcomes.

What makes a prompt regression test useful?

It uses repeatable cases with expected behavior and exposes failures caused by a prompt change.

Sources [28, 21, 25, 12].

## 1.6.5 Iterative prompt improvement

Prompt improvement is a controlled cycle of diagnosing a defect, changing a prompt, and measuring the result.

Use structured input, examples, output constraints, decomposition, or feedback where they address a specific failure.

**Decision**

Use this skill when a capable model repeatedly misses a format or task requirement.

Change the smallest prompt element tied to the observed error.

A focused change makes the cause of improvement or regression clearer.

Reject adding elaborate reasoning instructions without diagnosing the failure.

**Causal trace**

1. The model understands a task but returns inconsistent field names.
2. The team defines a schema and a representative example.
3. It changes the prompt and validates output after inference.
4. Evaluation compares format and answer correctness.

**Failure test**

Formatting improves while factual quality declines.

Measure both format and task correctness across the same cases.

Keep the change only if it meets the complete acceptance criteria.

**Experiment**

Add a structured output specification to the existing prompt.

Observe parse success, missing fields, and semantic correctness.

Prompt improvements can trade off against other behavior and need regression tests.

How should a team refine a prompt after a repeated defect?

Identify the defect, make one focused change, and compare all relevant quality measures.

Sources [25, 3, 43, 12].

## 1.6.6 Complex prompt systems

Complex prompt systems use multiple stages, reusable components, and conditional paths.

Separate preprocessing, prompt stages, branching rules, postprocessing, and error handling. Pass explicit state between stages.

### Decision

Use this skill when a task needs sequential prompts or conditional processing.

Use a workflow with visible state and branches rather than one opaque prompt.

Stages make decisions inspectable and let each step handle a defined purpose.

Reject a chain that repeats model calls without a measurable need.

### Causal trace

1. A request first needs intent classification.

2. The workflow routes a policy question to retrieval and an extraction task to parsing.

3. Each branch applies its prompt and validates its result.

4. A final stage combines accepted outputs into the response.

### Failure test

A later stage uses stale or incompatible output from an earlier stage.

Inspect stage inputs, outputs, branch conditions, and prompt versions.

Validate state contracts between stages and test each branch.

### Experiment

Change a branch condition for one task class in a test workflow.

Observe branch selection, intermediate validity, and total request behavior.

More stages add latency, cost, and failure points.

When is a multi-stage prompt system justified?

When distinct steps or branches improve a measured outcome and each stage has a clear contract.

Sources [13, 25, 28, 12].



## 2.1.1 Agent memory and state

Agent memory is application-managed context and state, not a guaranteed model capability.

Store session state, task progress, and durable facts with explicit scope, retention, and retrieval rules.

### Decision

Use this skill when an agent must continue across turns or coordinate multiple agents.

Separate short-term conversation state from durable task memory.

Clear state types reduce stale or cross-session context.

Reject sending every prior message or sharing memory across users without access checks.

### Causal trace

1. A user starts a multi-step task and receives a session identity.

2. The agent stores task status and relevant references under that session.

3. A later request retrieves only that session's permitted state.

4. The agent resumes from verified state and updates its progress.

### Failure test

The agent repeats work or recalls another session's data.

Check session keys, ownership, expiry, and persisted state contents.

Scope memory by identity and task, then validate state before reuse.

### Experiment

Change the memory lookup from global to session-scoped state.

Observe continuation accuracy and cross-session isolation.

Memory can be stale or incomplete and must remain subject to authorization.

Why should agent memory use explicit session scope?

It supports continuity while reducing stale context and cross-user data exposure.

Sources [26, 27, 44, 7].

## 2.1.2 Structured problem solving

Structured reasoning decomposes a complex objective into steps with observable inputs and results.

Use a planned workflow, tool observations, and decision points. Treat internal reasoning text as separate from verified evidence.

### Decision

Use this skill when a request has dependent subtasks or needs conditional execution.

Represent steps and evidence in an explicit workflow such as a state machine.

Explicit states support control, retry, audit, and recovery.

Reject reliance on hidden reasoning as proof that a task step occurred.

**Causal trace**

1. A request requires identifying an account and checking eligibility.
2. The workflow resolves the account from an approved tool.
3. A decision state checks eligibility from returned facts.
4. The workflow produces an answer or asks for missing information.

**Failure test**

The model claims a step was completed without a tool result.

Inspect state transitions and tool observations.

Require verified outputs before advancing dependent steps.

**Experiment**

Change the workflow to require an eligibility result before response generation.

Observe whether the agent stops or asks when evidence is missing.

Reasoning structure does not guarantee correct evidence or safe tool use.

What should a later step use as evidence that an earlier step succeeded?

A validated tool or workflow result, not a model statement that the step occurred.

Sources [45, 13, 26, 7].

## 2.1.3 Safeguarded agent workflows

Agent safeguards constrain actions, runtime, resources, and recovery paths.

Set stopping conditions, timeouts, IAM boundaries, validation, and circuit breakers around model-selected steps.

**Decision**

Use this skill whenever an agent can call tools or run for multiple steps.

Enforce limits in workflow and service code, outside model instructions.

Runtime controls remain effective when the model behaves unexpectedly.

Reject prompt-only controls for permissions or execution limits.

**Causal trace**

1. The agent receives a request and can call a bounded tool.
2. A workflow checks the allowed state and remaining execution budget.
3. A timeout or denied IAM action stops an unsafe path.
4. The user receives a controlled response and an audit event.



**Failure test**

An agent loops, exceeds time, or invokes a resource outside its role.

Inspect step count, timeout, IAM policy, and circuit state.

Add an enforced bound and narrow the execution role.

**Experiment**

Change the workflow stop condition or timeout in a test environment.

Observe termination, denied actions, and safe user response.

Controls may interrupt valid tasks and need clear recovery behavior.

Where should an agent's resource boundary be enforced?

In IAM and runtime controls that authorize each operation, not only in the prompt.

Sources [5, 26, 4, 7].

## 2.1.4 Model coordination

Model coordination assigns work to models or capabilities and combines their results.

Route specialized tasks to suitable models and define an aggregation rule that handles disagreement or missing output.

**Decision**

Use this skill when one model or capability does not meet all task needs.

Coordinate only where specialization improves an evaluated outcome.

An explicit aggregator can preserve evidence and expose conflicts.

Reject an ensemble that votes without a rule for confidence or disagreement.

**Causal trace**

1. A workflow needs extraction and a policy interpretation.
2. A specialized model extracts fields from source content.
3. A second route evaluates policy against those fields and evidence.
4. An aggregator checks required fields and reports unresolved conflicts.

**Failure test**

The combined answer hides disagreement or propagates a bad intermediate result.

Inspect each model result and aggregation rule.

Validate intermediate outputs and define conflict handling.

**Experiment**

Change one task route to a specialized model while retaining the same aggregate checks.

Observe per-stage quality, disagreement, latency, and total cost.

Multiple models add complexity and do not guarantee better results.

When should a team coordinate multiple models?

When distinct capabilities improve measured task performance enough to justify added cost and failure paths.

Sources [17, 13, 28, 7].

## 2.1.5 Human collaboration

Human review adds expertise or approval where automated output needs oversight.

Route uncertain, high-impact, or consequential outcomes to a reviewer with evidence and a clear decision action.

### Decision

Use this skill when a person must validate content or authorize an action.

Insert a defined review state before the consequential step.

A human can assess context that automated checks may not capture.

Reject vague “human in the loop” designs without authority or workflow state.

### Causal trace

1. An agent prepares a proposed account change.
2. The workflow displays the proposed change and source evidence to a reviewer.
3. The reviewer approves or rejects with a recorded decision.
4. Only an approval state permits the authorized tool call.

### Failure test

The system performs an action before review or cannot identify who approved it.

Check workflow order, reviewer identity, decision record, and action gate.

Move the action behind an explicit approval state.

### Experiment

Change the workflow to require approval before a write operation.

Observe blocked actions without approval and traceable completion with approval.

Review can add delay and depends on clear evidence and reviewer responsibility.

What must a human review step control?

It must provide decision authority, relevant evidence, recorded outcome, and a gate before the consequential action.

Sources [13, 26, 5, 7].

## 2.1.6 Reliable tool integration

Tool integration turns model requests into validated application operations.



Define narrow tool schemas, validate arguments, authorize each call, handle errors, and return useful observations.

### Decision

Use this skill when a model needs external data or application actions.

Expose task-specific tools rather than broad general-purpose access.

Narrow tools reduce invalid arguments and limit side effects.

Reject direct execution of unvalidated model-generated arguments.

### Causal trace

1. The model requests a customer lookup with an identifier.
2. The host validates the identifier and caller permission.
3. A tool executes the read and returns a structured result.
4. The host passes a bounded observation back to the model.

### Failure test

The tool acts on the wrong record or repeats a side effect.

Inspect schema validation, identity checks, idempotency, and tool result.

Reject invalid calls before execution and verify write outcomes.

### Experiment

Change a tool schema to require a validated resource identifier.

Observe invalid-call rejection and successful result structure.

Model tool selection can be wrong even when the tool itself works.

What should the host do before executing a model tool request?

Validate its schema and authorize the caller and requested resource before the operation.

Sources [26, 27, 5, 7].

## 2.1.7 MCP extension frameworks

MCP provides a standardized pattern for exposing tools and resources to connected clients.

Choose a lightweight server for narrow stateless operations or a managed container path for complex tools. Secure server identity, transport, and outputs.

### Decision

Use this skill when tools need reusable discovery and access across model clients.

Build an MCP server with explicit tool contracts and client-side policy.

The boundary supports reuse while leaving authorization and validation explicit.

Reject assuming protocol standardization makes every server safe.

**Causal trace**

1. A client discovers a tool from an MCP server.
2. It receives the tool schema and offers it to the model.
3. The host validates the requested arguments and server access.
4. The server returns data that the host treats as untrusted input.

**Failure test**

A server exposes excessive tools or returns content that changes host behavior.

Review discovery scope, server identity, tool permissions, and output handling.

Limit exposed tools and validate returned data before use.

**Experiment**

Change the server deployment or tool allowlist for a representative client.

Observe discovery, authentication, call reliability, and output safety.

The right hosting choice depends on the tool's runtime and operational needs.

What does MCP standardize, and what remains the host's responsibility?

It standardizes interaction patterns. The host still owns tool selection, identity, authorization, validation, and safe output handling.

Sources [27, 26, 5, 7].

## 2.2.1 Deployment by workload

Deployment choice must match traffic, control, and performance needs.

Compare on-demand invocation, provisioned throughput, and hosted endpoints against usage pattern, latency, model control, and operations.Include cold starts and peak periods in the workload profile.

**Decision**

Use this skill when moving an FM capability into a production application.

Select a deployment option using measured workload requirements.

The option should handle expected demand without unnecessary operations or cost.

Reject choosing solely from a prototype's low-volume behavior.

**Causal trace**

1. An application has variable user demand.
2. The team measures request pattern and latency requirements.
3. It selects an invocation or endpoint approach suited to the workload.
4. Monitoring verifies behavior under representative traffic.



**Failure test**

A deployment works in a demo but throttles or costs too much under actual traffic.

Compare real request profile, capacity, quotas, and endpoint configuration.

Tune deployment capacity or choose a better-fitting option.

**Experiment**

Change deployment mode while holding request and workload profile constant.

Observe latency, availability, scaling behavior, and cost signals.

Service options and limits can vary by model and Region.

What workload evidence should guide deployment selection?

Traffic shape, latency needs, capacity behavior, model control, and operating constraints.

Sources [38, 29, 18, 7].

## 2.2.2 LLM deployment constraints

LLM serving has resource behavior tied to model loading, memory, accelerator use, and token processing.

Plan artifact loading, memory capacity, concurrency, request length, and generation limits for the chosen serving pattern.

**Decision**

Use this skill when hosting or configuring large models on managed compute.

Profile the model and workload under expected input and output shapes.

Token work and model size affect resource use differently from many conventional services.

Reject scaling plans based only on request count.

**Causal trace**

1. A hosted model receives long prompts and generates responses.
2. The runtime loads model weights and allocates device memory.
3. Concurrency and token lengths raise active resource demand.
4. The endpoint queues or fails when capacity is inadequate.

**Failure test**

Endpoint capacity appears sufficient by request rate but fails on large contexts.

Measure token lengths, memory pressure, loading time, and accelerator utilization.

Adjust model placement, capacity, or request limits using profile evidence.

**Experiment**

Change the maximum input or output token budget in a test workload.

Observe memory use, throughput, latency, and quality effects.

Reducing token limits can harm task completeness.

Why is request count alone a weak capacity measure for LLM serving?

Requests can differ in token length and generation work, which changes resource demand.

Sources [29, 17, 38, 7].

## 2.2.3 Deployment optimization

Deployment optimization balances quality, latency, cost, and resource use.

Use smaller task-fit models, model cascades, caching, or API routes where evaluation supports the change.

**Decision**

Use this skill when a valid system needs lower cost or faster response.

Optimize the bottleneck or call type shown by measurement.

Targeted routing can reserve stronger models for requests that need them.

Reject reducing model quality globally without measuring task impact.

**Causal trace**

1. A simple request reaches a high-capability model.
2. A classifier or lightweight route identifies a routine task.
3. Routine work uses a smaller adequate model while complex cases escalate.
4. Evaluation checks quality across both routes.

**Failure test**

The cascade sends difficult questions to the weak model or adds more calls than it saves.

Inspect route decisions, escalation, per-class quality, and total calls.

Revise routing criteria or remove the cascade if gains do not hold.

**Experiment**

Change the cascade threshold or task routing rule in a test set.

Observe task quality, calls per request, latency, and cost.

A cascade adds routing errors and needs coverage for misclassified tasks.

When is a model cascade useful?

When a measured workload has simpler requests that a smaller model handles adequately and complex cases can escalate safely.

Sources [17, 37, 28, 7].

## 2.3.1 Enterprise connectivity

Enterprise connectivity links FM services with existing data, identity, and business systems.



Use APIs, events, or synchronization to connect systems while preserving ownership, permissions, and failure recovery.

**Decision**

Use this skill when AI features need legacy records or enterprise workflows.

Use a loose integration boundary with explicit contracts and change handling.

It limits coupling and provides a controlled path for data exchange.

Reject direct database access that bypasses the system's data rules.

**Causal trace**

1. A legacy system owns a service record.
2. An integration API retrieves permitted fields for a caller.
3. The AI workflow uses those fields to form a response.
4. A trace links the answer to the source record and integration result.

**Failure test**

The application receives stale or overbroad enterprise data.

Check source ownership, API scope, synchronization age, and access controls.

Narrow the integration and define update and failure behavior.

**Experiment**

Change the data integration from a broad export to a scoped API request.

Observe field minimization, freshness, and failure isolation.

Enterprise interfaces can impose latency and availability dependencies.

What should an AI integration preserve when connecting to a legacy system?

Data ownership, access rules, field scope, update behavior, and a recoverable error contract.

Sources [1, 10, 5, 7].

## 2.3.2 Application integration

GenAI integration adds model capability to an existing application through stable service boundaries.

Use APIs, webhooks, events, or microservices to invoke model workflows without coupling user interfaces to provider details.

**Decision**

Use this skill when adding GenAI behavior to an existing product or process.

Expose an application-level API with validation and consistent response states.

A stable boundary lets the application control identity, errors, and version changes.

Reject placing model credentials or provider logic in a client interface.

**Causal trace**

1. A user submits a request through an existing application.
2. The backend validates the caller and invokes a GenAI service.
3. The service returns a result or controlled error.
4. The application presents output and records a trace identifier.

**Failure test**

The UI hangs on a model timeout or exposes provider credentials.

Inspect client-server boundaries, auth, timeouts, and error mapping.

Keep credentials server-side and implement explicit timeout behavior.

**Experiment**

Change the integration from direct client invocation to a backend API.

Observe credential exposure, error handling, and application compatibility.

An API gateway or Lambda integration does not automatically solve application semantics.

Where should model credentials and provider-specific logic reside?

In a controlled backend integration, not in an exposed client application.

Sources [3, 33, 5, 7].

## 2.3.3 Secure access framework

Secure access controls which identities can invoke models and reach supporting data or tools.

Use federation, roles, resource policies, and least privilege. Enforce user-level data access within retrieval and application logic.

**Decision**

Use this skill when enterprise identities cross application and model service boundaries.

Map authenticated enterprise identity to scoped AWS and application permissions.

Separate roles and permissions limit impact if a component is misused.

Reject a shared powerful role for all users and actions.

**Causal trace**

1. An employee authenticates through enterprise federation.
2. The application maps identity to an approved role and data scope.
3. The request invokes only allowed models and retrieval filters.
4. Audit records capture the action and principal.

**Failure test**

A user can invoke a model or retrieve records outside their role.

Trace identity claims, role policy, resource policy, and query filters.



Remove broad permissions and bind data filters to trusted identity.

**Experiment**

Change a shared role into task-specific least-privilege roles.

Observe denied unauthorized actions and preserved allowed tasks.

IAM service permissions alone do not enforce every document-level rule.

How should an enterprise identity affect a RAG request?

It should determine allowed service actions and trusted record filters before content reaches the model.

Sources [5, 6, 24, 7].

## 2.3.4 Cross-environment compliance

Cross-environment design connects cloud, on-premises, and edge systems while respecting data boundaries.

Map where data is stored, processed, routed, and logged. Use suitable private connectivity or edge placement for the stated requirement.

**Decision**

Use this skill when jurisdiction, latency, or local data handling constrains FM access.

Keep sensitive data within the allowed boundary and move only approved content.

Data location and path determine whether the design meets compliance needs.

Reject assuming network connectivity alone satisfies jurisdictional policy.

**Causal trace**

1. A source record must remain in an approved environment.
2. A local component extracts only allowed fields.
3. A secure route sends approved content to an available inference service.
4. Logs and outputs follow the same location and retention rules.

**Failure test**

Data or logs cross a boundary that policy prohibits.

Trace all data paths, regions, endpoints, and logging destinations.

Remove or relocate the violating path and verify each destination.

**Experiment**

Change the inference path to process sensitive content at an approved location.

Observe data movement, latency, and retained copies.

Compliance requirements depend on the organization's rules and applicable service availability.

What must be mapped in a cross-environment AI design?

Data location, processing, network path, logs, outputs, identity, and retention.

Sources [8, 6, 9, 7].

## 2.3.5 GenAI CI/CD and gateway

Enterprise gateways control model access, while CI/CD controls how changes reach those gateways and applications.

Version prompts and routes, scan code and infrastructure, run evaluations, apply policy checks, and retain rollback state.

**Decision**

Use this skill when many teams consume FM services under shared security and compliance rules.

Combine a centralized consumption boundary with repeatable release gates.

It provides common controls and makes changes traceable.

Reject a gateway without governance or a pipeline without behavior checks.

**Causal trace**

1. A team proposes a prompt and route change.
2. CI validates configuration, security, and evaluation evidence.
3. An approved release updates the gateway policy or application.
4. Monitoring detects regression and rollback restores the prior version.

**Failure test**

An untested prompt or route reaches production through a configuration edit.

Inspect release records, approval gates, version links, and gateway logs.

Gate promotion on matching evidence and restrict policy change rights.

**Experiment**

Change the pipeline to require evaluation for each route or prompt version.

Observe blocked unapproved changes and traceable approved deployment.

Automated checks cover defined cases, not all possible model behavior.

What should CI/CD validate for a GenAI gateway change?

It should validate security, policy, request contracts, relevant evaluations, approval, and rollback readiness.

Sources [5, 28, 25, 30, 7].

## 2.4.1 Flexible API interaction

API integration must support synchronous and asynchronous work with validated request contracts.

Use SDK or service APIs for direct calls, queues for deferred work, and gateways for custom validation and client contracts.



**Decision**

Use this skill when different workloads need different request timing or compute environments.

Choose synchronous calls for immediate results and asynchronous patterns for longer tasks.

The interaction pattern should match user wait tolerance and task duration.

Reject forcing all tasks through a blocking request.

**Causal trace**

1. A client submits a task that can finish quickly.
2. The backend validates and invokes the model synchronously.
3. A longer task instead enters a queue with a tracking identifier.
4. The client receives the result or task status through its contract.

**Failure test**

Long-running requests time out or asynchronous work loses its correlation.

Check duration, queue state, request IDs, and result delivery.

Select the correct interaction pattern and persist task status.

**Experiment**

Change a long task from synchronous invocation to queue-backed processing.

Observe timeout rate, completion delivery, and trace continuity.

Asynchronous processing adds status, retry, and duplicate handling needs.

When should an API use asynchronous processing?

When task duration or workload behavior does not fit a reliable user-facing synchronous response.

Sources [2, 14, 33, 7].

## 2.4.2 Real-time streaming

Streaming delivers response pieces before the full model response completes.

Connect the model stream to server-sent events or WebSockets and handle partial, final, and error states.

**Decision**

Use this skill when early visible output improves an interactive experience.

Stream content through a backend that can preserve authorization and completion state.

The user can see progress without waiting for the entire response.

Reject treating the first token as a completed answer.

**Causal trace**

1. The application opens an authorized streaming request.

2. The model emits content events over time.

3. The backend forwards events and tracks completion.

4. The UI marks the response final only after the terminal event.

**Failure test**

The UI displays partial text as final or fails to handle a midstream error.

Inspect event order, terminal state, client disconnect, and error path.

Represent partial and complete states separately and show recovery behavior.

**Experiment**

Change the transport from buffered response to event streaming.

Observe time to first content, total completion, and disconnect handling.

Streaming does not guarantee lower total latency or support for every route.

What state must a client track during a streamed response?

It must distinguish open, partial, complete, and failed states.

Sources [31, 32, 33, 7].

## 2.4.3 Resilient API operation

Reliable API operation combines error classification, bounded recovery, and observability.

Use SDK retry behavior for transient conditions, rate limits, timeouts, and traces across service boundaries.

**Decision**

Use this skill when model calls must recover predictably from service variation.

Apply error-specific retry and rate controls with request tracing.

This avoids repeating permanent failures and reveals dependency behavior.

Reject retrying every error or hiding final failures from the caller.

**Causal trace**

1. A request receives a throttling response.

2. The SDK applies bounded backoff and records another attempt.

3. The service accepts a later attempt or returns a final error.

4. Tracing shows attempt count and final outcome.

**Failure test**

Retries increase load or a permanent error repeats.

Inspect error class, retry policy, rate limits, and trace spans.

Restrict retries to transient errors and return a clear failure state.

**Experiment**

Change the retry policy to exclude validation and access errors.



Observe attempts, success rate, latency, and load during test faults.

A timeout can leave operation status unknown, especially for side effects.

Which errors should not receive automatic retries?

Validation and access errors need correction. Retrying them cannot fix the underlying request or permission.

Sources [4, 33, 30, 7].

## 2.4.4 Intelligent model routing

Model routing selects a route based on configuration, request features, or measured conditions.

Keep route policy explicit, validate supported models, and preserve compatible response behavior.Log route reasons so operators can review unexpected selections.

**Decision**

Use this skill when tasks vary in complexity, modality, cost, or availability.

Route by a known task class or rule, then evaluate each path.

A decision rule is inspectable and can reserve specialized models for suitable work.

Reject hidden routing that cannot be explained or tested.

**Causal trace**

1. A request arrives with task type and modality.

2. A routing rule checks allowed routes and current policy.

3. The selected model receives a compatible request.

4. The result and chosen route are recorded for evaluation.

**Failure test**

Requests route to an unsupported model or a low-capability route.

Inspect classification, policy, model access, and per-route outcomes.

Add route validation and fallback behavior with explicit limits.

**Experiment**

Change the routing rule for one task category.

Observe route selection accuracy, task quality, and latency.

Dynamic routing can introduce classification errors and needs fallback tests.

What makes an intelligent route safe to operate?

Explicit policy, validated routes, traceability, per-route evaluation, and a tested fallback.

Sources [18, 19, 13, 7].

## 2.5.1 FM API interface design

A GenAI API must manage variable output, token limits, and failure states.

Define request validation, context and output budgets, timeout behavior, retries, and response status. Support streaming when needed.

**Decision**

Use this skill when building application-facing interfaces for model workloads.

Expose application-level operations with clear limits and errors.

Clients need stable behavior despite variable model response time and length.

Reject returning raw provider responses without a defined client contract.

**Causal trace**

1. A client submits a question and allowed context.
2. The API validates input and applies token budgets.
3. The service invokes inference with bounded recovery.
4. The response returns content, completion state, and trace reference.

**Failure test**

Oversized input, timeout, or partial output breaks client behavior.

Check validation, budget calculation, terminal state, and error mapping.

Enforce limits before inference and return explicit partial or failed status.

**Experiment**

Change the request token budget or timeout policy in a test environment.

Observe rejection behavior, latency, and response completeness.

Budgets and limits must reflect model support and task needs.

What should a custom FM API expose beyond answer text?

It should define request limits, completion or failure state, and useful trace information.

Sources [3, 33, 4, 7].

## 2.5.2 Accessible AI interfaces

Accessible interfaces lower integration effort through clear contracts and usable workflow components.

Use declarative UI, API specifications, or workflow builders while retaining backend authorization and validation.

**Decision**

Use this skill when user teams need a supported path to invoke AI capability.

Publish an API-first contract and accessible interface elements for the intended users.

A stable interface supports adoption without exposing service internals.

Reject a no-code surface that bypasses policy or hides error states.



**Causal trace**

1. A product team publishes an OpenAPI contract for an AI operation.
2. A UI invokes the contract through an authenticated backend.
3. The backend validates input and returns explicit states.
4. Users can understand success, uncertainty, or required correction.

**Failure test**

Users cannot tell whether output is incomplete, invalid, or awaiting review.

Test interface states, accessibility needs, auth flow, and API errors.

Add clear status and validation behavior at the user boundary.

**Experiment**

Change the interface to show a distinct review or incomplete state.

Observe task completion and user correction behavior.

A friendly interface cannot make unsafe backend behavior safe.

What makes an AI interface accessible and safe to integrate?

A clear contract, usable interaction, explicit status, and backend-enforced controls.

Sources [1, 13, 5, 7].

## 2.5.3 Business system enhancement

Business enhancements place model capability inside an existing process with source and action controls.

Use orchestration, webhooks, or data automation to process business records and route results to existing systems.Keep the original record available for audit and correction.

**Decision**

Use this skill when GenAI should improve a CRM, document workflow, or operations task.

Start with a bounded workflow step and preserve the system of record.

This adds AI assistance without replacing business ownership or audit state.

Reject automatic writes from unverified generated content.

**Causal trace**

1. A business record enters a document workflow.
2. A processing service extracts candidate fields.
3. Validation or human review accepts the fields.
4. The system of record receives only approved values.

**Failure test**

Generated data overwrites a business record without validation.

Trace extraction, validation, approval, and write permissions.

Gate updates on schema checks and authorized approval.

**Experiment**

Change the flow to add validation before the system write.

Observe rejected fields, correction rate, and verified updates.

Business value depends on workflow fit and input quality.

How should a GenAI workflow update a system of record?

Validate the generated fields and apply required approval before an authorized update.

Sources [41, 13, 5, 7].

## 2.5.4 Developer productivity tools

Developer AI tools can assist code tasks, but generated changes still need engineering review.

Use coding assistants for suggestions, explanation, tests, or refactoring within repository and security controls.

**Decision**

Use this skill when GenAI can reduce development effort without weakening quality gates.

Apply suggestions through the normal code review and test process.

The developer retains responsibility for correctness and data handling.

Reject unreviewed generated code or secrets pasted into prompts.

**Causal trace**

1. A developer requests help with an API integration.
2. The assistant proposes a code change.
3. The developer reviews it against the SDK contract and repository rules.
4. CI and code review check behavior before merge.

**Failure test**

Generated code compiles but uses an unsupported API or broad permissions.

Inspect service documentation, permissions, data handling, and tests.

Correct the implementation and preserve normal review gates.

**Experiment**

Change the workflow to require security and contract review for generated code.

Observe defect detection and developer time without skipping quality checks.

Assistant output may be outdated, incomplete, or insecure.

What role should an AI code assistant play in a production workflow?

It can propose or explain code. Engineers and quality gates must verify and approve changes.



Sources [2, 5, 28, 7].

## 2.5.5 Advanced GenAI applications

Advanced applications combine prompts, agents, and workflows to handle multi-step tasks.

Choose orchestration patterns from task structure and side effects. Use reusable prompt chains or agents with explicit tools and stopping rules.

**Decision**

Use this skill when a simple one-call interaction cannot meet the task requirement.

Compose explicit stages and contracts, using an agent only where dynamic tool choice helps.

Clear stages support validation and recovery.

Reject complexity that adds model calls without improving task outcome.

**Causal trace**

1. A document workflow needs extraction, policy comparison, and approval.
2. A workflow extracts structured facts and validates them.
3. A model compares facts with retrieved policy evidence.
4. A reviewer approves before a consequential system action.

**Failure test**

The agent makes an unsupported decision or loops through redundant calls.

Inspect stage contracts, evidence, tool calls, and stop behavior.

Use deterministic steps for known work and guard model-selected branches.

**Experiment**

Change the orchestration from a single prompt to explicit validated stages.

Observe correctness, completion, calls, and side effects.

More sophisticated architecture also increases failure and maintenance costs.

How should a team choose between workflow stages and agent decisions?

Use explicit workflow stages for known sequences and agent choice only where the path depends on context.

Sources [26, 13, 27, 7].

## 2.5.6 GenAI troubleshooting

Troubleshooting improves when logs and traces show each request boundary and failure class.

Use CloudWatch Logs Insights, X-Ray, request IDs, and model error details to inspect latency and failures.

### Decision

Use this skill when an FM application has unexplained errors, delay, or response regressions.

Trace the request from caller through retrieval, inference, tools, and response.

A boundary trace narrows root cause and avoids blind prompt changes.

Reject storing all sensitive prompts as the default diagnostic method.

### Causal trace

1. A user reports a delayed answer.
2. The operator finds the trace ID and checks stage spans.
3. The trace shows repeated retrieval retries before inference.
4. The owner corrects retrieval behavior and confirms recovery.

### Failure test

Teams change prompts even when the API or retrieval stage failed.

Check request ID, error class, stage timing, model route, and source results.

Fix the first failing boundary and verify the full request path.

### Experiment

Change instrumentation to record stage duration and error class.

Observe whether a test timeout can be localized without full prompt capture.

Logs can contain sensitive data and require access and retention controls.

What should an operator inspect before editing a prompt for a failed answer?

Trace request stages, errors, route, and retrieval evidence to locate the first failed boundary.

Sources [30, 4, 21, 7].

## 3.1.1 Input content safety

Inspect untrusted input before it reaches the model. Define harmful input classes, allowed use, and escalation path.

Run input validation, supported Guardrails filters, and application moderation in an ordered path. Block or route unsafe input before prompt assembly.

### Decision

When: A user submits a direct request for prohibited harmful instructions.

The request is blocked before model invocation, while benign requests continue.

Blocking before prompt assembly keeps a policy-violating request from reaching inference while labeled benign requests test overblocking.

Do not block only on a keyword. That approach misses paraphrases and can reject allowed requests that use the same word.



### Causal trace

1. Case: A user submits a direct request for prohibited harmful instructions.
2. Check: Compare input moderation verdicts with labeled requests and blocked or allowed outcomes.
3. Act: Block or route the request before prompt assembly. Recheck labeled harmful and benign cases.
4. Test: Measure harmful request blocks and benign request passes against labeled examples.

### Failure test

A labeled prohibited request passes input moderation and appears in the serialized model request.

Compare input moderation verdicts with labeled requests and blocked or allowed outcomes.

Block or route the request before prompt assembly. Recheck labeled harmful and benign cases.

### Experiment

Compare the input filter result with a labeled set of harmful and benign requests.

Measure harmful request blocks and benign request passes against labeled examples.

Input controls do not guarantee safe model output.

Which evidence confirms this failure's cause and supports the correction?

The request violates the input policy, so block it before inference. Compare the moderation verdict with the label and confirm the request never enters the model payload.

Sources [46, 34, 35].

## 3.1.2 Output safety

Check generated output before the application returns it. Define which harmful content classes and action risks require blocking or review.

Apply supported output filters after generation. Add deterministic business checks for high-impact fields or actions.

### Decision

When: A model produces a harmful or disallowed answer despite a safe user question.

The post-generation check blocks the answer and returns a safe fallback.

A post-generation check can inspect content that did not exist when input moderation ran. A safe fallback prevents release of a disallowed answer.

Input filtering alone blocks harmful generated output. It cannot inspect content created after the request passes.

**Causal trace**

1. Case: A model produces a harmful or disallowed answer despite a safe user question.

2. Check: Inspect output filter results and the final response returned to the caller.

3. Act: Run the output check before returning the answer. Block disallowed output and return the approved fallback.

4. Test: Measure harmful output misses and benign output blocks after the response filter.

**Failure test**

A disallowed answer reaches the caller although the user request passed input moderation.

Inspect output filter results and the final response returned to the caller.

Run the output check before returning the answer. Block disallowed output and return the approved fallback.

**Experiment**

Change one output filter threshold or policy rule, then compare misses and false blocks on a labeled set.

Measure harmful output misses and benign output blocks after the response filter.

Content filtering does not establish factual accuracy.

Which evidence confirms this failure's cause and supports the correction?

Input moderation cannot evaluate generated text. Inspect the output verdict and returned payload. If the answer violates policy, block it before the caller receives it.

Sources [46, 34, 35].

## 3.1.3 Accuracy verification

Treat a generated claim as unverified until evidence supports it. Identify authoritative evidence and the cost of an incorrect answer.

Retrieve permitted source passages, bind claims to evidence, and check structured outputs against schema and domain rules. Use a confidence score only when calibration supports its meaning.

**Decision**

When: An answer states a date that differs from its retrieved policy passage.

The verifier flags the mismatch and the application corrects or withholds the date.

Claim to passage comparison detects unsupported or contradicted facts. A schema check only validates structure.

Do not accept a citation merely because it exists. A real source can fail to support or can contradict the claim.



**Causal trace**

1. Case: An answer states a date that differs from its retrieved policy passage.

2. Check: Compare each factual claim with its cited source and verifier result.

3. Act: Compare the date with the cited passage. Correct it or abstain when evidence does not support it.

4. Test: Measure supported claims, unsupported claims, and abstentions against cited evidence.

**Failure test**

The answer gives a date that conflicts with its cited policy passage, but the application returns it.

Compare each factual claim with its cited source and verifier result.

Compare the date with the cited passage. Correct it or abstain when evidence does not support it.

**Experiment**

Remove the supporting passage from a test case and check whether the system abstains.

Measure supported claims, unsupported claims, and abstentions against cited evidence.

Similarity to context is not proof that a claim is true.

Which evidence confirms this failure's cause and supports the correction?

The passage contradicts the answer date, so treat the claim as unsupported. Verify the source text, then correct or withhold the date.

Sources [46, 36, 20, 43].

## 3.1.4 Defense in depth

Assign separate controls to input, model behavior, output, tool permissions, and monitoring. Map each risk to a control owner.

Use request validation and moderation before inference, supported Guardrails during or after inference, and business validation before side effects. Apply IAM to tools and retain actionable event records.

**Decision**

When: An attacker bypasses an input keyword rule with a paraphrase.

A semantic safety check or output validation catches the risk before release.

Independent controls reduce the chance that one bypass releases unsafe content. Separate logs also show which boundary failed.

One filter enforces tool permissions because content checks cannot authorize tools or validate every side effect.

**Causal trace**

1. Case: An attacker bypasses an input keyword rule with a paraphrase.

2. Check: Trace which control layer allowed the unsafe path and whether later checks fired.

3. Act: Add an independent semantic or output check. Keep tool authorization and business validation outside the model.

4. Test: Measure unsafe paths blocked at each independent layer and benign task completion.

**Failure test**

A paraphrase bypasses the input rule and no later control blocks the unsafe response.

Trace which control layer allowed the unsafe path and whether later checks fired.

Add an independent semantic or output check. Keep tool authorization and business validation outside the model.

**Experiment**

Disable one layer in a controlled test and verify that another independent layer still blocks the specified unsafe path.

Measure unsafe paths blocked at each independent layer and benign task completion.

Multiple controls help only when their failure modes differ.

Which evidence confirms this failure's cause and supports the correction?

The input rule missed the paraphrase. Trace later checks and block the unsafe response at an independent boundary before release.

Sources [46, 34, 35, 5].

## 3.1.5 Adversarial input defense

Treat prompt injection as an attempt to alter instruction priority, data access, or tool behavior. Include direct and indirect attack sources.

Keep trusted instructions separate from user and retrieved data. Restrict tools in server code, validate destinations and arguments, and test jailbreak variants.

**Decision**

When: A retrieved document tells an agent to reveal another tenant's records.

The tool service rejects access outside the authenticated tenant scope.

Server-side tenant authorization limits the action even when hostile retrieved text changes model behavior.

Do not let retrieved instructions set tenant scope or tool permissions. Retrieved text is untrusted input, not authorization.

**Causal trace**

1. Case: A retrieved document tells an agent to reveal another tenant's records.



2. Check: Inspect attack classifier results, retrieved instructions, and server-side tool arguments.

3. Act: Enforce tenant scope in the tool service. Reject arguments that exceed the authenticated caller scope.

4. Test: Measure attack detection and unauthorized tool attempts across direct and indirect inputs.

**Failure test**

The agent issues a tool request for records outside the authenticated tenant scope.

Inspect attack classifier results, retrieved instructions, and server-side tool arguments.

Enforce tenant scope in the tool service. Reject arguments that exceed the authenticated caller scope.

**Experiment**

Add one indirect injection example to the red-team set and observe tool arguments and access decisions.

Measure attack detection and unauthorized tool attempts across direct and indirect inputs.

Prompt delimiters alone cannot enforce access control.

Which evidence confirms this failure's cause and supports the correction?

The retrieved instruction has no authority to change access. Compare the tool arguments with the authenticated tenant and reject any cross-tenant request.

Sources [46, 40, 34, 5].

## 3.2.1 Protected inference environment

Separate network reachability, identity authorization, data access, and encryption. Identify each required service API and workload role.

Use supported VPC endpoints and restrictive endpoint policies. Grant scoped IAM actions to the runtime role.

**Decision**

When: A private workload reaches the inference endpoint but gets AccessDenied.

Inspect role policy, resource scope, endpoint policy, and applicable key policy.

The denied API action and policy evaluation identify an authorization failure. A successful route only proves network reachability.

Do not change the VPC route first when the endpoint is reachable and the response is AccessDenied. Routing does not grant IAM permission.

**Causal trace**

1. Case: A private workload reaches the inference endpoint but gets AccessDenied.

2. Check: Inspect workload role policy, endpoint policy, resource scope, and denied API action.

3. Act: Inspect the denied action and evaluate the role, resource, endpoint, and key policies. Grant only the missing scoped permission.

4. Test: Measure authorized requests that pass and unauthorized requests that fail at each boundary.

**Failure test**

The workload reaches the private endpoint but an allowed caller receives AccessDenied.

Inspect workload role policy, endpoint policy, resource scope, and denied API action.

Inspect the denied action and evaluate the role, resource, endpoint, and key policies. Grant only the missing scoped permission.

**Experiment**

Remove one non-required action from a test role and confirm the intended inference path still works.

Measure authorized requests that pass and unauthorized requests that fail at each boundary.

Private networking does not grant service permission.

Which evidence confirms this failure's cause and supports the correction?

Network reachability is already present. Use the denied action and policy scope to find the authorization boundary, then change only that permission.

Sources [6, 46, 5, 8].

## 3.2.2 Privacy-preserving inference

Classify personal data before model use. Identify whether the task needs raw identifiers, derived features, or no personal data.

Detect supported PII patterns, mask or minimize fields, and restrict access to prompts and logs. Review the selected service data terms, retention settings, and key controls.

**Decision**

When: A support prompt contains an email address that the model does not need.

Mask the email before inference and preserve only the fields required to solve the task.

Masking an unneeded email reduces exposure before inference while a before-and-after check reveals damage to task fields.



Encryption protects stored data, but it does not remove raw PII from model input or configured logs.

**Causal trace**

1. Case: A support prompt contains an email address that the model does not need.

2. Check: Inspect the prompt before and after redaction, log access, and retention settings.

3. Act: Mask the email before inference. Restrict access to retained prompts and verify the configured retention path.

4. Test: Measure residual sensitive data in model input and permitted access to retained logs.

**Failure test**

The serialized request or retained invocation record contains the unneeded email address.

Inspect the prompt before and after redaction, log access, and retention settings.

Mask the email before inference. Restrict access to retained prompts and verify the configured retention path.

**Experiment**

Compare task quality and residual PII on representative records before and after masking.

Measure residual sensitive data in model input and permitted access to retained logs.

PII detection can miss values that need context or custom patterns.

Which evidence confirms this failure's cause and supports the correction?

The email is not needed for the task. Confirm it is absent from the model request and inspect who can access any retained copy.

Sources [6, 11, 46, 34].

## 3.2.3 Privacy and utility balance

Minimize sensitive content while preserving fields that carry task meaning. Define a utility measure and privacy test before transformation.

Use masking, token replacement, aggregation, or anonymization based on the task. Keep any re-identification map in a separate restricted system.

**Decision**

When: A workflow must classify support themes without exposing customer names.

Replace names with stable task-local tokens or remove them if identity is unnecessary.

Task-local tokens can preserve entity relationships without exposing names. Quality and re-identification tests reveal the remaining tradeoff.

Do not remove every entity blindly. That can destroy task meaning and still leave identity clues in other fields.

**Causal trace**

1. Case: A workflow must classify support themes without exposing customer names.

2. Check: Compare sensitive fields before and after masking, then inspect task output quality.

3. Act: Keep only needed attributes. Replace names with task-local tokens when relationships matter, then test utility and re-identification risk.

4. Test: Measure task quality and residual identity risk after the selected data transformation.

**Failure test**

After name masking, classification quality falls or transformed records still identify a person through linked fields.

Compare sensitive fields before and after masking, then inspect task output quality.

Keep only needed attributes. Replace names with task-local tokens when relationships matter, then test utility and re-identification risk.

**Experiment**

Compare classification quality and re-identification exposure across transformed test records.

Measure task quality and residual identity risk after the selected data transformation.

Anonymization strength depends on the data and context.

Which evidence confirms this failure's cause and supports the correction?

Compare raw and transformed records. Check whether useful relationships remain and whether linked fields still identify a person.

Sources [6, 11, 46].

## 3.3.1 Compliance framework

Translate each applicable obligation into an owner, control, evidence record, and review cycle. Confirm the actual regulatory scope with the organization.

Document intended use, limitations, data lineage, model and prompt versions, approvals, and retention. Use suitable catalog and registry metadata.

**Decision**

When: A reviewer asks why a model used one dataset and which release served a decision.

Trace the deployed version to its approved data, prompt, evaluation, and change record.



Release-linked data, prompt, evaluation, and approval records let a reviewer reconstruct the deployed decision path.

A model card proves deployed release lineage even when its version does not match the running model.

**Causal trace**

1. Case: A reviewer asks why a model used one dataset and which release served a decision.

2. Check: Trace the deployed version to its model card, data lineage, approval, and audit record.

3. Act: Trace the deployed release to versioned data, prompt, evaluation, approval, and change records.

4. Test: Measure whether a selected release traces to complete approval, lineage, and audit evidence.

**Failure test**

The deployed model version cannot be linked to its data, prompt, evaluation, or approval record.

Trace the deployed version to its model card, data lineage, approval, and audit record.

Trace the deployed release to versioned data, prompt, evaluation, approval, and change records.

**Experiment**

Select a release and trace every declared dependency to a versioned source record.

Measure whether a selected release traces to complete approval, lineage, and audit evidence.

A course example does not establish legal compliance for a deployment.

Which evidence confirms this failure's cause and supports the correction?

The reviewer needs the records tied to the deployed version. Follow each release reference and report any missing lineage or approval link.

Sources [6, 46, 40, 30].

## 3.3.2 Data source traceability

Preserve the origin and version of evidence used to answer a question. Define what the user can see and what the audit needs.

Register sources, stable document identifiers, versions, effective dates, and retrieval references. Attach citations to outputs where supported.

**Decision**

When: A user challenges a generated policy answer after a source changed.

Trace the answer citation to the document version retrieved for that request.

A versioned passage reference ties a challenged answer to the source state used at request time.

Do not cite only a filename. The file can change after retrieval, so the citation cannot identify the evidence used.

**Causal trace**

1. Case: A user challenges a generated policy answer after a source changed.

2. Check: Trace the cited document version to catalog metadata, ingestion event, and request record.

3. Act: Trace the citation to the retrieved document version and passage for that request.

4. Test: Measure citation version accuracy after a source update and a retrieval request.

**Failure test**

The answer cites a filename, but the request record cannot identify the retrieved version or passage.

Trace the cited document version to catalog metadata, ingestion event, and request record.

Trace the citation to the retrieved document version and passage for that request.

**Experiment**

Change one source version and verify that both retrieval records and answer citations distinguish it.

Measure citation version accuracy after a source update and a retrieval request.

A trace record may contain sensitive content and needs access controls.

Which evidence confirms this failure's cause and supports the correction?

A filename does not identify the request-time evidence. Compare the citation with the retrieval record, ingestion version, and source passage.

Sources [39, 46, 10, 20, 30].

## 3.3.3 Organizational AI governance

Set policy ownership before a team builds a production model workflow. Define allowed uses, restricted data, exceptions, and escalation roles.

Create a review path for use case approval, data classification, model selection, testing, release, and incident response. Record decisions and assign control owners.

**Decision**

When: Two teams apply different rules to the same sensitive use case.

Use one approved policy and a named exception owner for any deviation.



One approved rule and a named exception owner reduce conflicting treatment and make deviations reviewable.

Do not leave policy ownership to each team. Local interpretations create inconsistent controls for the same use case.

**Causal trace**

1. Case: Two teams apply different rules to the same sensitive use case.

2. Check: Inspect approved use policy, control owner, exceptions, and evidence of enforcement.

3. Act: Publish one approved policy. Assign an owner to review and record every exception.

4. Test: Measure control adherence and exception handling for one approved use case.

**Failure test**

Two teams apply different controls to the same approved sensitive use case.

Inspect approved use policy, control owner, exceptions, and evidence of enforcement.

Publish one approved policy. Assign an owner to review and record every exception.

**Experiment**

Trace one use case from approval to production and identify each accountable owner and proof record.

Measure control adherence and exception handling for one approved use case.

Governance language without an operational control does not change system behavior.

Which evidence confirms this failure's cause and supports the correction?

The same use case receives conflicting controls. Compare both decisions with the approved rule and route any exception to its named owner.

Sources [6, 46, 40].

## 3.3.4 Continuous governance monitoring

Monitor policy compliance and model behavior after release. Define a measurable signal, alert owner, and safe response.

Track misuse, quality drift, bias indicators, sensitive data exposure, and policy violations. Redact telemetry where possible.

**Decision**

When: A quality segment declines while overall task success remains steady.

Use segment-level alerts to route a review before a broad rollback.

Segment alerts can reveal a localized regression that an overall average conceals. An owner can review evidence before wider action.

Do not rely on launch approval or an overall average. Neither detects later segment drift by itself.

**Causal trace**

1. Case: A quality segment declines while overall task success remains steady.

2. Check: Inspect segment-level quality alerts, policy events, assigned owner, and response record.

3. Act: Add a segment-level signal and alert owner. Route the alert for review before broad rollback.

4. Test: Measure detection delay, segment coverage, and owner response to a known policy event.

**Failure test**

One user segment has lower quality, but the aggregate dashboard remains within its usual range.

Inspect segment-level quality alerts, policy events, assigned owner, and response record.

Add a segment-level signal and alert owner. Route the alert for review before broad rollback.

**Experiment**

Inject a known segment regression in a controlled test and verify alert routing and response.

Measure detection delay, segment coverage, and owner response to a known policy event.

Automated remediation can create new failures when its trigger is noisy.

Which evidence confirms this failure's cause and supports the correction?

The aggregate hides the segment decline. Check segment results, alert thresholds, and owner response before deciding on a wider rollback.

Sources [28, 46, 40, 30].

## 3.4.1 Transparent model outputs

Show evidence and limits that help a user judge an answer. Choose explanations that the system can support with records.

Provide source references, uncertainty statements tied to validated measures, and relevant tool outcomes. Keep internal traces for diagnosis under restricted access.

**Decision**

When: A user needs to know which policy passage supports a response.

Show the passage reference and state when the source does not resolve the question.

A source reference and a clear limit give users inspectable support without exposing hidden internal reasoning.



Do not present fluent reasoning or a confidence claim as proof. It may lack verifiable evidence and can expose private internal content.

**Causal trace**

1. Case: A user needs to know which policy passage supports a response.

2. Check: Trace displayed evidence to its source passage and inspect the basis for uncertainty.

3. Act: Show the supporting passage and state when it does not resolve the question.

4. Test: Measure whether users can find supporting evidence and understand stated limits.

**Failure test**

The interface shows an explanation but gives no source passage for the policy claim.

Trace displayed evidence to its source passage and inspect the basis for uncertainty.

Show the supporting passage and state when it does not resolve the question.

**Experiment**

Remove source evidence from a test and check whether the UI communicates the missing support.

Measure whether users can find supporting evidence and understand stated limits.

A confidence number is useful only when its measure is validated for the task.

Which evidence confirms this failure's cause and supports the correction?

The user needs evidence they can inspect. Link the answer to its passage and state the unresolved part instead of exposing private internal traces.

Sources [47, 46, 20, 30].

## 3.4.2 Fairness evaluation

Define affected groups, meaningful outcomes, and relevant comparison criteria before testing. Use representative data and privacy safeguards.

Compare quality, refusal, error, and safety outcomes across justified segments. Review examples with domain experts.

**Decision**

When: A classifier serves users with different dialects and has uneven error patterns.

Compare errors by relevant language segment and inspect representative failures.

Group-level error and refusal checks can reveal unequal outcomes hidden by an improved average.

Do not approve on aggregate quality alone. It can improve while one affected group receives more errors or refusals.

**Causal trace**

1. Case: A classifier serves users with different dialects and has uneven error patterns.

2. Check: Compare labeled outcomes and refusal errors across the selected user segments.

3. Act: Compare justified group segments and inspect representative errors before approval.

4. Test: Compare error and refusal rates across justified groups against the baseline.

**Failure test**

The average score improves, but labeled results show more incorrect refusals for one relevant group.

Compare labeled outcomes and refusal errors across the selected user segments.

Compare justified group segments and inspect representative errors before approval.

**Experiment**

Change one data or prompt factor and measure segment-level outcomes against the baseline.

Compare error and refusal rates across justified groups against the baseline.

A metric cannot define fairness without context and stakeholder judgment.

Which evidence confirms this failure's cause and supports the correction?

The subgroup error pattern conflicts with the average. Verify the labels and compare the same outcome across relevant groups.

Sources [28, 21, 46, 40].

## 3.4.3 Responsible AI policy checks

Convert responsible AI requirements into controls that can be tested. Identify which requirements need human judgment.

Use policy-specific filters, model documentation, deterministic checks, and review workflows. Keep the policy version linked to test evidence and release approval.

**Decision**

When: An answer may be accurate but violate an approved use restriction.

Apply the relevant policy rule and route uncertain cases for review.

Mapping each policy rule to an enforceable check and escalation path makes compliance observable in the workflow.



Do not use a generic safety score as proof of policy compliance. It may not test the required business rule.

**Causal trace**

1. Case: An answer may be accurate but violate an approved use restriction.

2. Check: Map the policy requirement to its control, evaluation evidence, and release decision.

3. Act: Map the policy rule to a deterministic check or review step. Record its result and escalation owner.

4. Test: Measure whether policy edge cases are blocked, allowed, or routed for review as defined.

**Failure test**

A policy violation passes the generic safety filter because no control checks that specific rule.

Map the policy requirement to its control, evaluation evidence, and release decision.

Map the policy rule to a deterministic check or review step. Record its result and escalation owner.

**Experiment**

Add a policy edge case and confirm the system blocks, allows, or escalates it as specified.

Measure whether policy edge cases are blocked, allowed, or routed for review as defined.

Policy controls do not replace accountable review of the policy itself.

Which evidence confirms this failure's cause and supports the correction?

The generic filter does not test this obligation. Identify the missing rule-specific control and verify its result on the violation case.

Sources [28, 46, 40, 34].

## 4.1.1 Token efficiency

Reduce unnecessary input and output while preserving task evidence and answer completeness. Establish the quality floor first.

Estimate input and output tokens by request class. Remove duplicate context, stale history, and irrelevant retrieved text.

**Decision**

When: Long conversation history drives cost but contains few relevant turns.

Keep a tested summary and the recent turns needed for the current task.

A tested summary retains required facts while removing irrelevant history. Token and completion checks show whether quality survives.

Do not delete old context by position alone. The removed turn may contain a constraint needed for a correct answer.

**Causal trace**

1. Case: Long conversation history drives cost but contains few relevant turns.

2. Check: Inspect input and output token counts, context content, and required output fields.

3. Act: Keep a tested summary and the recent turns needed for the task. Verify required facts in the serialized request.

4. Test: Measure tokens per successful task and verify required answer fields remain complete.

**Failure test**

The serialized request omits a required qualifier after context reduction, or the output lacks a required field.

Inspect input and output token counts, context content, and required output fields.

Keep a tested summary and the recent turns needed for the task. Verify required facts in the serialized request.

**Experiment**

Reduce one context component and compare quality, completeness, and token use on fixed cases.

Measure tokens per successful task and verify required answer fields remain complete.

Lower token use is not a win if task quality falls below its floor.

Which evidence confirms this failure's cause and supports the correction?

Find whether the qualifier reached the model. Preserve it in the summary or recent context, then verify answer completeness.

Sources [3, 48, 17].

## 4.1.2 Cost-effective model choice

Choose the least costly model that meets the required quality, latency, modality, and availability constraints. Benchmark representative tasks.

Compare candidate models on the same test set. Measure successful completion, error types, input and output token work, latency, and current prices.

**Decision**

When: A small model handles routine classification but fails complex policy questions.

Use a tested route that escalates uncertain or high-impact cases.

A same-workload comparison reveals whether lower model price offsets errors, retries, and review effort.

Do not choose by unit price alone. A cheaper candidate can fail quality floors and raise total task cost.



**Causal trace**

1. Case: A small model handles routine classification but fails complex policy questions.

2. Check: Compare candidate quality on one task set with price and cost per successful task.

3. Act: Compare candidates on the same cases and settings. Keep only candidates that meet quality and safety floors.

4. Test: Compare cost per successful task while enforcing the same quality floor.

**Failure test**

The lower-priced candidate causes more failed cases, retries, or human review on the representative workload.

Compare candidate quality on one task set with price and cost per successful task.

Compare candidates on the same cases and settings. Keep only candidates that meet quality and safety floors.

**Experiment**

Compare cost per successful task for a fixed workload across candidate models.

Compare cost per successful task while enforcing the same quality floor.

Price alone does not measure business value.

Which evidence confirms this failure's cause and supports the correction?

Unit price omits failure costs. Compare quality, retries, review, latency, and cost on identical cases before selecting the candidate.

Sources [18, 48, 19, 17].

## 4.1.3 Capacity and throughput

Match capacity to request shape and service quotas. Separate request rate, token rate, concurrency, and monthly cost.

Measure arrival bursts, input and output tokens, latency, and concurrency by workload. Assess supported batch or provisioned options against measured demand.

**Decision**

When: Peak demand triggers throttling even though average usage is low.

Use bounded queues or suitable capacity after identifying the limiting quota dimension.

A measured workload profile supports capacity choice and bounded scaling. Queue and throttle signals reveal overload.

Do not buy fixed capacity or add callers before measuring demand. Either choice can waste capacity or deepen throttling.

**Causal trace**

1. Case: Peak demand triggers throttling even though average usage is low.

2. Check: Inspect throttling events, request and token rates, queue depth, and concurrency.

3. Act: Measure request rate, token demand, concurrency, and bursts. Set capacity and scaling limits against those signals.

4. Test: Measure useful throughput, throttles, queue age, and capacity use during representative peaks.

**Failure test**

Queue age or throttling rises while configured capacity does not match request and token demand.

Inspect throttling events, request and token rates, queue depth, and concurrency.

Measure request rate, token demand, concurrency, and bursts. Set capacity and scaling limits against those signals.

**Experiment**

Replay a representative peak trace and observe throttles, queue delay, and utilization.

Measure useful throughput, throttles, queue age, and capacity use during representative peaks.

Quota and pricing vary by model and service mode.

Which evidence confirms this failure's cause and supports the correction?

The queue and throttle evidence show demand exceeds the current path. Compare workload shape with capacity before changing allocation.

Sources [14, 38, 9, 48].

## 4.1.4 Caching for inference

Cache only when reuse is likely and the cache key preserves authorization and freshness. Choose prompt, exact result, or semantic result caching by behavior.

Prompt caching can reuse supported repeated prefixes under service rules. Exact result caches use deterministic request identity.

**Decision**

When: Many requests repeat a stable system prefix but ask different user questions.

Assess prompt caching for the shared prefix and measure supported cache hits.

A cache key that includes identity and source state prevents reuse across users or stale source versions.

Do not cache only by semantic similarity. That can return a stale answer or expose another user's result.



**Causal trace**

1. Case: Many requests repeat a stable system prefix but ask different user questions.

2. Check: Inspect cache key, tenant scope, source version, hit status, and freshness.

3. Act: Include authorization and freshness state in the cache key. Invalidate entries when their source or access state changes.

4. Test: Measure safe cache hits, stale results, cross-scope results, and end-to-end latency.

**Failure test**

A cache hit returns an answer after its source changed or for a caller with different access.

Inspect cache key, tenant scope, source version, hit status, and freshness.

Include authorization and freshness state in the cache key. Invalidate entries when their source or access state changes.

**Experiment**

Change one cache policy and observe hit rate, stale-answer rate, and total latency.

Measure safe cache hits, stale results, cross-scope results, and end-to-end latency.

Caching can amplify stale or cross-tenant data exposure.

Which evidence confirms this failure's cause and supports the correction?

The cached answer does not match current source or caller scope. Compare key inputs and invalidation events before allowing reuse.

Sources [48, 20, 37].

## 4.2.1 Latency and cost tradeoffs

Optimize the critical path against a stated user deadline. Identify which delay matters to the user.

Measure time to first token and total response time by stage. Precompute stable results where freshness permits.

**Decision**

When: A user sees no response while independent retrieval steps run in sequence.

Parallelize independent retrieval work and preserve a bounded end-to-end deadline.

Stage timing distinguishes first-token delay from total completion delay and shows whether parallel work reduces the critical path.

Do not claim streaming shortens total work. It can improve first-token time while retrieval still delays completion.

**Causal trace**

1. Case: A user sees no response while independent retrieval steps run in sequence.

2. Check: Inspect stage timestamps, first-token time, full latency, and request cost.

3. Act: Measure each stage and parallelize only independent retrieval work. Keep the end-to-end deadline and quality checks.

4. Test: Measure first-token time, completion time, quality, and cost after the path change.

**Failure test**

First-token time improves, but full completion remains late because a retrieval stage dominates.

Inspect stage timestamps, first-token time, full latency, and request cost.

Measure each stage and parallelize only independent retrieval work. Keep the end-to-end deadline and quality checks.

**Experiment**

Change one critical-path stage and compare first-token time, total time, quality, and cost.

Measure first-token time, completion time, quality, and cost after the path change.

Lower first-token latency does not prove faster task completion.

Which evidence confirms this failure's cause and supports the correction?

The timestamps separate perceived response time from completion time. Fix the slow stage and compare both latency measures and quality.

Sources [33, 38, 48, 32].

## 4.2.2 Retrieval performance

Tune retrieval for both evidence quality and query time. Use labeled queries that reflect real terms and access filters.

Measure recall and precision before generation. Inspect filters, chunk boundaries, lexical and vector signals, index layout, and reranker use.

**Decision**

When: Exact product codes are missed by semantic-only retrieval.

Add or tune lexical matching within a tested hybrid retrieval path.

Labeled queries reveal relevance loss while fixed access filters preserve the required security boundary.

Do not reduce top-k or remove filters based only on latency. That can hide required evidence or expose restricted data.

**Causal trace**

1. Case: Exact product codes are missed by semantic-only retrieval.



2. Check: Inspect labeled relevant results, access filters, index version, recall, and latency.

3. Act: Hold access filters fixed. Tune retrieval against labeled relevance and latency results.

4. Test: Measure labeled recall, precision, and latency after one retrieval configuration change.

**Failure test**

Relevant passages disappear from top results or access-filter changes alter which users can retrieve them.

Inspect labeled relevant results, access filters, index version, recall, and latency.

Hold access filters fixed. Tune retrieval against labeled relevance and latency results.

**Experiment**

Change one index or query parameter and compare recall, precision, and latency.

Measure labeled recall, precision, and latency after one retrieval configuration change.

A fast retrieval result is poor if it omits required evidence.

Which evidence confirms this failure's cause and supports the correction?

The missing labeled passage shows a retrieval defect. Compare ranks and latency without weakening caller access filters.

Sources [21, 48, 20, 16].

## 4.2.3 Throughput optimization

Increase completed useful work without violating quality or service limits. Identify the bottleneck before adding concurrency.

Use batch inference for suitable offline work. Bound concurrent calls for interactive work.

**Decision**

When: Workers issue more parallel requests, then throttling causes retries and lower completion.

Reduce or shape concurrency and use a queue aligned with capacity.

Bounded concurrency protects service capacity while throughput tests expose queue growth and throttling.

Do not raise worker count without a concurrency limit. More callers can increase throttling and queue age.

**Causal trace**

1. Case: Workers issue more parallel requests, then throttling causes retries and lower completion.

2. Check: Inspect completed throughput, throttling, retries, and oldest queue age.

3. Act: Set a concurrency bound. Increase throughput only when fixed-workload tests show stable errors and queue age.

4. Test: Measure successful requests, token throughput, retries, and queue age after the capacity change.

**Failure test**

Request rate rises, but errors or queue age also rise under the higher worker count.

Inspect completed throughput, throttling, retries, and oldest queue age.

Set a concurrency bound. Increase throughput only when fixed-workload tests show stable errors and queue age.

**Experiment**

Change one batch size or concurrency limit and compare throughput, failures, and completion time.

Measure successful requests, token throughput, retries, and queue age after the capacity change.

Maximum submitted requests do not equal maximum successful throughput.

Which evidence confirms this failure's cause and supports the correction?

The queue and error trace show overload rather than insufficient worker count alone. Bound callers and measure completion rate and queue age.

Sources [14, 4, 38, 48].

## 4.2.4 Inference parameter tuning

Tune model parameters for the task and model contract. Keep prompts, data, and evaluation fixed during a comparison.

Change temperature, top-p, top-k, or output limit only where the chosen model supports them. Assess repeatability, diversity, schema validity, and task correctness.

**Decision**

When: A classification task produces variable labels across repeated calls.

Test lower variation settings with schema and label checks.

A controlled parameter change isolates its effect on output quality and repeatability for the supported model.

Do not assume temperature zero guarantees correctness or identical output across systems. It controls only a supported sampling setting.

**Causal trace**

1. Case: A classification task produces variable labels across repeated calls.

2. Check: Compare fixed test outputs, schema validity, and the exact inference parameters.



3. Act: Change one supported parameter at a time. Compare quality and variability on fixed cases.

4. Test: Measure correctness, repeatability, and schema validity after one parameter change.

**Failure test**

A parameter change improves style but reduces required content or uses a value the operation does not support.

Compare fixed test outputs, schema validity, and the exact inference parameters.

Change one supported parameter at a time. Compare quality and variability on fixed cases.

**Experiment**

Change one parameter and compare repeated outputs on a fixed labeled set.

Measure correctness, repeatability, and schema validity after one parameter change.

Parameter behavior differs by model and request API.

Which evidence confirms this failure's cause and supports the correction?

Check the request schema and fixed-case results. A supported setting can alter output behavior, but it cannot establish factual correctness.

Sources [3, 48, 17].

## 4.2.5 Resource allocation

Allocate compute and concurrency to observed workload classes. Include burst behavior and queue delay.

Monitor utilization, token patterns, request concurrency, and response time. Scale the relevant worker or endpoint while respecting upstream model capacity.

**Decision**

When: A small number of very long prompts saturates token capacity.

Scale or shape work using token and concurrency signals, not request count alone.

Queue and token demand provide a closer allocation signal than a single host metric for model workloads.

Do not scale workers without checking service quotas. Extra callers can increase throttling instead of completed work.

**Causal trace**

1. Case: A small number of very long prompts saturates token capacity.

2. Check: Inspect token rate, concurrency, queue age, scaling events, and downstream throttles.

3. Act: Scale against measured queue and token demand. Keep caller concurrency within the confirmed service capacity.

4. Test: Measure queue age, token saturation, throttle rate, and worker utilization after scaling.

**Failure test**

Workers increase, but queue age and throttling stay high or rise.

Inspect token rate, concurrency, queue age, scaling events, and downstream throttles.

Scale against measured queue and token demand. Keep caller concurrency within the confirmed service capacity.

**Experiment**

Change one scaling signal and compare queue age, saturation, and service errors.

Measure queue age, token saturation, throttle rate, and worker utilization after scaling.

Scaling policy needs a safe upper bound and a quota check.

Which evidence confirms this failure's cause and supports the correction?

The added workers did not clear the queue. Check throttling and token demand, then cap callers before allocating more resources.

Sources [14, 38, 48].

## 4.2.6 Workflow performance

Profile the whole workflow before tuning model inference. Trace each service call and dependency.

Measure retrieval, reranking, network, model, and tool durations. Remove duplicate calls and parallelize only independent work.

**Decision**

When: Model time is small, but a workflow spends most time on serial tool calls.

Reduce duplicate calls or parallelize independent tool operations.

Stage timing reveals the actual critical path, so independent tasks can run concurrently without changing required order.

Do not replace the model first when trace time shows sequential tool calls dominate. That leaves the bottleneck intact.

**Causal trace**

1. Case: Model time is small, but a workflow spends most time on serial tool calls.

2. Check: Compare stage traces and duplicate calls against the end-to-end request path.



3. Act: Parallelize independent stages. Preserve dependencies and validate final data freshness and task outcome.

4. Test: Measure end-to-end latency and correctness after changing the identified slow stage.

**Failure test**

Trace spans show idle wait between independent retrieval or tool stages while model time remains small.

Compare stage traces and duplicate calls against the end-to-end request path.

Parallelize independent stages. Preserve dependencies and validate final data freshness and task outcome.

**Experiment**

Change one stage and compare end-to-end latency and task correctness.

Measure end-to-end latency and correctness after changing the identified slow stage.

A faster component may not reduce total path time.

Which evidence confirms this failure's cause and supports the correction?

The trace shows waiting between independent stages. Parallelize those stages and verify that the result still uses current evidence.

Sources [48, 30, 32, 16].

## 4.3.1 End-to-end observability

Connect technical signals to user outcomes with a shared request identifier. Define operational and business questions first.

Trace gateway, retrieval, model, and tools. Record latency, errors, token use, completion, and user outcome measures.

**Decision**

When: Success rate drops while model API success remains stable.

Follow the request trace through retrieval and tools to find where task completion changed.

A shared request identifier links application, model, retrieval, and tool events for end-to-end diagnosis.

Do not infer answer quality from service health metrics alone. Those metrics do not measure factual support or task success.

**Causal trace**

1. Case: Success rate drops while model API success remains stable.

2. Check: Follow one request ID from business outcome through service metrics and traces.

3. Act: Propagate a request identifier across stages. Add evaluated task metrics and protect sensitive traces.

4. Test: Measure whether the new trace links a known task failure to its responsible stage.

**Failure test**

A failed answer cannot be traced across application and model events because correlation identifiers are missing.

Follow one request ID from business outcome through service metrics and traces.

Propagate a request identifier across stages. Add evaluated task metrics and protect sensitive traces.

**Experiment**

Add one stage-level metric and verify it explains a known test failure.

Measure whether the new trace links a known task failure to its responsible stage.

Correlation IDs do not make sensitive trace data safe by themselves.

Which evidence confirms this failure's cause and supports the correction?

Service health does not explain answer quality. Join the request trace and evaluated outcome to locate the failing stage.

Sources [39, 48, 40, 30].

## 4.3.2 GenAI monitoring

Monitor both service health and behavior quality. Choose indicators that can trigger a useful response.

Track token use, latency, failures, supported invocation logs, prompt and model versions, and quality signals from evaluation. Set anomaly thresholds from baseline traffic.

**Decision**

When: Token use rises after a prompt release while completion quality declines.

Compare versioned traces and evaluation results, then inspect the prompt change.

Workload-specific baselines and access-controlled traces help distinguish quality regression from harmless traffic change.

Do not alert on one global threshold without checking workload mix. A harmless shift can trigger noise or hide a segment failure.

**Causal trace**

1. Case: Token use rises after a prompt release while completion quality declines.

2. Check: Compare invocation logs, token trends, release versions, and the established baseline.

3. Act: Set baselines and alert thresholds by relevant workload. Route alerts to an owner and restrict trace access.



4. Test: Measure alert precision and detection of a known token or quality regression.

**Failure test**

Alerts fire on normal traffic changes or miss a quality decline in one task segment.

Compare invocation logs, token trends, release versions, and the established baseline.

Set baselines and alert thresholds by relevant workload. Route alerts to an owner and restrict trace access.

**Experiment**

Inject a known token increase and check alert precision and owner response.

Measure alert precision and detection of a known token or quality regression.

Invocation logging support and fields can vary by service path.

Which evidence confirms this failure's cause and supports the correction?

Compare alert behavior with labeled workload segments. Check whether the threshold detects the regression without exposing prompt content broadly.

Sources [39, 28, 48, 30].

## 4.3.3 Actionable observability

Make dashboards answer an operational question and point to an owner. Join behavior, business, and compliance evidence carefully.

Present trends by version, task type, and justified segment. Link alerts to trace samples with sensitive values redacted.

**Decision**

When: Stakeholders see a quality trend but cannot find the affected release.

Add version and task labels that link aggregate metrics to a controlled trace.

A threshold, owner, and response path turn a metric into an operator decision.

Do not add charts as a substitute for an owner and action threshold. More metrics alone do not resolve failures.

**Causal trace**

1. Case: Stakeholders see a quality trend but cannot find the affected release.

2. Check: Inspect the affected dashboard segment, source trace, threshold, and response owner.

3. Act: Define a threshold, name an owner, and document the response for each operational alert.

4. Test: Measure whether an operator can identify affected version, scope, and action from the dashboard.

**Failure test**

A dashboard shows a metric breach, but no owner or response action exists.

Inspect the affected dashboard segment, source trace, threshold, and response owner.

Define a threshold, name an owner, and document the response for each operational alert.

**Experiment**

Give an operator a known regression and observe whether the dashboard reveals its scope and next action.

Measure whether an operator can identify affected version, scope, and action from the dashboard.

A dashboard is evidence for diagnosis, not proof of cause.

Which evidence confirms this failure's cause and supports the correction?

The breach has no assigned response. Identify the metric limit, owner, and action, then test alert routing.

Sources [28, 48, 40, 30].

## 4.3.4 Tool performance

Measure tool use as part of agent behavior and service performance. Define valid arguments and useful outcomes.

Record tool name, normalized argument status, latency, retries, errors, and result class. Redact sensitive values.

**Decision**

When: An agent calls the same search tool repeatedly without new evidence.

Use traces to detect the loop and apply step or retry bounds.

The tool's business result can reveal rejection even when transport succeeds.

Do not treat HTTP success as task success. The response body can report a rejected or incomplete operation.

**Causal trace**

1. Case: An agent calls the same search tool repeatedly without new evidence.

2. Check: Inspect redacted tool arguments, retries, errors, and task completion traces.

3. Act: Validate the tool response contract and business outcome before reporting success to the user.

4. Test: Measure tool success, retry rate, repeated calls, and agent task completion.



**Failure test**

The tool returns HTTP success, but its result field reports that the requested business action failed.

Inspect redacted tool arguments, retries, errors, and task completion traces.

Validate the tool response contract and business outcome before reporting success to the user.

**Experiment**

Change one tool timeout or schema rule and compare completion, errors, and call count.

Measure tool success, retry rate, repeated calls, and agent task completion.

Tool telemetry should not retain secrets or unnecessary raw arguments.

Which evidence confirms this failure's cause and supports the correction?

Transport success proves only that the request received a response. Inspect the operation result and report failure when its outcome says so.

Sources [27, 47, 48, 30].

## 4.3.5 Vector store operations

Monitor ingestion health and query behavior together. Define freshness and retrieval quality needs.

Track sync failures, source versions, index size, query latency, filter behavior, and labeled retrieval quality. Optimize index layout only after measuring query and update patterns.

**Decision**

When: Query latency remains stable while a source update never reaches the index.

Inspect ingestion status and indexed version, then retry the failed sync.

Source version, sync status, and retrieved results distinguish a fresh index from an index that only responds quickly.

Do not infer freshness from query latency. Fast results can come from an index that has not received the update.

**Causal trace**

1. Case: Query latency remains stable while a source update never reaches the index.

2. Check: Compare source version, ingestion status, index freshness, and retrieved results.

3. Act: Compare source and indexed versions. Resolve ingestion errors and verify the updated passage appears in retrieval.

4. Test: Measure source freshness, labeled retrieval quality, sync errors, and query latency.

**Failure test**

The source version advances, but the indexed version or retrieved passage remains old.

Compare source version, ingestion status, index freshness, and retrieved results.

Compare source and indexed versions. Resolve ingestion errors and verify the updated passage appears in retrieval.

**Experiment**

Update a test document and verify freshness, retrieval, and deletion signals end to end.

Measure source freshness, labeled retrieval quality, sync errors, and query latency.

Index health alone does not establish evidence correctness.

Which evidence confirms this failure's cause and supports the correction?

The version mismatch locates a stale index. Trace sync status and test a labeled query after the update.

Sources [48, 10, 20, 16].

## 4.3.6 GenAI failure diagnosis

Separate data, retrieval, prompt, model, tool, and application failures. Preserve a reproducible test case.

Replay a versioned golden input and compare evidence, request fields, model version, stop reason, tool trace, and output. Change one suspected cause at a time.

**Decision**

When: A response is wrong after a source document update.

Compare source ingestion, retrieved passages, prompt assembly, and model response in order.

A stage-by-stage trace finds the first divergence between expected evidence and the returned answer.

Do not rewrite the prompt first. A missing source, filter, or tool result can cause the same answer symptom.

**Causal trace**

1. Case: A response is wrong after a source document update.

2. Check: Replay the golden input and find the first difference in evidence, prompt, tool, or response.

3. Act: Trace source, retrieval, prompt, model, and tool stages. Fix the earliest stage that drops required evidence.

4. Test: Measure whether replay finds the first divergent stage and reproduces the error.



**Failure test**

The answer omits a current fact, but the team has not checked retrieval, prompt assembly, or tool output.

Replay the golden input and find the first difference in evidence, prompt, tool, or response.

Trace source, retrieval, prompt, model, and tool stages. Fix the earliest stage that drops required evidence.

**Experiment**

Freeze one failing request and vary one pipeline stage to identify the first divergence.

Measure whether replay finds the first divergent stage and reproduces the error.

One reproducible case can locate a defect but cannot prove broad quality.

Which evidence confirms this failure's cause and supports the correction?

Compare each stage with the expected evidence. Correct the first divergence before changing downstream instructions.

Sources [28, 48, 20, 30].

## 5.1.1 Quality assessment framework

Evaluate output on the task's real quality dimensions. Define acceptable evidence and failure costs.

Combine factual accuracy, relevance, completeness, consistency, and fluency as needed. Add safety and privacy criteria for the use case.

**Decision**

When: Answers are fluent but omit a required policy condition.

Measure completeness and factual support, not fluency alone.

Separate rubric dimensions expose omitted requirements that fluency or a single score can hide.

Fluency or one aggregate score proves completeness. Either can hide a missing fact or required condition.

**Causal trace**

1. Case: Answers are fluent but omit a required policy condition.

2. Check: Inspect the rubric, expected evidence, score dimensions, and failed examples.

3. Act: Score completeness and factual support separately against the fixed rubric.

4. Test: Measure agreement with expert labels for each declared quality dimension.

**Failure test**

The overall score passes although answers omit a required policy condition.

Inspect the rubric, expected evidence, score dimensions, and failed examples.

Score completeness and factual support separately against the fixed rubric.

**Experiment**

Add a completeness criterion and rescore a fixed sample against expert labels.

Measure agreement with expert labels for each declared quality dimension.

A metric is only useful when its target and limits are clear.

Which evidence confirms this failure's cause and supports the correction?

The missing policy condition is a completeness failure even when prose is fluent. Score the claim against evidence and required fields.

Sources [28, 21, 49].

## 5.1.2 Model comparison

Compare candidates under the same workload and evaluation contract. Include quality, latency, cost, safety, and business outcome.

Use representative held-out examples and fixed settings. Compare candidates with deterministic checks, calibrated judges, and human review where needed.

**Decision**

When: A candidate is cheaper but fails rare high-impact cases.

Keep the candidate only if it meets each required quality and safety floor.

Identical held-out cases and settings make model outcomes comparable while rare high-impact cases preserve required floors.

Do not rank candidates from different datasets or settings. The observed difference may come from the test setup.

**Causal trace**

1. Case: A candidate is cheaper but fails rare high-impact cases.

2. Check: Compare candidates on identical test data, settings, quality, latency, and cost.

3. Act: Run both candidates on the same versioned cases and settings. Reject any candidate below a required floor.

4. Test: Compare quality, safety, latency, cost, and business outcome on fixed candidate tests.

**Failure test**

The cheaper model appears better on a different sample but fails a rare required case.

Compare candidates on identical test data, settings, quality, latency, and cost.



Run both candidates on the same versioned cases and settings. Reject any candidate below a required floor.

**Experiment**

Change one model or configuration and compare on the same versioned dataset.

Compare quality, safety, latency, cost, and business outcome on fixed candidate tests.

Test evidence applies to the tested workload and settings.

Which evidence confirms this failure's cause and supports the correction?

The unequal tests cannot support a model ranking. Use the same held-out cases and compare quality, safety, latency, and cost.

Sources [18, 28, 21, 49].

## 5.1.3 User feedback evaluation

Collect feedback that identifies a task failure and supports action. Make feedback easy and safe to provide.

Capture task context, output version, rating, and error category. Provide a path for correction or escalation.

**Decision**

When: Users rate answers poorly but the team cannot tell whether retrieval or wording failed.

Add a small structured reason choice and optional explanation tied to the response version.

A structured reason tied to an output version helps distinguish retrieval, wording, and factual complaints.

Do not treat a thumbs-up or thumbs-down as ground truth. It reports a reaction without identifying the failure cause.

**Causal trace**

1. Case: Users rate answers poorly but the team cannot tell whether retrieval or wording failed.

2. Check: Inspect task and response versions, feedback choice, and assigned error category.

3. Act: Tie feedback to the response version. Ask for a short reason and protect any free-text details.

4. Test: Measure feedback completion, diagnostic value, and sensitive data exposure.

**Failure test**

Negative ratings accumulate, but no task version or reason identifies what failed.

Inspect task and response versions, feedback choice, and assigned error category.

Tie feedback to the response version. Ask for a short reason and protect any free-text details.

**Experiment**

Add one feedback field and observe completion rate, diagnostic value, and privacy exposure.

Measure feedback completion, diagnostic value, and sensitive data exposure.

User ratings are signals and do not replace objective evaluation.

Which evidence confirms this failure's cause and supports the correction?

The rating alone does not locate the defect. Use the response version and reason category to route the feedback for review.

Sources [28, 21, 49].

## 5.1.4 Quality assurance process

Turn quality expectations into repeatable release gates. Version datasets, prompts, model settings, and thresholds.

Run regression checks on fixed cases before release. Include critical segments and safety tests.

**Decision**

When: A prompt change fixes one case but breaks a previously correct workflow.

Run the fixed regression set before release and block on critical failures.

A fixed regression set makes prompt changes comparable and a critical-case gate prevents average scores from hiding failures.

Do not update the test set silently or gate only on an average. Either can hide a known regression.

**Causal trace**

1. Case: A prompt change fixes one case but breaks a previously correct workflow.

2. Check: Inspect dataset and configuration versions, failed gate, and baseline comparison.

3. Act: Run the versioned regression set and block release when a critical case fails.

4. Test: Measure whether a known regression fails the release gate before deployment.

**Failure test**

A prompt change passes the average score but fails a previously correct critical case.

Inspect dataset and configuration versions, failed gate, and baseline comparison.



Run the versioned regression set and block release when a critical case fails.

**Experiment**

Introduce a known regression and verify that the release gate blocks deployment.

Measure whether a known regression fails the release gate before deployment.

A passing test set cannot cover every possible user input.

Which evidence confirms this failure's cause and supports the correction?

The critical failure overrides the improved average. Check the baseline and require the regression gate to block release.

Sources [28, 49, 25].

## 5.1.5 Multi-perspective output evaluation

Use different evaluators for different claims. Define human, judge, and programmatic roles.

Apply deterministic checks to schemas and exact rules. Use a model judge for rubric-based comparisons after calibration.

**Decision**

When: A response has valid JSON but cites an unsupported policy claim.

Check schema validity and evidence support as separate dimensions.

Separate deterministic schema checks from evidence checks and judge scores so one evaluator cannot hide another failure class.

Do not accept one judge score as ground truth. A judge can miss unsupported claims or share systematic bias.

**Causal trace**

1. Case: A response has valid JSON but cites an unsupported policy claim.

2. Check: Compare deterministic results, judge scores, human labels, and disagreement cases.

3. Act: Run schema and evidence checks separately. Calibrate judge results against expert labels and inspect disagreements.

4. Test: Measure judge agreement, human disagreement, deterministic errors, and rubric coverage.

**Failure test**

The JSON schema passes, but expert review finds an unsupported factual claim and the judge score remains high.

Compare deterministic results, judge scores, human labels, and disagreement cases.

Run schema and evidence checks separately. Calibrate judge results against expert labels and inspect disagreements.

### Experiment

Compare judge decisions with expert labels on a fixed sample and inspect disagreements.

Measure judge agreement, human disagreement, deterministic errors, and rubric coverage.

Judge calibration is limited to the tested rubric and data.

Which evidence confirms this failure's cause and supports the correction?

Schema validity answers whether the shape is correct. The source comparison answers whether the claim is supported. Review both results and judge disagreements.

Sources [28, 21, 49, 36].

## 5.1.6 Retrieval quality testing

Test retrieval independently from generation. Label relevant evidence for representative questions.

Measure recall at k, precision at k, rank of the first relevant result, and retrieval latency as suitable. Inspect filters, chunking, and source versions on misses.

### Decision

When: The answer model cannot cite a passage that the retriever omitted.

Inspect retrieval results and recall before changing answer prompts.

Labeled retrieval results show whether the answer model received relevant evidence before generation.

Do not judge retrieval only by final answers. A model can guess correctly when retrieval misses the source.

### Causal trace

1. Case: The answer model cannot cite a passage that the retriever omitted.

2. Check: Inspect labeled relevant items, retrieved ranks, filters, and retrieval latency.

3. Act: Measure retrieval against labeled queries before scoring generated answers. Keep the access filters fixed.

4. Test: Measure recall, precision, first relevant rank, and latency on labeled queries.

### Failure test

The final answer is sometimes correct, but labeled relevant passages do not appear in retrieved results.

Inspect labeled relevant items, retrieved ranks, filters, and retrieval latency.

Measure retrieval against labeled queries before scoring generated answers. Keep the access filters fixed.



### Experiment

Change one retriever setting and compare labeled retrieval metrics and latency.

Measure recall, precision, first relevant rank, and latency on labeled queries.

Metric values depend on query labels and the selected k.

Which evidence confirms this failure's cause and supports the correction?

A correct guess does not prove retrieval worked. Compare returned ranks with relevant labels, then assess answer quality separately.

Sources [21, 49, 20, 16].

## 5.1.7 Agent evaluation

Evaluate task completion and action correctness, not just final text. Define allowed tools and side effects.

Use versioned scenarios with expected tool choices, arguments, order, authorization, and final result. Track steps, errors, retries, and time.

### Decision

When: The agent returns a correct summary but used an unauthorized tool.

Fail the test because action validity matters independently from answer quality.

Action traces expose unauthorized tool use that final-answer scoring cannot reveal.

Do not pass an agent because its final summary is correct. It may have used an unauthorized tool or argument.

### Causal trace

1. Case: The agent returns a correct summary but used an unauthorized tool.

2. Check: Inspect tool choice, arguments, action order, permission result, and task completion.

3. Act: Fail the case on the unauthorized action. Check tool choice, arguments, order, permission, and side effects.

4. Test: Measure valid tool choice, authorized arguments, correct action order, and completion.

### Failure test

The final response is correct, but the trace shows an unauthorized tool call.

Inspect tool choice, arguments, action order, permission result, and task completion.

Fail the case on the unauthorized action. Check tool choice, arguments, order, permission, and side effects.

### Experiment

Change one tool schema or policy and compare action traces on the same scenarios.

Measure valid tool choice, authorized arguments, correct action order, and completion.

A successful simulated trace does not prove live integration reliability.

Which evidence confirms this failure's cause and supports the correction?

The final summary does not erase an unsafe action. Inspect the trace and fail when tool use exceeds the approved permission.

Sources [47, 28, 49].

## 5.1.8 Evaluation reporting

Report enough context for stakeholders to make a release decision. State sample, versions, criteria, and limits.

Compare baseline and candidate across quality, safety, latency, cost, and relevant segments. Show uncertainty and important failures.

### Decision

When: A score improves but the report omits sample composition and test version.

Require dataset, rubric, model, and prompt versions before making a decision.

Versions, sample composition, segments, and failures let reviewers interpret whether a score supports the release decision.

Do not report only a precise average. It can hide sample bias, segment failures, and uncertainty.

### Causal trace

1. Case: A score improves but the report omits sample composition and test version.

2. Check: Inspect sample count, dataset version, rubric, segment results, and stated limits.

3. Act: Add dataset and configuration versions, sample context, segment results, and material failures to the report.

4. Test: Measure whether decision makers can interpret results with sample and segment context.

### Failure test

The report shows a higher average but omits test version, sample makeup, or critical failures.

Inspect sample count, dataset version, rubric, segment results, and stated limits.

Add dataset and configuration versions, sample context, segment results, and material failures to the report.

### Experiment

Add a segment breakdown and test whether the release recommendation changes.



Measure whether decision makers can interpret results with sample and segment context.

A chart cannot replace review of material failure examples.

Which evidence confirms this failure's cause and supports the correction?

The score lacks context for a release decision. Verify the sample, versions, segments, and failures before interpreting the change.

Sources [28, 21, 49].

## 5.1.9 Deployment validation

Validate a model or prompt update before expanding user exposure. Define stop conditions.

Run synthetic workflows and regression tests. Compare output, hallucination indicators, semantic drift, latency, and cost.

### Decision

When: A canary has acceptable latency but new unsupported claims.

Pause expansion and review quality evidence before rollout.

A quality regression is a stop signal even when canary latency remains acceptable. Rollback rules limit exposure.

Do not expand a canary because latency is healthy. That ignores unsupported claims and other quality failures.

### Causal trace

1. Case: A canary has acceptable latency but new unsupported claims.

2. Check: Compare canary and baseline for safety, quality, drift, latency, and cost.

3. Act: Pause expansion. Compare quality and safety with the baseline, then use the tested rollback path if required.

4. Test: Measure canary quality, safety, drift, latency, and cost against a fixed baseline.

### Failure test

The canary meets latency targets but produces unsupported claims absent from the baseline.

Compare canary and baseline for safety, quality, drift, latency, and cost.

Pause expansion. Compare quality and safety with the baseline, then use the tested rollback path if required.

### Experiment

Add one recent failure workflow to the release suite and verify canary response.

Measure canary quality, safety, drift, latency, and cost against a fixed baseline.

Canary results apply to its tested traffic and observation period.

Which evidence confirms this failure's cause and supports the correction?

Latency does not offset unsupported claims. Compare the canary output with the fixed baseline and stop expansion until quality passes.

Sources [28, 21, 49].

### 5.2.1 Context handling diagnosis

Find what content the model actually received. Count all prompt components and output reserve.

Inspect request size, truncation behavior, stop reason, retrieved chunks, and validation errors. Reduce irrelevant history or evidence.

#### Decision

When: The model omits the final instruction after a long conversation.

Inspect token use and prompt assembly, then remove or summarize tested history.

The serialized request and stop evidence distinguish input truncation from output token exhaustion.

Do not raise the output token limit before checking input assembly. That cannot restore omitted input and may exceed the request budget.

#### Causal trace

1. Case: The model omits the final instruction after a long conversation.

2. Check: Inspect serialized request, token budget, truncation evidence, and stop reason.

3. Act: Inspect token use and truncation. Preserve required context with a tested summary before tuning output limits.

4. Test: Measure required content retained in the serialized request and completeness of the answer.

#### Failure test

The final instruction is absent from the serialized request before inference.

Inspect serialized request, token budget, truncation evidence, and stop reason.

Inspect token use and truncation. Preserve required context with a tested summary before tuning output limits.

#### Experiment

Change one context component and verify required facts remain in the actual request.

Measure required content retained in the serialized request and completeness of the answer.

Token estimation can vary by model tokenizer and request structure.

Which evidence confirms this failure's cause and supports the correction?



The request omitted the instruction before generation. Fix context assembly and verify its presence before changing output limits.

Sources [22, 3, 49, 17].

### 5.2.2 Inference integration diagnosis

Classify API errors before changing model or prompt behavior. Preserve the exact request and caller context.

Check operation, model identifier, Region, permissions, request schema, size limits, throttling, timeout, and response parsing. Inspect relevant service errors and request IDs.

#### Decision

When: A request fails with ValidationException after an API field change.

Compare the serialized request with the operation's supported schema.

The exact operation schema identifies a malformed field, while the same-role replay tests the correction.

Do not retry an unchanged invalid request. Retries do not change the schema error or permission decision.

#### Causal trace

1. Case: A request fails with ValidationException after an API field change.

2. Check: Inspect operation, serialized fields, model identifier, role, Region, and error code.

3. Act: Compare serialized fields with the selected operation schema. Correct the invalid field before replay.

4. Test: Measure whether the corrected request succeeds under the same role, model, and Region.

#### Failure test

The same malformed request repeatedly returns ValidationException after the field change.

Inspect operation, serialized fields, model identifier, role, Region, and error code.

Compare serialized fields with the selected operation schema. Correct the invalid field before replay.

#### Experiment

Change one request field and replay the same call under the same role and Region.

Measure whether the corrected request succeeds under the same role, model, and Region.

Service errors can change. Use current service documentation and logs.

Which evidence confirms this failure's cause and supports the correction?

The repeated validation error identifies a request-shape problem. Check the operation and field types, then replay the corrected request once.

Sources [4, 3, 49, 5].

## 5.2.3 Prompt behavior diagnosis

Treat prompt quality as a testable interaction among instructions, evidence, examples, and output contract.

Use a fixed failure set and compare prompt versions. Check instruction conflicts, missing context, examples, variable substitution, and schema validation.

### Decision

When: A prompt update improves tone but causes required fields to disappear.

Compare schema validity and task criteria across the old and new prompt versions.

A one-component prompt comparison isolates which instruction change caused required fields to disappear.

Do not rewrite several instructions at once. That prevents causal attribution and can add conflicts.

### Causal trace

1. Case: A prompt update improves tone but causes required fields to disappear.

2. Check: Diff prompt versions and rendered variables, then inspect failures against the rubric.

3. Act: Diff the prompt and rendered variables. Change one component per test and score the fixed rubric.

4. Test: Measure rubric failures and schema validity after changing one prompt component.

### Failure test

Required fields disappear after a prompt update that changed several instructions and examples together.

Diff prompt versions and rendered variables, then inspect failures against the rubric.

Diff the prompt and rendered variables. Change one component per test and score the fixed rubric.

### Experiment

Change one instruction or example and measure exact failure categories on fixed cases.

Measure rubric failures and schema validity after changing one prompt component.

Prompt improvements may not transfer to unseen tasks.



Which evidence confirms this failure's cause and supports the correction?

The multi-change update obscures the cause. Compare rendered versions and isolate one changed component against the missing-field test.

Sources [3, 28, 49, 25].

## 5.2.4 Retrieval defect diagnosis

Trace a missing or poor answer backward from generation to indexed source data.

Check source existence, latest sync, extraction quality, chunk boundaries, metadata filters, embedding compatibility, and ranking. Compare query and document vector configuration.

### Decision

When: A current policy exists in storage but never appears in retrieved evidence.

Check ingestion state and filters, then inspect the indexed version and top results.

Source sync, index version, filters, and ranked results separate ingestion defects from query or generation defects.

Do not upgrade the model first. A model cannot retrieve a source absent from its evidence set or blocked by an access filter.

### Causal trace

1. Case: A current policy exists in storage but never appears in retrieved evidence.

2. Check: Inspect source sync, chunk boundaries, embedding settings, filters, and ranked evidence.

3. Act: Check ingestion state, indexed version, filters, and rankings. Fix the first stage that drops the current source.

4. Test: Measure source freshness, relevant retrieval, and latency after the ingestion or query fix.

### Failure test

The current source exists in storage, but sync or ranked retrieval evidence shows only an older version.

Inspect source sync, chunk boundaries, embedding settings, filters, and ranked evidence.

Check ingestion state, indexed version, filters, and rankings. Fix the first stage that drops the current source.

### Experiment

Update one source record and trace it through ingestion and retrieval with a labeled query.

Measure source freshness, relevant retrieval, and latency after the ingestion or query fix.

A successful sync event does not prove useful retrieval.

Which evidence confirms this failure's cause and supports the correction?

Storage presence does not prove index presence. Trace the source through sync and retrieval, then fix the first missing stage.

Sources [22, 49, 10, 20, 16].

### 5.2.5 Prompt maintenance diagnosis

Manage prompt templates as versioned application dependencies. Test variables, schema, and release links.

Inspect unresolved variables, incompatible request fields, prompt version, and changed instructions. Compare logs and traces with a known-good version.

#### Decision

When: A deployed prompt version returns literal placeholder text.

Inspect variable names and values at invocation, then test the rendered prompt.

Invocation-time bindings reveal why a valid stored template can render a literal placeholder at runtime.

Do not edit template wording before checking runtime bindings. The missing or stale value can persist after the edit.

#### Causal trace

1. Case: A deployed prompt version returns literal placeholder text.

2. Check: Inspect prompt version, variable bindings, request fields, schema result, and trace.

3. Act: Inspect deployed version and invocation variables. Correct bindings, then validate the rendered request.

4. Test: Measure rendered variable correctness and output validity for each prompt version.

#### Failure test

The deployed answer contains a literal placeholder although the stored template looks correct.

Inspect prompt version, variable bindings, request fields, schema result, and trace.

Inspect deployed version and invocation variables. Correct bindings, then validate the rendered request.

#### Experiment

Change one template variable contract and compare rendered requests and output validity.

Measure rendered variable correctness and output validity for each prompt version.



Logs can contain sensitive prompts and need strict access and retention.

Which evidence confirms this failure's cause and supports the correction?

The literal placeholder shows that rendering failed. Compare the deployed version and bound variable names and values before editing prompt text.

Sources [3, 49, 30, 25].

# Decision reference

## AIP-C01: the two-day course

Use this course for decisions, architecture, and recall. The repository reference provides the full source depth.

Read each core slide. Explain its decision aloud.

Reveal the answer after you attempt each recall prompt.

Use the repository reference for weak topics and all source questions.

Complete the mock without notes. Review each error before exam day.

AWS states that its published guide is non-exhaustive. This course cannot guarantee every exam topic or a pass.

Sources [1].

## Exam facts and score

The official guide identifies 65 scored questions and 10 unscored questions. Unscored questions are not identified.

Multiple choice has one correct answer and three distractors.

Multiple response requires all correct options for credit.

The score scale is 100 to 1,000. The minimum pass score is 750.

Unanswered questions count as incorrect. A guess has no penalty.

AWS lists 75 total questions and 180 minutes for the exam.

A scaled score of 750 does not mean 75 percent correct. Domain question counts can vary.

Sources [1, 50].

## Two-day route

This is a proposed study schedule. It assumes two full days and prior AWS application experience.

Day 1: spend 2 hours on foundations and model APIs.

Day 1: spend 4 hours on RAG, prompts, and agents.

Day 2: spend 3 hours on security, operations, and evaluation.

Day 2: reserve 3 hours for the repository mock and 2 hours for error review.

These times are study allocations, not evidence of readiness. Use recall results to move time toward weak skills.

Sources [1].

## Scenario decisions

The credited option must satisfy every stated requirement at once.

Identify the outcome, data source, trust boundary, and failure mode.

Mark hard limits for residency, response time, cost, and operations.

Eliminate options that fail one hard requirement.

Compare the remaining options against the requested objective.

A managed service is not automatically correct. A residency or control requirement can force another design.

Sources [1].

## Tokens, context, and inference

Inference predicts output from input. A token is a model-specific unit, not a fixed number of words.

The context budget includes instructions, conversation, retrieved data, tools, and output reserve.

maxTokens limits generated output. It does not shorten input.

Count tokens with a supported tokenizer or CountTokens API.

Compare models with representative inputs and the same quality threshold.

A larger context window does not guarantee that the model uses every fact well.

Sources [17, 3, 12].

## Prompt, RAG, tools, or customization

Select the smallest architecture that meets the task and its constraints.

Use prompts and examples for instructions, format, and task demonstrations.

Use RAG for external facts, current documents, and evidence references.

Use tools for current system state and authorized actions.

Consider customization for repeated behavior after prompt and retrieval tests.

Fine-tuning can encode knowledge, but it does not provide automatic freshness or reliable document citations.

Sources [20, 15, 12].

## Requirements and proof of concept

A proof of concept must test a business task with representative data and measurable acceptance limits.

Record accuracy, latency percentiles, cost per successful task, and safety requirements.

Include long inputs, ambiguous requests, unavailable tools, and restricted documents.



Compare a baseline prompt with RAG or tools when the task requires them.

Decide whether to continue from measured results.

Do not purchase fixed capacity before the workload and quality threshold are clear.

Sources [40, 28, 12].

## Reusable architecture and the GenAI Lens

The Generative AI Lens applies architectural guidance to the GenAI lifecycle.

Reuse a model gateway, prompt contract, evaluation harness, and telemetry schema.

Define ownership across data, application, model, and platform layers.

Review operational excellence, security, reliability, performance, cost, and sustainability.

Version infrastructure, prompts, retrieval configuration, and model configuration together.

A framework review does not replace measurable tests or application authorization.

Sources [40, 12].

## Model selection

Select a model from measured task performance and required features.

Check modality, context, language, tool use, structured output, and regional availability.

Benchmark factual quality and task completion on representative data.

Compare time to first token, total time, throughput, and cost.

Record unsupported features and fallback behavior before release.

The largest model and the newest model are not universal answers.

Sources [18, 28, 12].

## Model access and authorization

Model activation and IAM authorization are separate checks.

Many models use default access with the required Marketplace permissions.

Some providers or accounts require additional prerequisites.

Check model availability in the selected Region and account.

Check IAM, SCPs, permission boundaries, endpoint policy, and resource scope.

The repository claim that every model requires a manual access request is outdated. AccessDenied can reflect IAM policy failures.

Sources [19, 5, 12].

## Converse and InvokeModel

Converse provides a common message interface for supported models. InvokeModel uses model-specific payloads.

Use Converse for supported conversation and tool-use workloads.

Use InvokeModel for supported model-specific inference, such as embeddings.

Use the runtime client for inference and the correct agent-runtime client for retrieval or agents.

Validate the model feature matrix before a provider switch.

One Converse interface does not imply identical capabilities across models.

Sources [51, 3, 2, 12].

## Request and response contracts

Treat each model request and response as a versioned interface.

Converse uses modelId, messages, optional system instructions, and inferenceConfig.

Pass supported toolConfig, guardrailConfig, or outputConfig only when appropriate.

Read content blocks, stopReason, usage, and metrics.

Validate structure and business meaning before any downstream action.

Valid JSON does not establish factual accuracy, authorization, or safe tool parameters.

Sources [3, 43, 12].

## Temperature, top-p, and top-k

These controls affect token selection. They do not establish factual truth.

Lower temperature generally reduces output variation.

Top-p limits the candidate set by cumulative probability.

Top-k limits the candidate set by count when the model supports it.

Tune supported parameters with an evaluation set and fixed prompt versions.

Temperature zero does not guarantee identical output across all systems or correct facts.

Sources [17, 12].

## Configuration without a code release

Separate model selection policy from provider-specific code.

Use AppConfig for validated model and prompt configuration.

Route requests through an API and a stable internal model contract.

Use capability checks before a fallback or provider change.



Roll out configuration gradually with alarms and rollback.

Changing the model ID alone can break tool schemas, token limits, safety settings, or response formats.

Sources [51, 40, 12].

## Cross-Region inference and resilience

Inference profiles can route supported model requests across eligible Regions.

Check the profile destination set against residency rules.

Authorize the source and required destination model resources.

Combine bounded retries with fallback and a circuit breaker.

Test degraded operation when the preferred model is unavailable.

Cross-Region inference does not mean unrestricted global failover. A geography profile and a global profile have different boundaries.

Sources [9, 12].

## Customization choices

Customization changes model behavior or knowledge through an additional process.

Fine-tuning uses task examples and target outputs.

Continued pre-training uses domain data where the model and service support it.

Distillation transfers behavior from a teacher to a smaller student where supported.

LoRA and adapters reduce the trainable parameter set in supported model workflows.

Customization does not remove the need for retrieval, evaluation, or security controls.

Sources [15, 29, 12].

## Customized model lifecycle

Treat a custom model as a versioned production dependency.

Register model versions, input data versions, metrics, and approval status.

Use SageMaker Model Registry for governed SageMaker deployment workflows.

Test candidates with a canary and a rollback target.

Retire old versions after retention and recovery requirements permit it.

A successful training job does not prove safe or cost-effective production behavior.

Sources [29, 15, 12].

## Data quality before inference

Bad source data becomes bad retrieval and bad answers.

Check completeness, schema, duplicates, encoding, freshness, and document ownership.

Use Glue Data Quality for rules and quality metrics.

Use Lambda or SageMaker Processing for normalization and validation.

Quarantine failed records and emit actionable metrics.

An LLM can reformat text but can also alter facts. Verify extracted values against the source.

Sources [20, 12].

## Multimodal data preparation

Each modality needs a suitable extraction and validation path.

Use Textract for document text, forms, and tables where appropriate.

Use Transcribe for audio transcripts and Comprehend for entity or PII analysis.

Use Bedrock Data Automation for supported multimodal extraction workflows.

Preserve source location, timestamp, confidence, and access metadata.

OCR text can lose table relationships. An image-capable model does not automatically solve every layout error.

Sources [41, 20, 12].

## Vector search from first principles

An embedding maps content into a vector space. A vector index retrieves nearby representations.

Embed the document chunks and query with compatible model settings.

Store source ID, text reference, version, and access metadata beside each vector.

Use similarity search for conceptual matches.

Use lexical search for exact identifiers or terms when needed.

Embedding similarity is not a probability that a passage or answer is true.

Sources [20, 16, 12].

## Vector store selection

Select a store from retrieval, scale, transaction, and operational requirements.

OpenSearch supports search-focused architectures with vectors and lexical retrieval.

Aurora PostgreSQL with pgvector can combine relational data and vector queries.



Knowledge Bases supports a defined set of vector store integrations.

Validate filtering, hybrid search, dimensions, region, and cost for the chosen integration.

DynamoDB metadata storage does not automatically make DynamoDB a substitute for a supported vector index.

Sources [20, 16, 12].

## Metadata and authorized retrieval

Metadata gives each retrieved passage context and access boundaries.

Store tenant, document ID, version, effective date, language, and classification.

Derive tenant filters from authenticated identity on the server.

Apply access filters before evidence reaches the model.

Preserve source references through retrieval, response, and audit records.

A prompt instruction such as use only my tenant cannot replace authorization.

Sources [16, 20, 12].

## Vector scale and integration

Index layout and connector behavior affect relevance, isolation, and latency.

Partition by domain or tenant when the access and scale model requires it.

Balance shards and replicas against query and update traffic.

Use connectors, APIs, AppFlow, or custom extraction for enterprise sources.

Monitor index memory, refresh lag, query latency, and failed updates.

More shards can increase overhead. Tune from representative load tests.

Sources [20, 40, 12].

## Fresh vectors and document deletion

A source edit does not help answers until retrieval sees the current version.

Use version-aware incremental sync and change detection.

Retry failed ingestion with idempotent document identifiers.

Remove stale vectors and cached responses after source deletion or permission change.

Record ingestion status, update lag, and the last successful sync.

A successful source upload is not proof of successful indexing.

Sources [10, 12].

## Chunking choices

Chunk size trades focused retrieval against complete context.

Fixed-size chunks use a token limit and overlap.

Semantic chunks follow meaning changes and add processing cost.

Hierarchical chunks retrieve children and return broader parent context.

No chunking treats each document as one chunk and suits carefully prepared small documents.

Hierarchy can return fewer results than requested after parent replacement. More chunks are not always more evidence.

Sources [22, 12].

## Embedding selection and migration

An embedding model defines the space in which similarity has meaning.

Compare domain fit, language, dimensions, input length, latency, and storage cost.

Use the same compatible configuration for document and query vectors.

Batch embedding jobs where the pipeline permits it.

Re-embed and rebuild the index when a model change creates an incompatible space.

Equal dimensions do not imply compatible vector spaces.

Sources [20, 12].

## Hybrid retrieval and reranking

A search stage finds candidates. A reranker scores the candidates for the query.

Vector search matches meaning. Lexical search matches exact terms.

Hybrid search combines both signals where the integration supports it.

Retrieve a candidate set and rerank before prompt construction.

Balance final context size against recall, cost, and latency.

A reranker cannot recover a relevant passage that the candidate stage never returns.

Sources [16, 12].

## Query transformation and decomposition

A user question can require several retrieval operations.

Rewrite unclear queries without changing their meaning.

Expand terms when domain synonyms matter.

Decompose multi-part questions into evidence requests.

Aggregate evidence with source references and an explicit context budget.

Uncontrolled expansion can add irrelevant evidence or broaden data access.

Sources [16, 12].

## Retrieval APIs and tool contracts

A stable retrieval contract separates evidence access from answer generation.



Retrieve returns evidence for custom generation workflows.

RetrieveAndGenerate combines managed retrieval and generation with source references.

A function or MCP interface can expose controlled search to an agent.

Return source, access scope, score meaning, and failure status explicitly.

Converse does not automatically query a Knowledge Base. Connect retrieval through application logic or a supported managed workflow.

Sources [24, 52, 12].

## Evidence path

Trace each stage. Identify its input, output, access rule, and failure behavior.

Authenticate the caller and identify the task.

Search current authorized chunks with metadata filters.

Send bounded evidence and instructions to the model.

Check source support, citations, schema, and safety.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Source to index

Trace each stage. Identify its input, output, access rule, and failure behavior.

S3, connector, or approved enterprise change event.

Parse text and tables. Check data quality and permissions.

Preserve source version and use a compatible embedding model.

Upsert or delete vectors. Measure freshness and ingestion errors.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Tenant boundary

Trace each stage. Identify its input, output, access rule, and failure behavior.

Derive tenant and role on the server.

Apply source permissions before the model sees evidence.

Use approved model, prompt, and safety policy.

Validate evidence and restrict logs and caches.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Prompt structure and instruction boundaries

A prompt states the task, constraints, evidence, and required output.

Place stable behavior in system instructions where the model supports them.

Separate user content and retrieved evidence from trusted instructions.

Use examples for the exact task and output format.

Require source-supported answers and an explicit insufficient-evidence response.

Instruction hierarchy helps model behavior but does not create a security boundary.

Sources [25, 34, 12].

## Conversation state and clarification

Conversation history is application data with a token budget and an access scope.

Persist session records with authenticated ownership, such as in DynamoDB.

Summarize or prune history under tested rules.

Ask for missing required fields before a tool action.

Separate durable user preferences from short-lived request state.

Bedrock inference calls do not replace application session control. A summary can lose important constraints.

Sources [51, 12].

## Prompt management and regression

Prompts require version control, release control, and tests.

Use Prompt Management for reusable prompt resources and versions.

Store test datasets, parameter values, and evaluation results with the prompt version.

Use CI/CD or orchestration for approval steps and policy gates.

Track prompt access and operations with the appropriate logs.

A prompt resource does not establish a complete organizational approval workflow by itself.

Sources [25, 28, 12].

## Flows and complex prompt systems

Bedrock Flows connects supported workflow nodes in an explicit process.

Use sequential nodes for fixed prompt and retrieval stages.

Use conditions for branches with clear business rules.

Use reusable components and validated input-output contracts.

Compare a flow with Step Functions when integrations, retries, or approvals require broader orchestration.

A fixed workflow and an autonomous agent solve different control problems.

Sources [13, 12].



## Agent loop and state

An agent uses model output to choose a tool, observe its result, and continue toward a goal.

Maintain task state, conversation state, and durable memory separately.

Use ReAct-style cycles for action and observation.

Persist tool results with correlation IDs and the responsible identity.

Stop on success, a step limit, a deadline, or an unrecoverable error.

A model proposal is not an authorized action. The tool service must enforce permission and input validity.

Sources [47, 44, 53, 7].

## Strands, Agent Squad, and managed agents

Distinguish agent development frameworks from managed service components.

Strands provides an SDK for model-driven agent loops and tools.

Agent Squad supports coordination across agents and routing patterns.

Bedrock Agents provides managed orchestration with supported tools and knowledge sources.

AgentCore provides components for agent deployment and operation.

The framework name does not prove that memory, authorization, or evaluation is automatic.

Sources [44, 53, 7].

## Bounded agent execution

Agent reliability needs explicit stopping rules and resource limits.

Set a maximum step count, elapsed time, token budget, and tool budget.

Use timeouts and circuit breakers around unreliable tools.

Limit role permissions to approved resources and actions.

Return a clear partial result when a safe limit ends execution.

A system prompt that says do not loop cannot replace an enforced step limit.

Sources [47, 40, 7].

## Multi-agent coordination

Specialized agents can divide work when their interfaces and authority remain clear.

Use a supervisor for routing and result synthesis where that pattern fits.

Use specialized models or agents for distinct capabilities.

Validate collaborator outputs against a shared task contract.

Measure coordination cost, failure propagation, and task completion.

More agents add latency and failure paths. A single workflow can be better for a fixed process.

Sources [47, 53, 7].

## Human review and approvals

Human review controls consequential actions and unresolved uncertainty.

Pause the workflow before the side effect.

Show the exact proposed action, evidence, and relevant limits.

Bind approval to the actor, action, parameters, and expiry.

Record the decision and resume with an idempotency key.

Approval of a general goal does not validate changed parameters or a different transaction.

Sources [40, 7].

## Tool schemas and reliable side effects

A tool definition describes an interface. The implementation controls its behavior.

Define names, parameter types, required fields, and response types precisely.

Validate input, authorization, and business rules on the server.

Use idempotency for retryable write operations.

Return structured errors that distinguish retryable and permanent failures.

A timeout does not prove that a side effect failed. A blind retry can duplicate the action.

Sources [3, 47, 7].

## MCP client and server

MCP standardizes how a client discovers and uses server capabilities.

Expose narrow tools with explicit schemas and least privilege.

Use Lambda for suitable lightweight stateless integrations.

Use containers when long-lived or complex tool services require them.

Authenticate the client and authorize each downstream operation.

MCP is an interface protocol. It does not make every tool safe or every result trustworthy.

Sources [44, 53, 7].

## AgentCore component boundaries

Select AgentCore components by the responsibility each one supplies.

Runtime executes agents and tools in supported environments.

Gateway exposes supported tools through managed access interfaces.

Memory manages supported state and memory use cases.



Identity and observability address access and operational insight.

A managed runtime does not remove application-specific authorization or evaluation duties.

Sources [44, 27, 7].

## AWS agent build-time architecture

Separate configuration resources from runtime request processing.

Action groups define supported tools and their contracts.

Knowledge Bases supply external evidence.

Agent instructions, model selection, and prompt templates define orchestration configuration.

Version and test changes before production use.

A configuration change still requires the correct preparation and deployment lifecycle.

Sources [26, 7].

## AWS agent runtime architecture

Follow the orchestration loop from user input through tool execution and observations.

The runtime assembles context and evaluates the next step.

The model can request actions or knowledge retrieval.

The application must enforce permission and side-effect safety.

The loop ends with a result or a request for more input.

The figure explains the service flow. Add explicit task budgets, authorization, and evaluation for your application.

Sources [26, 7].

## Agent action loop

Trace each stage. Identify its input, output, access rule, and failure behavior.

Use task state and available tool schemas.

Validate identity, parameters, budget, and any required approval.

Call the tool with retry safety. Record the result.

Update state. Enforce success, step limit, and deadline.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Controlled transaction

Trace each stage. Identify its input, output, access rule, and failure behavior.

Prepare the exact operation from current authorized evidence.

Check business rules, actor scope, amount, and parameters.

Bind human approval to the exact action and expiry.

Use an idempotency key and confirm transaction status.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Deployment modes

Match deployment to traffic, model support, control requirements, and operational effort.

Use Bedrock on-demand inference for supported variable workloads.

Assess Provisioned Throughput for supported models and capacity requirements.

Use SageMaker endpoints when supported custom hosting and runtime control matter.

Keep invocation and deployment capacity identifiers consistent.

Lambda can invoke a model. It is not the host for a large GPU model.

Sources [38, 29, 7].

## LLM hosting mechanics

Large models can face memory and token throughput limits that differ from small predictive models.

Plan for model weights, runtime overhead, KV cache, and concurrent requests.

Assess quantization, model parallelism, and supported inference containers.

Measure cold start, model load time, and warm capacity.

Scale against token work and latency, not request count alone.

A GPU with enough memory for weights can still lack memory for the target concurrency.

Sources [29, 7].

## Small models and cascades

A cascade uses a lower-cost path for suitable tasks and escalates when the task requires more capability.

Route with validated task classes, rules, or calibrated evaluation signals.

Use a smaller model only when it meets the required quality floor.

Include routing and escalation costs in the comparison.

Track quality by task segment after deployment.

A model saying I am confident is not a calibrated probability of correctness.

Sources [28, 40, 7].



## Enterprise events and APIs

Integration style follows the response contract and failure behavior.

Use synchronous APIs when callers need an immediate result.

Use SQS for buffered work and backpressure.

Use EventBridge for event routing across decoupled systems.

Use Step Functions for explicit multi-step state, retry, and recovery.

An event bus routes events. It does not replace every queue or durable workflow.

Sources [40, 7].

## Federation and secure enterprise access

User identity and service identity serve different purposes.

Use federation or Cognito for supported user authentication.

Use execution roles and temporary credentials for AWS service calls.

Propagate trusted tenant and role context through the gateway.

Enforce object-level authorization at data and tool boundaries.

An authenticated user can still lack permission for a particular document or action.

Sources [5, 8, 7].

## Hybrid, edge, and residency

Data location and model execution location are separate architectural facts.

Use approved connectivity between cloud and on-premises systems.

Assess Outposts for supported on-premises AWS workloads.

Assess Wavelength for supported edge latency requirements.

Verify service and model availability before an edge or hybrid commitment.

Outposts and Wavelength do not imply that every Bedrock foundation model runs locally.

Sources [8, 40, 7].

## GenAI gateway and CI/CD

A gateway centralizes policy while CI/CD controls release quality.

Apply request validation, identity checks, budgets, and model routing at the gateway.

Trace requests across retrieval, generation, and tool calls.

Use CodeBuild and CodePipeline for tests, scans, and release gates.

Version the complete application configuration and preserve rollback targets.

Central routing without per-user data authorization can concentrate a security defect.

Sources [28, 40, 7].

## Governed model access

Trace each stage. Identify its input, output, access rule, and failure behavior.

Authenticate and derive trusted access context.

Validate requests and enforce budgets and policy.

Apply compatible schema, prompt version, and model route.

Correlate model use, evidence, quality, and cost.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Synchronous, asynchronous, and batch calls

These patterns differ in caller behavior and service support.

Synchronous inference keeps the caller attached to one result.

Application-level async uses a queue, worker, and status record.

Service async operations exist only for supported models and tasks.

Batch inference processes supported bulk jobs without an interactive response.

An application queue does not turn every model into a service-native asynchronous model.

Sources [14, 2, 7].

## Streaming from model to client

Streaming can improve time to first token while the full response still takes time.

Use ConverseStream or InvokeModelWithResponseStream where supported.

Preserve streaming through every intermediary and client.

Distinguish partial text from a completed, validated response.

Plan moderation, interruption, cancellation, and error behavior.

Streaming does not automatically reduce total token work or prevent unsafe partial content.

Sources [31, 32, 7].

## Retries, backpressure, and circuit breakers

Retry only when the failure class and remaining request budget permit it.

Use exponential backoff with jitter for appropriate transient errors.

Bound retry count and total elapsed time.

Queue excess work and cap concurrent invocations.

Use a circuit breaker or tested fallback after repeated service failures.



Immediate retries amplify throttling. Retries cannot fix a malformed request or missing permission.

Sources [4, 9, 7].

## Routing policy and fallback quality

Routing needs both an availability policy and a capability policy.

Use static routes for known task classes.

Use dynamic routing when measured signals justify its overhead.

Preserve residency and authorization constraints on every route.

Re-evaluate prompts, output contracts, and tools for each fallback model.

A fallback that returns an invalid business result is not a successful recovery.

Sources [51, 9, 7].

## Application interfaces and workflow tools

A useful AI interface exposes task progress, uncertainty, and the next permitted action.

Use Amplify for suitable application interface development.

Use OpenAPI contracts for stable client integration.

Use Flows for supported visual workflow authoring.

Add GenAI to CRM or document systems through controlled service interfaces.

A polished chat interface cannot compensate for an incorrect evidence or authorization path.

Sources [13, 41, 7].

## Developer tools and diagnosis

Amazon Q Developer can assist development. The engineer remains responsible for correctness and security.

Use assistance for code, refactoring, API examples, and error analysis.

Validate generated code against current API contracts.

Use Logs Insights for correlated application evidence.

Use X-Ray or compatible traces to locate latency and failure boundaries.

Generated code can use old SDKs or invent fields. Verify execution and permissions.

Sources [4, 40, 7].

## Input safety and prompt attacks

Treat user content, documents, tool responses, and external pages as potentially adversarial.

Use supported Guardrails content and prompt-attack controls.

Separate trusted instructions from untrusted evidence.

Sanitize and validate data at ingestion and tool boundaries.

Test direct injection, indirect injection, encoding tricks, and data exfiltration attempts.

Keyword filters alone miss indirect and transformed attacks.

Sources [34, 46].

## Guardrails policy selection

Match the policy to the exact harm or business restriction.

Use content filters for supported harmful content categories.

Use denied topics for defined prohibited subjects.

Use word filters for specific words or phrases.

Use sensitive information filters for supported PII and regex patterns.

A denied topic and a word filter are not interchangeable. A business topic can use many different words.

Sources [34, 46].

## Guardrails placement and output safety

Protect the input, retrieved context, tool path, and final output with suitable controls.

Apply Guardrails through supported inference integrations or ApplyGuardrail.

Validate output structure and business rules after generation.

Filter or mask sensitive outputs where policy requires it.

Design buffering and release rules for streaming output.

One content check does not enforce retrieval permissions, tool authorization, or SQL safety.

Sources [34, 35, 46].

## Grounding, relevance, and hallucination

Grounding asks whether the answer follows supplied evidence. Relevance asks whether it addresses the question.

Use current authorized sources and preserve citations.

Evaluate support for each factual claim.

Use supported contextual grounding checks when appropriate.

Return uncertainty or insufficient evidence when support is absent.

A citation can be real while its passage does not support the claim. JSON Schema cannot establish factual truth.

Sources [36, 28, 46].



## Automated Reasoning checks

Automated Reasoning checks assess responses against a formal policy within supported use cases.

Translate policy material into a validated formal representation.

Check the response against policy rules and stated facts.

Inspect findings and route unsupported or ambiguous cases appropriately.

Maintain policy versions and regression tests after rule changes.

Formal policy consistency is not a universal guarantee of truth outside the policy model.

Sources [54, 46].

## Private network and IAM controls

PrivateLink controls network access paths. IAM controls permitted actions and resources.

Create the appropriate interface endpoints for runtime and other required APIs.

Configure endpoint security groups, private DNS, and endpoint policy.

Use scoped execution roles and temporary credentials.

Verify data service, KMS, and tool permissions separately.

A VPC endpoint does not grant IAM permission or prove that the application never uses another public path.

Sources [8, 5, 46].

## Private inference path

Trace each stage. Identify its input, output, access rule, and failure behavior.

Use a role and private subnet network controls.

Configure security groups, private DNS, and endpoint policy.

Authorize the exact model or approved inference profile.

Use separate access and encryption controls for sources and logs.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## Encryption and secrets

Protect data in transit, stored content, indexes, logs, backups, and secrets.

Use TLS for service traffic.

Use supported KMS encryption controls for stored data.

Restrict key use and inspect both IAM and key policy.

Store credentials in Secrets Manager and use temporary role credentials when possible.

Encryption does not make an unauthorized read acceptable. KMS access can independently block an otherwise permitted workflow.

Sources [6, 5, 46].

## PII discovery, redaction, and retention

Choose a control based on where the sensitive data exists and when it must be protected.

Use Macie to discover sensitive data in supported S3 analysis workflows.

Use Comprehend for supported PII detection in text.

Use Guardrails sensitive information controls in supported model paths.

Apply retention and deletion rules to sources, vectors, conversation state, and logs.

Masking can leave re-identification risk. Deleting an S3 object does not remove every derivative automatically.

Sources [6, 11, 34, 46].

## Bedrock privacy boundaries

Distinguish model provider access from application storage and service-specific retention.

AWS states that model providers lack access to Bedrock deployment accounts and customer prompts there.

Review current data retention and abuse detection terms for the selected service and model.

Treat enabled invocation logs as sensitive application data.

Apply your own access, deletion, and encryption controls.

Do not generalize a claim about base inference to every optional feature, logging setting, or model offering.

Sources [6, 11, 30, 46].

## Governance, model cards, and lineage

Governance links decisions to data, model versions, policies, and accountable owners.

Document intended use, limits, evaluation evidence, and approval status in model cards.

Register source datasets and metadata with suitable catalog controls.

Preserve document and prompt versions in decision records.

Retain audit evidence under defined access and retention rules.

A catalog entry does not automatically prove complete data lineage or regulatory compliance.

Sources [40, 30, 46].



## Policy ownership and continuous controls

An organization needs policy owners, release rules, and controls that continue after deployment.

Define permitted use cases, data classes, escalation, and exception owners.

Monitor misuse, policy violations, quality drift, and cost anomalies.

Redact sensitive telemetry and restrict audit access.

Connect alerts to tested remediation and review workflows.

A one-time launch review cannot detect future data, model, or usage changes.

Sources [40, 28, 46].

## Transparency and uncertainty

A useful explanation shows evidence, limits, and the basis for a decision.

Show source passages and relevant task constraints.

Record trace events and tool outcomes for diagnosis.

Report measured uncertainty only when the measure has a valid interpretation.

Give users a review path for consequential errors.

Do not equate hidden chain-of-thought, a fluent explanation, or a model confidence claim with proof.

Sources [28, 40, 46].

## Fairness and responsible AI evaluation

Fairness tests compare relevant user groups and scenarios under a defined task policy.

Use representative and adversarial evaluation sets.

Compare outcome quality, refusal errors, and safety by relevant segment.

Use human review for context that automatic metrics miss.

Repeat tests after model, prompt, source, or policy changes.

An LLM judge can reproduce bias. An overall average can hide a poor subgroup result.

Sources [28, 40, 46].

## Token cost and context efficiency

Cost depends on the selected model, pricing mode, tokens, and optional services.

Estimate input and output tokens separately.

Remove duplicate evidence, obsolete history, and unnecessary output.

Reserve enough output budget for the task.

Compare cost per successful task with its quality and latency.

A shorter prompt can omit decisive evidence. Cost reduction must preserve the acceptance criteria.

Sources [37, 40, 48].

## A worked cost comparison

The following values are illustrative assumptions from the repository, not a current AWS price quote.

Input price: $3 per million tokens. Output price: $15 per million tokens.

Input: 4,000 tokens. Output: 1,000 tokens.

Input cost: $0.012. Output cost: $0.015. Total: $0.027.

Compare alternatives at the same successful-task volume and quality level.

The example excludes storage, retrieval, logs, networking, tools, retries, and taxes.

Sources [55, 48].

## Prompt cache and result cache

Prompt caching reuses eligible model context. Result caching reuses an application answer.

Use prompt caching for supported repeated prefixes and model-specific rules.

Use exact result caching when request identity and freshness permit it.

Use semantic result caching only with tested equivalence and access controls.

Include model, prompt, source, tenant, and policy versions in cache decisions.

A cache hit can return an outdated or unauthorized answer if the key omits state or identity.

Sources [37, 48].

## Prompt cache limits and measurements

Cache support, token thresholds, TTL, and price differ by model and API.

Place stable content before changing content when the cache design requires a shared prefix.

Confirm supported cache checkpoints and minimum token thresholds.

Read cache write and cache read usage fields.

Compare actual hit rates and effective cost on representative traffic.

Do not memorize a universal five-minute TTL. Current documentation lists model-specific cache behavior.

Sources [37, 48].

## Capacity and throughput

Capacity planning must account for prompt length, output length, concurrency, and quota.



Measure token demand and latency by task class.

Use supported batching or bulk inference for suitable work.

Compare on-demand and provisioned modes for measured traffic.

Scale self-hosted endpoints from relevant utilization and queue signals.

Provisioned capacity does not guarantee every response latency. Model support and commitment terms vary.

Sources [38, 14, 48].

## Latency and critical path

Measure each stage before a latency change.

Separate authentication, retrieval, reranking, generation, and tool time.

Track time to first token and full response time.

Parallelize independent work when resource and policy limits permit it.

Use supported latency-optimized modes only after quality and cost tests.

Streaming improves perceived response time but does not remove a slow retrieval stage.

Sources [40, 16, 48].

## Retrieval optimization

Optimize search with relevance and latency metrics together.

Inspect candidate recall before the generation stage.

Tune index layout, metadata filters, and query preprocessing.

Test hybrid scoring and reranker candidate counts.

Monitor stale documents, failed embeddings, and source synchronization.

A smaller top-k can reduce cost while hiding required evidence.

Sources [16, 28, 48].

## Observability by request and task

Telemetry must explain technical health and business outcomes.

Propagate a correlation ID across gateway, retrieval, model, and tools.

Capture model and prompt versions, token use, stop reasons, and latency.

Track task completion, user correction, quality scores, and safety events.

Restrict telemetry access and remove unnecessary sensitive content.

CloudWatch does not infer hallucination rate from raw service metrics. Supply evaluated custom metrics.

Sources [30, 28, 48].

## CloudTrail and invocation logs

Audit events and model request bodies have different purposes and controls.

Use CloudTrail for supported API audit events and identity evidence.

Configure required data event coverage explicitly.

Enable supported model invocation logging for detailed request and response analysis.

Use CloudWatch Logs or S3 destinations with encryption, access limits, and retention.

CloudTrail is not a universal store for prompt and completion bodies. Invocation logging needs explicit configuration.

Sources [39, 30, 48].

## Tools, agents, and vector telemetry

An agent can fail in a tool or retrieval stage while model inference succeeds.

Track tool name, arguments after redaction, latency, retry count, and result status.

Measure agent steps, repeated calls, completion rate, and cost.

Monitor vector query latency, ingestion lag, and retrieval quality.

Alert on deviations from task-specific baselines.

A successful HTTP response can still contain a failed business operation.

Sources [44, 30, 48].

## GenAI failure diagnosis

Use a reproducible case and change one relevant layer at a time.

Replay a fixed golden dataset with versioned configuration.

Compare evidence, prompts, tool traces, and outputs.

Separate retrieval failure from unsupported generation.

Route failures to the correct owner with source evidence.

Exact output diffs can overcount harmless wording changes. Use semantic and task-aware checks.

Sources [28, 30, 48].

## Evaluation dimensions

A useful evaluation set measures the task rather than only text similarity.

Measure correctness, relevance, completeness, consistency, and fluency as appropriate.

Add safety, privacy, latency, and cost thresholds.

Include expected outputs or evidence with clear rubrics.

Report results by important task and user segments.

A fluent answer can be wrong. A lexically different answer can be correct.

Sources [28, 49].



## Programmatic, judge, and human evaluation

These methods provide different evidence and can complement each other.

Use deterministic checks for schema, required values, and business rules.

Use model-based judges for rubric-based assessment at scale.

Use domain experts for difficult or consequential judgments.

Use Bedrock evaluation jobs where the required models, data, and metrics are supported.

A judge model is a measurement tool with bias and uncertainty. It is not ground truth.

Sources [28, 49].

## Judge design and calibration

A judge needs a clear rubric and calibration against human labels.

Define the criterion and score anchors before the test.

Hide model identity when it could bias the judgment.

Test position bias and reverse pair order for pairwise comparisons.

Measure agreement with expert labels and review disagreements.

A judge may favor verbose answers or its own model family. Do not treat one score as objective truth.

Sources [28, 49].

## RAG retrieval metrics

Retrieval tests ask whether the necessary evidence appears in the candidate set.

Recall@k measures the share of relevant items found within k results.

Precision@k measures the share of returned items that are relevant.

MRR emphasizes the rank of the first relevant result.

Evaluate ranking metrics and retrieval latency against query-level ground truth.

Higher recall alone can add noisy context. Score generation quality as well.

Sources [28, 16, 49].

## RAG answer metrics

Score the retrieval stage and the answer stage separately.

Check context relevance for the retrieved evidence.

Check faithfulness for support in the supplied context.

Check answer correctness against authoritative ground truth.

Check completeness and citation support for the user question.

An answer can be faithful to an outdated source and still be factually wrong.

Sources [28, 21, 49].

## Feedback and annotation

User feedback provides signals that require context and review.

Capture the task, output version, and a useful error category.

Let reviewers label unsupported facts, missing evidence, and unsafe behavior.

Remove sensitive content that the evaluation process does not need.

Feed reviewed failures into the regression set.

A thumbs-up signal alone cannot establish factual correctness.

Sources [28, 49].

## Regression gates and model comparisons

A release comparison must hold the task and evaluation method steady.

Use a held-out golden dataset with versioned references.

Compare candidate and baseline on quality, safety, latency, and cost.

Define release and rollback thresholds before the experiment.

Run the gate after prompt, model, data, retrieval, or tool changes.

Do not tune repeatedly on the only test set. Preserve a held-out measure of generalization.

Sources [28, 40, 49].

## Release quality gate

Trace each stage. Identify its input, output, access rule, and failure behavior.

Freeze model, prompt, retrieval, data, and tool settings.

Run task, safety, latency, and cost evaluations.

Compare required thresholds and critical failures.

Use canary or shadow validation and a rollback target.

This is a conceptual architecture. Service support and deployment details still require validation.

Sources [40].

## A/B, canary, and shadow tests

These deployment patterns answer different questions.

A/B tests compare alternatives on controlled traffic groups.

Canary releases expose a limited production share before expansion.

Shadow tests send copied inputs to a candidate without its normal production effect.

Synthetic tests probe critical user journeys with known expectations.

Shadow tools must not duplicate real side effects. A canary still needs rollback rules.

Sources [28, 40, 49].



## Agent evaluation

Evaluate the complete task path, including tools, state, and side effects.

Measure task completion and correct tool selection.

Validate arguments, action order, and authorization behavior.

Measure step count, tool failures, recovery, latency, and cost.

Test long tasks, interruptions, repeated requests, and malicious tool results.

A good final answer can hide an unsafe intermediate action.

Sources [28, 44, 49].

## Reports that support a decision

A report should show the release decision and the evidence behind it.

Present sample count, dataset version, rubric, and important segments.

Compare baseline and candidate with quality, latency, safety, and cost.

Show known failures and uncertainty instead of only averages.

State the owner, decision, and rollback or follow-up action.

A precise percentage from a tiny or biased sample can mislead.

Sources [28, 49].

## Context overflow and truncation

Distinguish an oversized request from an output that stops at its limit.

Count system, messages, tools, retrieved data, and output reserve.

Inspect stopReason and any validation error.

Prune or summarize tested history and reduce noisy evidence.

Use segmentation or a suitable model when the complete task exceeds the limit.

Raising maxTokens can worsen the total budget. Character count is not a reliable token count.

Sources [3, 4, 49].

## API failure classification

The error class determines the next step.

AccessDenied: inspect identity, policy, model access, Region, and profile permissions.

ValidationException: inspect schema, unsupported parameters, and size limits.

Throttling: inspect quotas, token load, concurrency, and retry behavior.

Timeout or service failure: inspect latency, retry safety, and fallback policy.

Do not retry permission failures or invalid requests without a relevant change.

Sources [4, 19, 49].

## Prompt defect diagnosis

Prompt diagnosis needs versioned inputs, outputs, and expected behavior.

Check unresolved template variables and conflicting instructions.

Inspect message order, examples, system support, and source delimiters.

Validate output format and stop conditions.

Change one prompt feature and run the regression set.

Adding more instructions can create conflicts and consume the evidence budget.

Sources [25, 3, 49].

## Retrieval defect diagnosis

Trace the evidence path before a generation change.

Verify source presence, latest version, and ingestion success.

Verify embedding model, dimensions, and document-query compatibility.

Inspect chunk boundaries, metadata, filters, and ranked candidates.

Check reranking, final context assembly, and cached answers.

A model upgrade cannot reliably fix missing or unauthorized evidence.

Sources [16, 10, 49].

## End-to-end secure RAG case

A policy assistant needs current evidence, tenant isolation, citations, and an auditable response path.

Authenticate the user and derive the permitted data scope.

Retrieve current authorized passages and rerank when justified.

Construct a bounded prompt and apply the required safety controls.

Validate the answer, return supporting citations, and record protected telemetry.

Citations, encryption, and Guardrails do not replace tenant authorization.

Sources [20, 34, 8, 12].

## End-to-end action agent case

A refund agent must inspect policy and order state before it proposes a controlled transaction.

Retrieve the current refund policy and authorized order state.

Validate amount, customer scope, and policy conditions.

Obtain required approval for the exact refund.

Execute once with an idempotency key and verify operation status.

A fluent explanation of the refund does not authorize the transaction.

Sources [47, 40, 7].



## Prompt technique comparison

Select the technique from the defect you need to correct.

Zero-shot supplies a task without task examples.

Few-shot supplies representative examples of input and desired output.

Chain-of-thought patterns request intermediate reasoning or decomposition.

ReAct alternates reasoning, tool action, and observations.

A reasoning explanation is not evidence of truth. Use verified sources, tool results, and answer checks.

Sources [25, 3, 12].

## Prompt injection and tool output

Indirect injection reaches an agent through a document or a tool result.

Treat retrieved instructions as data, even when the source appears trusted.

Restrict tool destinations, data scope, and permitted side effects.

Avoid unnecessary secrets in model context.

Test attacks that ask the model to alter filters or expose prior messages.

A tool result can be accurate data and still contain an injected instruction.

Sources [34, 40, 46].

## Inference permission matrix

API method names and IAM action names are not always identical.

Converse requires the relevant bedrock:InvokeModel permission.

ConverseStream requires the relevant bedrock:InvokeModelWithResponseStream permission.

InvokeModel uses bedrock:InvokeModel for supported resources.

Retrieval and agent APIs use their own relevant actions and resource scope.

Do not invent bedrock:Converse as a substitute IAM action for the Converse runtime API.

Sources [3, 31, 5, 7].

## Converse with a prompt version

A Prompt Management resource can define parts of the model request.

A modelId can reference a supported prompt version ARN.

Use promptVariables for values that the prompt template requires.

Respect request field restrictions when a prompt resource defines configuration.

Keep prompt version and variable contract in the release record.

Do not assume that system, toolConfig, inferenceConfig, and additionalModelRequestFields can override a prompt resource request.

Sources [3, 25, 12].

## Structured output and business validity

A supported structured output feature constrains shape. The application still checks meaning.

Confirm the chosen model and API support the required JSON Schema feature.

Use supported outputConfig or tool schema settings as appropriate.

Validate required fields, types, bounds, and allowed values.

Check factual support, permissions, and business rules separately.

Do not claim that every Converse model supports strict structured output. A valid object can still contain fabricated values.

Sources [43, 3, 12].

## Bedrock agent lifecycle

Separate the editable agent configuration from the version that production invokes.

Define instructions, action groups, knowledge sources, and execution permissions.

Prepare and test the draft agent through the supported lifecycle.

Use versions and aliases for controlled production routing.

Review tool and source changes with task and safety tests.

A changed draft does not imply that every existing production alias now uses that configuration.

Sources [26, 47, 7].

## Bedrock action groups and return control

An action group connects a model decision to a defined external capability.

Describe operations through a supported API schema or function definition.

Use Lambda execution where the integration requires it.

Use return control when the application must handle the proposed action.

Keep approvals and idempotency at the actual side-effect boundary.

Return control gives application control over execution. It is not automatic human approval.

Sources [26, 47, 7].

## Knowledge Bases metadata and sync

A managed ingestion path still needs correct metadata and observable update behavior.

Supply supported metadata in the data source format.

Preserve document identifiers, versions, effective dates, and access scope.



Run or trigger ingestion after source changes under the supported connector behavior.

Inspect ingestion statistics and deletion handling before answers use the update.

A scheduled sync cannot promise instant freshness. Define the acceptable source-to-index delay.

Sources [10, 16, 12].

## Search and assistant service boundaries

A search service, an assistant, and a custom RAG application have different control surfaces.

Kendra supplies enterprise search with supported connectors and access features.

Q Business supplies a managed enterprise assistant over supported business data.

Bedrock Knowledge Bases supplies retrieval and generation integration for custom applications.

A custom vector path gives control over indexing, retrieval, and application assembly.

Choose from required customization, connectors, permissions, and operational effort. Do not select from product names alone.

Sources [20, 40, 12].

## Graph retrieval and relational retrieval

Some questions depend on relationships or exact structured values instead of semantic proximity alone.

Graph retrieval can follow entity relationships and multi-hop context.

Relational queries can supply exact values from a system of record.

Vector retrieval can supply semantic document context.

Combine results only after access checks and explicit source handling.

Do not generate and execute unrestricted SQL or graph queries under a broad service role.

Sources [20, 40, 12].

## API Gateway response streaming

Current AWS documentation supports response streaming for compatible REST API proxy integrations.

Set the supported response transfer mode to STREAM where the design requires it.

Check HTTP_PROXY or AWS_PROXY integration requirements.

Validate endpoint type, backend behavior, timeout, and client protocol.

Measure time to first byte through the complete route.

Do not reuse an old rule that all API Gateway responses buffer. Do not assume every HTTP or WebSocket integration supports the same behavior.

Sources [32, 31, 7].

## SageMaker inference pattern comparison

Choose an endpoint or batch pattern from latency, payload, runtime, and hardware requirements.

Real-time endpoints fit interactive requests with suitable provisioned resources.

Asynchronous inference fits supported queued requests and delayed results.

Batch Transform fits offline bulk processing.

Serverless inference fits supported intermittent workloads and has distinct resource limits.

Serverless inference does not imply GPU support for every large model. Verify runtime and model compatibility.

Sources [29, 7].

## Provisioned capacity and overflow

A mixed capacity design requires explicit routing and capacity assumptions.

Confirm that the chosen model supports the required provisioned offering.

Invoke the correct provisioned resource identifier.

Measure baseline token work and the relevant commitment terms.

Implement and test any separate on-demand overflow path.

Do not assume automatic spillover from provisioned capacity to a base on-demand model.

Sources [38, 7].

## Batch input, output, and failure handling

Batch inference is a job workflow with model-specific support and data contracts.

Prepare the required supported records and unique identifiers.

Place inputs and outputs in approved S3 locations.

Grant the service role the required data and KMS access.

Track job state and inspect failed records before a selective retry.

Batch inference is not a universal replacement for interactive or service-native async inference.

Sources [14, 7].

## Cost and quota are separate

A lower bill does not prove that a request avoids every quota.

Inspect the documented quota dimensions for the selected model and mode.



Measure request rate, input token rate, output token work, and concurrency as applicable.

Account for retries and parallel model calls.

Use queue limits and admission control before overload spreads.

A cache hit can affect quota accounting differently by model. Use current model-specific documentation.

Sources [37, 4, 48].

## Retrieval metrics with a small example

Metric definitions separate evidence coverage from evidence noise.

Recall@k is relevant returned passages divided by all relevant passages.

Precision@k is relevant returned passages divided by returned passages.

Reciprocal rank is one divided by the rank of the first relevant result.

A worked numerical example is not in source.

A metric depends on the relevance labels and evaluation unit. Record those definitions.

Sources [28, 16, 49].

## Bedrock RAG metric names

Know what each built-in measure checks before you select it.

Correctness assesses answer accuracy. Completeness assesses coverage of the question.

Faithfulness assesses support relative to retrieved text.

Citation precision assesses whether cited passages support their use.

Citation coverage assesses whether citations cover the response evidence needs.

Some metrics require ground-truth data. Check the dataset contract and supported job type.

Sources [21, 49].

## Automated Reasoning runtime findings

Automated Reasoning checks return findings. The application decides the next action.

VALID concerns only the policy scope captured by its variables.

INVALID indicates a detected conflict with the modeled policy.

Ambiguous or complex cases can require clarification or review.

Configure the supported content qualifiers and verify actual policy evaluation.

Detect mode does not automatically block the response. Missing qualifiers can cause no evaluation in some invocation paths.

Sources [45, 54, 46].

## Data poisoning and evidence integrity

A retrieved passage can be incorrect because the source or ingestion path was compromised.

Control who can write and approve trusted source content.

Track version, origin, and access metadata through ingestion.

Detect unexpected source changes and quality regressions.

Quarantine suspect content and invalidate affected derived data.

A grounded response can faithfully repeat poisoned evidence. A grounding score alone cannot establish source truth.

Sources [40, 20, 46].

## Exam-day time allocation

Use a time plan that leaves room for questions with uncertain constraints.

AWS lists 75 questions in 180 minutes.

The simple average is 2.4 minutes per question.

Answer all questions because unanswered items count as incorrect.

For multiple response, select every required correct option.

A time average is a planning aid. It is not a required pace for each question or a scoring formula.

Sources [1, 50, 49].

## Choose an architecture from workload limits

Choose the smallest architecture that meets the stated user outcome and hard limits.

Record users, task, data class, source of truth, traffic shape, latency target, quality floor, and failure action.

Separate required limits from preferences.

Compare prompt-only, RAG, tool use, customization, and a managed workflow against the same representative test set.

A proof of concept must test the riskiest assumption, not only show a successful demo.

Record cost per successful task and define a stop rule before scale-up.

A larger model or more complex architecture does not prove better fitness.

Sources [12, 1].

## Choose retrieval, tools, or customization

Choose a method by the source and behavior that must change.

Use retrieval when the answer needs current or private source facts.

Use a tool when the application must read or change a live system.



Use fine-tuning when representative examples must teach a stable task behavior or style.

Use prompt instructions for clear task rules and output shape.

A method can combine with another method.

Define how the team will measure each method before implementation.

Fine-tuning does not make changing facts current. Retrieval does not authorize a transaction.

Sources [12, 20].

## Validate multimodal inputs at each boundary

Validate the file, extracted content, and model request as separate stages.

Check file type, size, ownership, malware controls, and permitted data class before processing.

Select extraction by content: Textract for supported document text or forms, Transcribe for audio, and a supported multimodal model for image understanding.

Check extraction output for missing pages, encoding faults, and confidence limits.

Normalize the result, preserve source identifiers, then validate the model-specific request schema.

Successful extraction does not prove that extracted text is complete or correct.

Sources [12, 42].

## Plan vector index changes safely

Treat embedding model and vector dimensions as part of the index contract.

Store the embedding model, dimension, normalization rule, and source version with the index configuration.

Keep document and query vectors compatible.

Measure recall and latency before choosing index parameters or shard layout.

For a model change, build a separate index, embed a representative corpus, compare retrieval quality, and switch traffic only after validation.

Retain a rollback path until old references and caches expire.

Changing embedding dimensions in place can make vectors incompatible.

Sources [12, 16, 20].

## Enforce document deletion across derived data

A source deletion must reach every stored copy and derived result.

Define a deletion event with a stable source identifier and version.

Remove the document from the source, ingestion queue, vector store, lexical index, cached answers, and any retained summaries that contain its content.

Track completion by system and retry failed steps.

Apply the approved retention policy to logs and backups.

Removing an S3 object alone may leave indexed text and cached answers.

Sources [12, 10, 11].

## Build authorization into retrieval

Derive retrieval scope from trusted identity on the server.

Authenticate the caller before retrieval.

Map the identity to approved tenant, role, and document attributes.

Apply filters in the retrieval request before evidence enters the prompt.

Deny access when identity or scope is missing.

Test cross-tenant and stale-permission cases.

Do not accept a client-supplied tenant filter as proof of authorization.

A metadata filter is an access control only when trusted server logic sets it.

Sources [12, 5, 20].

## Select retrieval metrics by failure type

Use retrieval metrics to find where evidence quality fails.

Recall at k asks whether relevant evidence appears in the first k results.

Precision at k asks how much of that result set is relevant.

Mean reciprocal rank rewards an early first relevant result.

Compare metrics on labeled queries by task and document class.

Inspect failed examples because aggregate scores can hide a weak segment.

A strong answer score cannot prove retrieval found the right evidence.

Sources [49, 21, 12].

## Keep prompt injection outside the trust boundary

Treat user text, retrieved text, and tool output as untrusted data.

Keep trusted instructions separate from user content and retrieved passages.

Delimit evidence and tell the model that evidence cannot change policy or tool permissions.

Enforce authorization and allowed destinations in application code.

Validate tool arguments before execution.

Test indirect attacks in documents and direct attacks in user prompts.

A prompt delimiter can help parsing but cannot enforce security by itself.

Sources [46, 34, 35, 12].

## Choose the right safety control

Match each safety control to its failure mode.



Content filters address supported harmful content categories.

Denied topics address defined subjects.

Word filters detect configured terms.

Sensitive information filters detect or mask supported data patterns.

Grounding checks assess response support against supplied context.

Application validation enforces business rules.

Combine controls where one control cannot cover the full risk.

A content filter does not verify that a factual claim is true.

Sources [46, 36, 34].

## Apply least privilege to private inference

Private network paths and IAM permissions solve different access risks.

Use interface endpoints for supported private service access.

Restrict endpoint security groups and endpoint policies.

Grant the execution role only required model and resource actions.

Check model resource and inference profile permissions where applicable.

Use KMS key policies and IAM policies together for encrypted data.

Test access from the actual workload role and network.

A VPC endpoint does not grant permission to invoke a model.

Sources [46, 5, 8].

## Separate audit events from prompt logs

Use audit records and invocation logs for different questions.

CloudTrail records supported API activity and identity context.

Model invocation logging can capture request and response detail when configured and supported.

Set data event coverage where required.

Restrict log access, redact sensitive values, and set retention.

Verify that the selected model and API support the needed logging path before relying on it.

CloudTrail alone does not provide full prompt and response content for every invocation.

Sources [48, 30, 39, 46].

## Separate token cost from request quotas

Cost, request rate, token rate, and concurrency are separate capacity limits.

Estimate input and output token work by task class.

Apply the current model price and mode to estimate cost.

Check the selected model quota dimensions, such as request rate or token throughput, in the required Region.

Measure burst traffic and retries.

Size queue depth and concurrency to avoid turning throttling into unbounded work.

A budget estimate does not prove that a workload fits service quotas.

Sources [48, 38, 14].

## Use prompt cache only for repeatable prefixes

Prompt caching helps only when supported content repeats under service rules.

Identify stable prefix content that repeats across requests.

Check supported model, API, cache checkpoints, minimum token requirements, and cache lifetime.

Keep changing user data after the shared prefix when the API requires a matching prefix.

Measure cache read and write tokens and total latency.

Do not use a cached answer when freshness or authorization can change.

Prompt caching and semantic answer caching have different safety and freshness risks.

Sources [48, 37].

## Design timeout budgets across the request path

Set each timeout inside one end-to-end request budget.

Allocate time for authentication, retrieval, reranking, model response, and tools.

Set each component timeout below the caller deadline.

Bound retries by both attempt count and total time.

Use backoff with jitter for retryable transient errors.

Return a clear partial or timeout state when the deadline expires.

Retries can worsen latency and throttling when they exceed the caller budget.

Sources [7, 4, 32].

## Preserve streaming through every layer

A streaming model API helps only when every layer forwards the stream.

Select a supported streaming operation for the chosen model.

Configure the gateway and integration for streaming.

Forward chunks without buffering in the backend and client.

Distinguish partial text from a completed response.

Handle disconnects, stop reasons, and errors after the first chunk.

Test time to first token and total response time.

Streaming does not guarantee lower total generation time.



Sources [7, 33, 32].

## Evaluate judge reliability before scaling

A model judge needs a tested rubric and human calibration.

Write criteria with score anchors and examples.

Compare judge scores with expert labels on representative cases.

Test position bias by reversing pair order.

Hide model identity where it can bias judgment.

Track disagreement and judge version.

Use deterministic checks for schemas and exact business rules.

A fluent judge explanation does not prove valid scoring.

Sources [49, 28, 21].

## Choose a deployment mode by traffic shape

Match deployment mode to request timing, volume, and capacity needs.

Use on-demand inference for supported variable traffic when shared capacity fits.

Assess Provisioned Throughput for supported models when predictable capacity and sustained demand justify it.

Use batch inference for suitable offline workloads.

Use SageMaker endpoints when the model or hosting control requires that path.

Compare cold starts, capacity, quotas, and cost with measured traffic.

A deployment mode cannot compensate for an unsupported model or Region.

Sources [7, 38, 14, 29].

## Separate agent identity from user approval

An approval step must authorize one exact action.

Show the user the action, target, parameters, evidence, and expiry before approval.

Bind approval to the authenticated actor and action payload.

Revalidate authorization and business rules at execution time.

Use idempotency for retries and record the outcome.

Stop if the approved parameters change.

A broad approval for an agent does not approve every future side effect.

Sources [7, 47, 5].

## Distinguish retrieval systems by source need

Choose a search or assistant service by its data and control needs.

Use a Knowledge Base when the application needs managed RAG over connected sources and supported retrieval controls.

Use OpenSearch when the application needs direct control of search indexes and query behavior.

Use Kendra or Q Business when their supported enterprise search or assistant functions fit the requirement.

Use a system of record for exact transactional values.

Verify current source connectors and access behavior.

A managed assistant or search index is not automatically the system of record.

Sources [12, 20, 42].

## Trace an answer to its evidence path

Trace retrieval and generation as separate stages of one request.

Propagate a request identifier through gateway, retrieval, reranking, model, and tool calls.

Record prompt and model versions, filters, document identifiers, latency, token use, stop reason, and result status.

Redact sensitive content.

Replay the same versioned test input.

Decide whether failure came from missing source data, filtering, ranking, prompt construction, or unsupported generation.

A correct final answer alone does not identify which stage failed.

Sources [48, 30, 20, 28].

## Final decision rules

Use these rules as recall prompts after the full course.

Fresh facts and citations require a controlled evidence path.

Tool actions require server-side permission, validation, and retry safety.

Optimization must preserve task quality and hard constraints.

Release decisions require versioned tests and a rollback path.

Treat repository rules such as always choose managed or always choose RAG as heuristics that require context.

Sources [1, 40].



# Architecture reference

## Evidence path

| | |
|---|---|
| 1. Question<br>Authenticate the caller and identify the task. | 2. Retrieve<br>Search current authorized chunks with metadata filters. |
| 3. Generate<br>Send bounded evidence and instructions to the model. | 4. Verify<br>Check source support, citations, schema, and safety. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Tenant boundary

| | |
|---|---|
| 1. Verified identity<br>Derive tenant and role on the server. | 2. Filtered retrieval<br>Apply source permissions before the model sees evidence. |
| 3. Bounded inference<br>Use approved model, prompt, and safety policy. | 4. Protected response<br>Validate evidence and restrict logs and caches. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Source to index

| | |
|---|---|
| 1. Source event<br>S3, connector, or approved enterprise change event. | 2. Extract and validate<br>Parse text and tables. Check data quality and permissions. |
| 3. Chunk and embed<br>Preserve source version and use a compatible embedding model. | 4. Index and verify<br>Upsert or delete vectors. Measure freshness and ingestion errors. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Governed model access

| | |
|---|---|
| 1. Client identity<br>Authenticate and derive trusted access context. | 2. Gateway<br>Validate requests and enforce budgets and policy. |
| 3. Model adapter<br>Apply compatible schema, prompt version, and model route. | 4. Telemetry<br>Correlate model use, evidence, quality, and cost. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Controlled failure path

| | |
|---|---|
| 1. Invoke<br>Apply deadline and concurrency limits. | 2. Classify failure<br>Distinguish invalid input, denied access, throttle, and transient fault. |
| 3. Retry or break<br>Use bounded jittered retries and a circuit breaker. | 4. Fallback or stop<br>Preserve quality, residency, permissions, and user-visible status. |

This path applies after a failure. A successful invocation returns normally.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Semantic retrieval

| | |
|---|---|
| 1. Document vectors<br>Embed each permitted chunk and preserve source metadata. | 2. Query vector<br>Use a compatible embedding configuration. |
| 3. Candidate search<br>Apply access filters and compare vectors or hybrid scores. | 4. Ranked evidence<br>Return passages with references. Score meaning remains model-specific. |

Document and query encoders are independent inputs to candidate search. They must share a compatible embedding contract.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Parent and child context

| | |
|---|---|
| 1. Document<br>Keep sections, tables, and source locations intact. | 2. Child chunks<br>Embed smaller passages for precise candidate matches. |
| 3. Parent context<br>Replace matching children with broader parents where configured. | 4. Prompt budget<br>Deduplicate context and reserve output tokens. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Two-stage retrieval

| | |
|---|---|
| 1. User query<br>Preserve intent, exact terms, and data access scope. | 2. Candidate set<br>Use vector or hybrid search to obtain relevant candidates. |
| 3. Reranker<br>Score the available candidates against the query. | 4. Final context<br>Select supported evidence within the prompt budget. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.



## Conversation state

| | |
|---|---|
| 1. Session lookup<br>Verify ownership and load required structured constraints. | 2. Context assembly<br>Select recent history, tested summaries, and evidence. |
| 3. Model call<br>Use approved instructions and a bounded token budget. | 4. State update<br>Validate the result and store only permitted state. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Fixed document workflow

| | |
|---|---|
| 1. Extract<br>Convert the supported document into fields and evidence. | 2. Validate<br>Check schema, required values, and source support. |
| 3. Branch<br>Route exceptions to review under explicit rules. | 4. Persist<br>Write authorized results and record the operation. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Agent action loop

| | |
|---|---|
| 1. Plan next step<br>Use task state and available tool schemas. | 2. Authorize tool<br>Validate identity, parameters, budget, and any required approval. |
| 3. Execute and observe<br>Call the tool with retry safety. Record the result. | 4. Continue or stop<br>Update state. Enforce success, step limit, and deadline. |

If more work remains, repeat from stage 1 within enforced budgets. Otherwise stop.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Specialist coordination

| | |
|---|---|
| 1. Supervisor<br>Split the task and enforce shared constraints. | 2. Specialists<br>Use narrow task contracts and scoped tool access. |
| 3. Evidence synthesis<br>Compare findings and resolve conflicting evidence. | 4. Final validation<br>Evaluate the combined result and total task cost. |

Specialists can run in parallel when their tasks are independent. The supervisor still validates the combined result.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Controlled transaction

| | |
|---|---|
| 1. Propose<br>Prepare the exact operation from current authorized evidence. | 2. Validate<br>Check business rules, actor scope, amount, and parameters. |
| 3. Approve<br>Bind human approval to the exact action and expiry. | 4. Execute once<br>Use an idempotency key and confirm transaction status. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Tool trust boundary

| | |
|---|---|
| 1. Model proposal<br>Treat arguments as untrusted input. | 2. Server validation<br>Validate schema, permissions, business rules, and destination. |
| 3. Operation<br>Execute with deadline and side-effect safeguards. | 4. Tool result<br>Return a typed result or classified failure. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## MCP tool access

| | |
|---|---|
| 1. Agent client<br>Discover tools through the supported protocol. | 2. MCP server<br>Expose explicit names, schemas, and controlled capabilities. |
| 3. Authorization<br>Resolve identity and restrict resource and operation scope. | 4. System of record<br>Call the approved API and return verified results. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## AgentCore responsibilities

| | |
|---|---|
| 1. Runtime<br>Execute the selected agent framework in a supported environment. | 2. Gateway and identity<br>Connect approved tools with suitable identity controls. |
| 3. Memory<br>Maintain permitted task and user state. | 4. Observability<br>Trace task behavior, tool operations, latency, and failures. |

These are component responsibilities. Select and connect the components that the application requires.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.



## Conditional model escalation

| | |
|---|---|
| 1. Task classification<br>Identify required capability and hard constraints. | 2. Validated small path<br>Use a smaller model for proven suitable tasks. |
| 3. Escalate only when required<br>Invoke an approved capable path when the validated small path cannot satisfy the task. | 4. Evaluation<br>Compare total task quality, latency, and cost. |

Escalation is conditional. A successful small-model path can return without a larger-model call.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Durable async request

| | |
|---|---|
| 1. API request<br>Validate the task and return an operation identifier. | 2. Queue and worker<br>Apply backpressure and perform supported inference work. |
| 3. Result store<br>Persist status and output with access controls. | 4. Client status<br>Poll or receive an approved notification of completion. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## End-to-end streaming

| | |
|---|---|
| 1. Model stream<br>Receive supported content blocks and completion events. | 2. Application<br>Track state, safety checks, cancellation, and errors. |
| 3. Transport<br>Use a compatible unbuffered integration and client protocol. | 4. Client<br>Show partial output and distinguish final validated content. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

## Defense in depth

| | |
|---|---|
| 1. Input and evidence<br>Check harmful content and indirect instruction attacks. | 2. Model controls<br>Use supported safety policies and bounded instructions. |
| 3. Tool boundary<br>Enforce permission and validate every side effect. | 4. Output controls<br>Check policy, sensitive data, structure, and evidence. |

Apply these controls at each relevant boundary. No single filter replaces authorization.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

### Private inference path

| | |
|---|---|
| 1. Private application<br>Use a role and private subnet network controls. | 2. Interface endpoint<br>Configure security groups, private DNS, and endpoint policy. |
| 3. Bedrock runtime<br>Authorize the exact model or approved inference profile. | 4. Data services<br>Use separate access and encryption controls for sources and logs. |

These are separate access boundaries. The application can call data services independently of Bedrock.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

### Sensitive data lifecycle

| | |
|---|---|
| 1. Discover<br>Classify source data and locate sensitive fields. | 2. Minimize<br>Redact or tokenize fields that the task does not require. |
| 3. Process<br>Use approved model access, permissions, and output policy. | 4. Retain or delete<br>Apply policy to sources, vectors, caches, histories, and logs. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

### Cache lookup and miss path

| | |
|---|---|
| 1. Request scope<br>Resolve identity, source version, model, prompt, and policy. | 2. Result cache lookup<br>Return a valid authorized hit. On a miss, continue to the model path. |
| 3. Prompt reuse<br>Use supported prefix caching on a model invocation. | 4. Cache maintenance<br>Invalidate eligible entries after data, permission, model, or policy changes. |

A valid result hit returns immediately. Prefix reuse applies to a model call. Invalidation is a separate event.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.



### Request critical path

| | |
|---|---|
| 1. Access<br>Measure authentication and gateway work. | 2. Evidence<br>Measure retrieval and rerank time. |
| 3. Inference<br>Measure first token, completion time, and output length. | 4. Tools and response<br>Measure downstream work and client delivery. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

### Correlated operations

| | |
|---|---|
| 1. Request context<br>Propagate correlation ID and version identifiers. | 2. Stage telemetry<br>Record retrieval, model, and tool performance. |
| 3. Quality signals<br>Evaluate sampled answers and task completion. | 4. Action<br>Alert the owner and execute tested remediation. |

Collect these signals across the request path. They are not a mandatory sequence of API calls.

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

### Release quality gate

| | |
|---|---|
| 1. Versioned candidate<br>Freeze model, prompt, retrieval, data, and tool settings. | 2. Held-out tests<br>Run task, safety, latency, and cost evaluations. |
| 3. Decision gate<br>Compare required thresholds and critical failures. | 4. Controlled rollout<br>Use canary or shadow validation and a rollback target. |

Numbers identify components or stages. Follow the stated relationship. Independent controls do not form a strict sequence.

# Service boundaries

## Analytics

Amazon Athena. SQL queries over S3 data and logs.

Amazon EMR. Managed big-data processing for large data preparation jobs.

AWS Glue. Data integration, catalog, transformation, and data quality rules.

Amazon Kinesis. Streaming event ingestion and processing.

Amazon OpenSearch Service. Lexical and vector search for retrieval workloads.

Amazon Quick Sight. Business dashboards and analysis.

## Analytics

Amazon Managed Streaming for Apache Kafka (Amazon MSK). Managed Kafka streams for enterprise events.

## Application Integration

Amazon AppFlow. Managed SaaS data transfer and integration.

AWS AppConfig. Validated application configuration and controlled rollout.

Amazon EventBridge. Event routing across decoupled systems.

Amazon SNS. Publish and subscribe notifications.

Amazon SQS. Buffered work queues with backpressure.

AWS Step Functions. Durable workflow state, branches, retries, and approvals.

## Compute

AWS App Runner. Managed deployment for supported web applications.

Amazon EC2. Virtual machines for custom compute and hosting control.

AWS Lambda. Event-driven functions for orchestration, validation, and tool code.

AWS Lambda@Edge. Supported CloudFront edge functions for request and response logic.

AWS Outposts. Supported AWS infrastructure and services on premises.

AWS Wavelength. Supported edge infrastructure for low-latency workloads.

## Containers

Amazon ECR. Container image registry.

Amazon ECS. Managed container orchestration.

Amazon EKS. Managed Kubernetes control plane.

AWS Fargate. Serverless container compute for supported ECS and EKS workloads.

## Customer Engagement

Amazon Connect. Cloud contact center with AI integration use cases.

## Database

Amazon Aurora. Managed relational database with supported PostgreSQL vector extensions.

Amazon DocumentDB. Managed document database with supported vector capabilities.

Amazon DynamoDB. Durable key-value state, conversation records, and metadata.



Amazon DynamoDB Streams. Change records for downstream update workflows.

Amazon ElastiCache. Managed in-memory cache for permitted application data.

Amazon Neptune. Graph data and graph-based retrieval use cases.

## Database

Amazon RDS. Managed relational databases and supported extensions.

## Developer Tools

AWS Amplify. Application development and deployment workflows.

AWS CDK. Infrastructure definitions in supported programming languages.

AWS CLI. Command-line access to AWS service APIs.

AWS CloudFormation. Declarative infrastructure provisioning.

AWS CodeArtifact. Managed software package repositories.

AWS CodeBuild. Builds, tests, and evaluation jobs in CI workflows.

## Developer Tools

AWS CodeDeploy. Supported automated application deployment patterns.

AWS CodePipeline. Continuous delivery workflow orchestration.

Kiro. AI-assisted development workflows.

AWS Tools and SDKs. Language-specific service clients, credentials, retries, and API calls.

AWS X-Ray. Distributed request traces for supported application paths.

## Machine Learning

Amazon Augmented AI. Human review workflows for supported ML predictions.

Amazon Bedrock. Managed foundation model access and GenAI application features.

Amazon Bedrock AgentCore. Components for agent runtime, tools, identity, memory, and operations.

Amazon Bedrock Knowledge Bases. Managed retrieval and generation integration over supported data stores.

Amazon Bedrock Prompt Management. Reusable prompt resources, templates, and versions.

Amazon Bedrock Prompt Flows. Visual composition of supported GenAI workflow nodes.

## Machine Learning

Amazon Comprehend. Text analysis such as entities and supported PII detection.

Amazon Kendra. Enterprise search and supported data source connectors.

Amazon Lex. Conversational bot interfaces and intent workflows.

Amazon Q Business. Enterprise assistant over supported authorized business data.

Amazon Q Business Apps. Business app creation within supported Q capabilities.

Amazon Q Developer. Developer assistance for code, AWS tasks, and diagnosis.

## Machine Learning

Amazon Quick. Business AI capabilities. Check current product documentation for exact scope.

Amazon Rekognition. Image and video analysis.

Amazon SageMaker AI. Managed workflows for model development, deployment, and operations.

Amazon SageMaker Clarify. Bias analysis and model explanation tools for supported workflows.

Amazon SageMaker Data Wrangler. Data preparation workflows for supported model development.

Amazon SageMaker Ground Truth. Data labeling workflows.

## Machine Learning

Amazon SageMaker JumpStart. Prebuilt solutions and supported model deployment resources.

Amazon SageMaker Model Monitor. Monitoring of supported deployed model quality and data conditions.

Amazon SageMaker Model Registry. Versioned model packages and approval metadata.

Amazon SageMaker Neo. Model compilation for supported target devices.

Amazon SageMaker Processing. Managed data processing and evaluation workloads.

Amazon SageMaker Unified Studio. Integrated environment for supported analytics and AI work.

## Machine Learning

Amazon Textract. Document text, form, and table extraction.

Amazon Titan. Amazon model family with supported text, image, and embedding variants.



Amazon Transcribe. Speech-to-text transcription.

## Management and Governance

AWS Auto Scaling. Scaling policies for supported resources.

AWS Chatbot. Chat-based operational integrations. Product names can change.

AWS CloudTrail. Supported API audit events and identity history.

Amazon CloudWatch. Metrics, alarms, dashboards, and operational signals.

Amazon CloudWatch Logs. Application and service log collection and queries.

Amazon CloudWatch Synthetics. Synthetic probes for application workflows.

## Management and Governance

AWS Cost Anomaly Detection. Detection of unusual AWS cost patterns.

AWS Cost Explorer. AWS cost and usage analysis.

Amazon Managed Grafana. Managed operational visualization workspace.

AWS Service Catalog. Governed portfolios of approved provisioned products.

AWS Systems Manager. Supported operational management and configuration tools.

AWS Well-Architected Tool. Workload reviews and supported lenses.

## Migration and Transfer

AWS DataSync. Managed transfer of data between supported storage locations.

AWS Transfer Family. Managed file transfer protocols and workflows.

## Networking and Content Delivery

Amazon API Gateway. Managed REST, HTTP, and WebSocket API interfaces.

AWS AppSync. Managed application APIs with supported real-time capabilities.

Amazon CloudFront. Content delivery and permitted edge caching.

Elastic Load Balancing (ELB). Traffic distribution to healthy application targets.

AWS Global Accelerator. Global routing to supported regional application endpoints.

AWS PrivateLink. Private service access through supported endpoints.

## Networking and Content Delivery

Amazon Route 53. DNS and supported routing and health-check policies.

Amazon VPC. Network boundaries, subnets, routes, and traffic controls.

### Security, Identity, and Compliance

Amazon Cognito. Application user identity and federation.

AWS Encryption SDK. Client-side encryption library for application data.

IAM. Identity policies, roles, and resource authorization.

IAM Access Analyzer. Supported access analysis and policy validation.

IAM Identity Center. Workforce access and federation to supported accounts and applications.

AWS KMS. Managed encryption keys and key use policies.

### Security, Identity, and Compliance

Amazon Macie. Sensitive data discovery and security findings for S3.

AWS Secrets Manager. Secret storage, access, and supported rotation.

AWS WAF. Web request filters and application protection rules.

### Storage

Amazon EBS. Block storage for supported compute workloads.

Amazon EFS. Shared file storage for supported applications.

Amazon S3. Object sources, datasets, outputs, and protected log storage.

Amazon S3 Intelligent-Tiering. Automated storage tier access optimization under its rules.

Amazon S3 Lifecycle policies. Object transition and expiration rules.

Amazon S3 Cross-Region Replication. Replication of eligible objects to another Region.

## Corrections to inherited material

#### Model access and AccessDenied

Many models now use default access with prerequisites. AccessDenied can reflect IAM, SCP, endpoint, Region, subscription, or account restrictions. Do not assume missing console activation.

Sources [19, 5].

#### Temperature zero

Temperature zero does not guarantee universal deterministic output or factual accuracy. Evaluate the complete model configuration.

Sources [17].

#### Converse feature support

Converse standardizes a supported interface. Tool use, structured output, streaming, and other features remain model-specific. Validate business meaning after schema validation.



Sources [3, 43].

#### Audit events and model bodies

CloudTrail audit coverage and model invocation body logs are separate. Configure required event and invocation logging. Protect sensitive content and retention.

Sources [39, 30].

#### Architecture heuristics

Always choose managed, always use RAG, and similar rules are source heuristics. Select the option that meets every explicit requirement.

Sources [40].

#### Cache TTL and price

Cache TTL, token thresholds, pricing, and supported APIs vary by model. Example prices are illustrative unless a current source establishes otherwise.

Sources [37].

#### Cross-Region residency

Check every eligible destination and required permission. A cross-Region profile does not preserve single-Region residency by default.

Sources [9].

#### Guardrails limits

Guardrails do not replace data authorization, safe SQL execution, or tool validation. Automated Reasoning findings require application action and only cover the modeled policy.

Sources [34, 45].

#### Practice and score limits

These are repository practice questions, not verified AWS exam items. The mock reuses bank questions. Its raw percentage does not predict the AWS scaled score.

Sources [1].

## Readiness evidence

Confidence must come from unseen decisions and observable evidence. Familiarity with these pages alone does not show readiness.

1. Explain the model, retrieval, identity, safety, and operations path without notes.

2. Predict the effect of one configuration change before you run the experiment.

3. Explain why every rejected option fails a stated requirement.

4. Repeat the test with a new scenario and different wording.

5. Use the official AWS practice set and practice exam to obtain independent evidence.

6. Delay any claim of practical competence until you execute and inspect representative labs.

The official practice set is available through AWS Skill Builder. Its question bodies were not accessible in the public documentation fetch. They are not reproduced or claimed as audited here.

https://skillbuilder.aws/category/exam-prep/generative-ai-developer-professional-AIP-C01

The HTML edition retains the complete source teaching archive. The print edition contains the authored course, skill cases, labs, and service reference. Print the question booklet separately.

# Official source index


[1] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01.html

[2] https://docs.aws.amazon.com/bedrock/latest/APIReference/API_Operations_Amazon_Bedrock_Runtime.html

[3] https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_Converse.html

[4] https://docs.aws.amazon.com/bedrock/latest/userguide/troubleshooting-api-error-codes.html

[5] https://docs.aws.amazon.com/bedrock/latest/userguide/security_iam_id-based-policy-examples.html

[6] https://docs.aws.amazon.com/bedrock/latest/userguide/data-protection.html

[7] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01-domain2.html

[8] https://docs.aws.amazon.com/bedrock/latest/userguide/vpc-interface-endpoints.html

[9] https://docs.aws.amazon.com/bedrock/latest/userguide/cross-region-inference.html

[10] https://docs.aws.amazon.com/bedrock/latest/userguide/kb-data-source-sync-ingest.html

[11] https://docs.aws.amazon.com/bedrock/latest/userguide/data-retention.html

[12] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01-domain1.html

[13] https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html

[14] https://docs.aws.amazon.com/bedrock/latest/userguide/batch-inference.html

[15] https://docs.aws.amazon.com/bedrock/latest/userguide/custom-models.html

[16] https://docs.aws.amazon.com/bedrock/latest/userguide/kb-test-config.html

[17] https://docs.aws.amazon.com/bedrock/latest/userguide/inference-parameters.html

[18] https://docs.aws.amazon.com/bedrock/latest/userguide/what-is-bedrock.html

[19] https://docs.aws.amazon.com/bedrock/latest/userguide/model-access.html

[20] https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html

[21] https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-evaluation-metrics.html

[22] https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html

[23] https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-it-works.html

[24] https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_Retrieve.html





[25] https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-management.html

[26] https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html

[27] https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html

[28] https://docs.aws.amazon.com/bedrock/latest/userguide/evaluation.html

[29] https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-model.html

[30] https://docs.aws.amazon.com/bedrock/latest/userguide/model-invocation-logging.html

[31] https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ConverseStream.html

[32] https://docs.aws.amazon.com/apigateway/latest/developerguide/response-transfer-mode.html

[33] https://docs.aws.amazon.com/apigateway/latest/developerguide/response-streaming.html

[34] https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html

[35] https://docs.aws.amazon.com/bedrock/latest/APIReference/API_runtime_ApplyGuardrail.html

[36] https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html

[37] https://docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html

[38] https://docs.aws.amazon.com/bedrock/latest/userguide/prov-throughput.html

[39] https://docs.aws.amazon.com/bedrock/latest/userguide/logging-using-cloudtrail.html

[40] https://docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens/generative-ai-lens.html

[41] https://docs.aws.amazon.com/bedrock/latest/userguide/bda.html

[42] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/aip-01-in-scope-services.html

[43] https://docs.aws.amazon.com/bedrock/latest/userguide/structured-output.html

[44] https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html

[45] https://docs.aws.amazon.com/bedrock/latest/userguide/integrate-automated-reasoning-checks.html

[46] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01-domain3.html

[47] https://docs.aws.amazon.com/bedrock/latest/userguide/agents.html

[48] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01-domain4.html

[49] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01-domain5.html

[50] https://aws.amazon.com/certification/certified-generative-ai-developer-professional/

[51] https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html

[52] https://docs.aws.amazon.com/bedrock/latest/APIReference/API_agent-runtime_RetrieveAndGenerate.html

[53] https://docs.aws.amazon.com/prescriptive-guidance/latest/gen-ai-inference-architecture-and-best-practices-on-aws/amazon-bedrock.html

[54] https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html

[55] https://github.com/jrajath94/aws-aip-c01-study-guide/blob/823006efa64f98b560dd4fe40445e6a8042ef79b/volume-d1-foundation-models.html

[56] https://docs.aws.amazon.com/aws-certification/latest/ai-professional-01/ai-professional-01-technologies-concepts.html

[57] https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-serverless/grounding-and-rag.html

[58] https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base-create.html

[59] https://docs.aws.amazon.com/bedrock/latest/userguide/trace-events.html

[60] https://github.com/jrajath94/aws-aip-c01-study-guide/blob/823006efa64f98b560dd4fe40445e6a8042ef79b/prereq-aif-c01.html

[61] https://github.com/jrajath94/aws-aip-c01-study-guide/blob/823006efa64f98b560dd4fe40445e6a8042ef79b/prereq-aws-cloud-foundations.html

[62] https://github.com/jrajath94/aws-aip-c01-study-guide/blob/823006efa64f98b560dd4fe40445e6a8042ef79b/prereq-ml-specialty.html